CSE 209 — Rifat Sir (RS)

Term-Final question bank with solutions

81 questions from the 2019-20, 2020-21, 2021-22, 2022-23 and 2023-24 papers (the last five term finals), grouped by the practice-tracker tabs and lessons. Each entry has a header line from the tracker (number, exam year, question number, marks, priority, type, lesson), followed by the question and its worked solution. Where the same question was asked in several papers, the stacked cards share one solution and later entries point back to it.

Priority: every question here is High (2019-20 onward); Medium and Low apply only to older papers.

Tab	Questions
1 Ch4 P1 Single-Cycle	15
2 Ch4 P2 Pipelining	27
3 Ch5 Memory	30
4 GPU + Integration	6
Outside Notes	3

Solutions are taken from the CSE 209 Computer Architecture Topic-wise Solved Questions set (credited in that book to Mahdi Hasan, written with Claude Opus 5.5). Check any answer against your lecture notes and the note PDFs in the notes folder before relying on it in the exam.

1 Ch4 P1 Single-Cycle — Chapter 4 Part 1 — Single-Cycle Datapath & Control

Notes: CSE209_Ch4_SingleCycle_Notes.pdf

1. 2023-24 · Q8(a)(i) · 5 marks · ★★★ High · Theory · L2

2023–2024 • Q8(a) CSE 209 • 29/04/2026 • DP02 **MARKS 20**

A single cycle datapath implementation of a subset of 32-bit MIPS instruction set has been shown in Figure for Question 8(a). Answer the following:

(i) Why is PC incremented by 4?

(ii) How is the length of the clock cycle determined in a single cycle execution?

(iii) Why is the upper 4 bits of PC + 4 concatenated in jump address calculation?

(iv) "A single physical memory unit can be used to implement both instruction memory and data memory in a hardware based implementation of a single cycle datapath" – agree or disagree? Justify your answer.

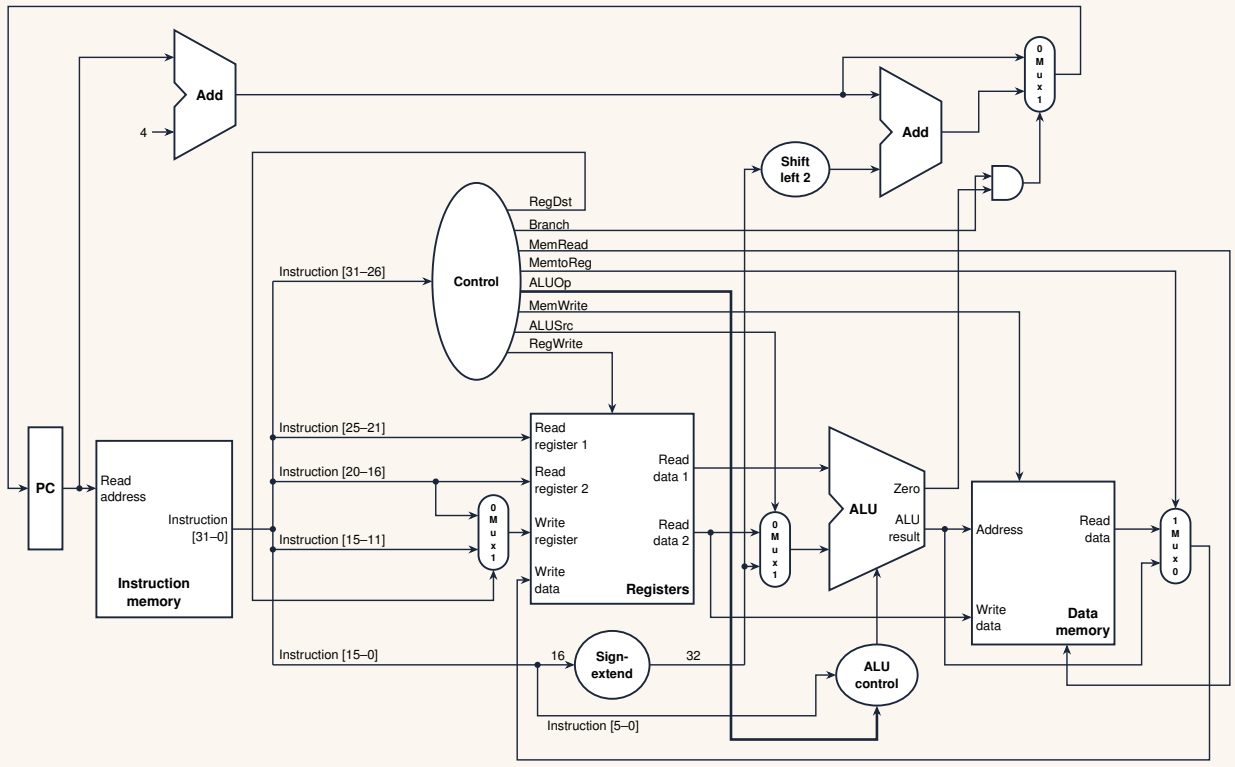


Figure for Question 8(a): Single cycle datapath with control signals

SOLUTION

(i) PC + 4. MIPS memory is *byte addressed*, and every instruction is exactly 32 bits = 4 bytes, stored word-aligned. The next sequential instruction therefore starts 4 byte-addresses later, so the PC adder adds 4. Branch offsets are also counted from PC+4.

(ii) Clock cycle length. Each instruction completes in one cycle, so the period must cover the *longest* (critical) combinational path from one state element to the next, plus register setup

and clock-to-Q times. That path belongs to `lw`:

$$T_{clk} \geq t_{\text{I-mem}} + t_{\text{reg read}} + t_{\text{ALU}} + t_{\text{D-mem}} + t_{\text{mux}} + t_{\text{reg setup}}.$$

For example, with 200/100/200/200/100 ps stages this gives 800 ps. Every instruction, even a fast `add` or `j`, then takes 800 ps.

(iii) Upper 4 bits of PC+4 in jumps. The J-format has only 26 bits for the target. Since instructions are word aligned, the two low bits are always 00, so Instr[25–0]$\ll$ 2 gives 28 bits. The missing 4 high bits are taken from PC+4 (pseudo-direct addressing): JTA $= \{(\text{PC}+4)[31{:}28], \text{Instr}[25{:}0], 00\}$. This gives a full 32-bit address, and the jump stays inside the current 256 MB region. PC+4 is used rather than PC because the incremented value is already available, and the region is defined relative to the next instruction.

(iv) Disagree. In a single-cycle datapath every functional unit can be used only *once per clock cycle*. A `lw` or `sw` must, in the same cycle, (1) fetch its own instruction and (2) read or write a data word. A single-ported memory cannot do both at once, so there is a structural conflict. That is why the single-cycle design has *separate* instruction and data memories, just as it has separate adders for PC+4 and the branch target. A single unified memory works in a *multicycle* design, where fetch and data access happen in different cycles. A true dual-ported memory would really be two access paths, i.e. logically two memories.

2. 2019-20 · Q8(a) · 15 marks · ★★★ High · Trace · L3

2019–2020 • Q8(a) CSE 305 • 18/09/2021 • DP08 **MARKS 5×3 = 15**

Figure 8a shows a single cycle datapath implementation of a subset of MIPS instruction set. For each of the following three instructions: **add**, **sw**, **beq**, determine the values (0/1) of the following control signals shown in Figure 8a.

RegDst, Branch, MemRead, MemtoReg, MemWrite, ALUSrc, RegWrite

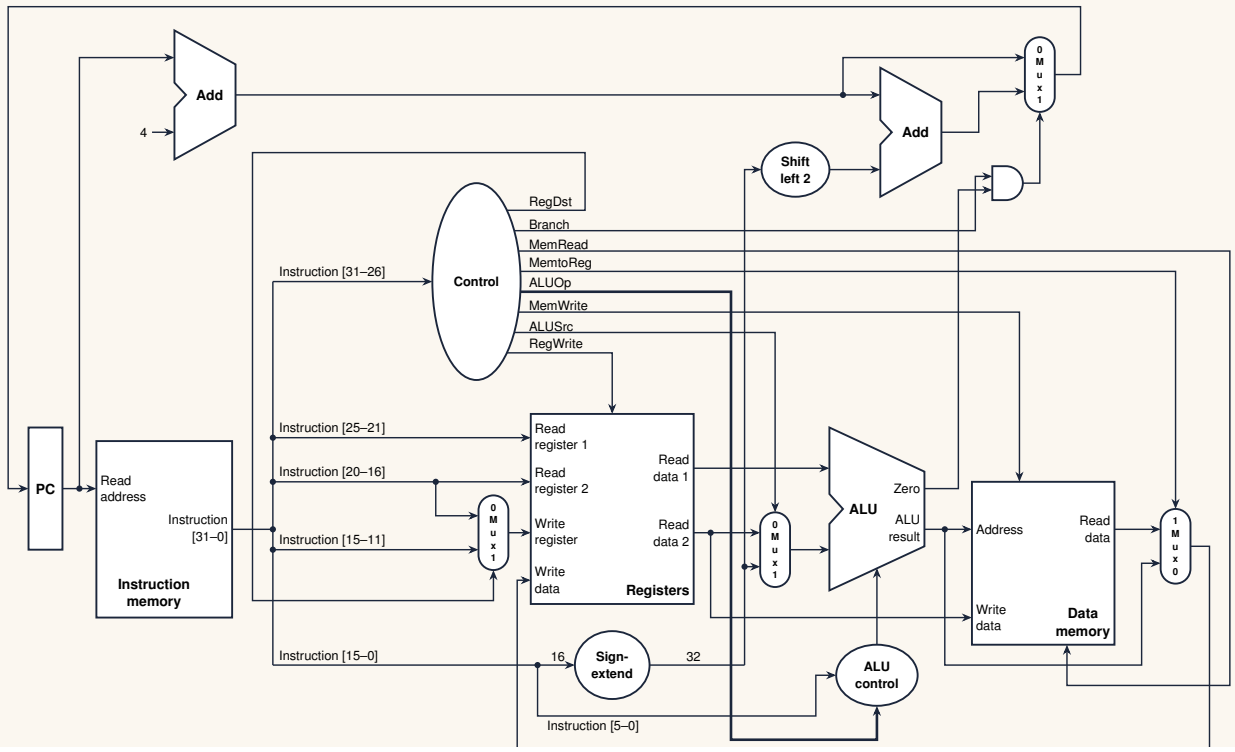


Figure 8a: Single cycle datapath with control signals

SOLUTION

- `add` (R-type): writes rd (RegDst = 1, RegWrite = 1). The second operand is Read data 2 (ALUSrc = 0), the result comes from the ALU (MemtoReg = 0), and there is no memory access and no branch.
- `sw`: the address is rs + offset (ALUSrc = 1). It writes memory (MemWrite = 1) and *no* register (RegWrite = 0), so RegDst and MemtoReg are don't-care.
- `beq`: compares rs and rt by subtraction (ALUSrc = 0, ALUOp = 01) with Branch = 1, and writes neither memory nor a register.

Instr.	RegDst	Branch	MemRead	MemtoReg	MemWrite	ALUSrc	RegWrite
`add`	1	0	0	0	0	0	1
`sw`	X (0)	0	0	X (0)	1	1	0
`beq`	X (0)	1	0	X (0)	0	0	0

X = don't care: the instruction writes no register, so the value is irrelevant. Write 0 if a strict 0/1 answer is required. For completeness, ALUOp = 10 (add), 00 (sw), 01 (beq).

3. 2020-21 · Q3 · 15 marks · ★★★ High · Trace · L4

2020–2021 • Q3 CSE 305 • 24/09/2022 • DP07 **MARKS 15**

A full datapath with a control unit is given in Figure 1. Highlight the datapath elements that will be active for the `ori $t1, 100($t2)` instruction. Also, mention which control bits will be set. [You may highlight the given figure and attach it to your answer script.]

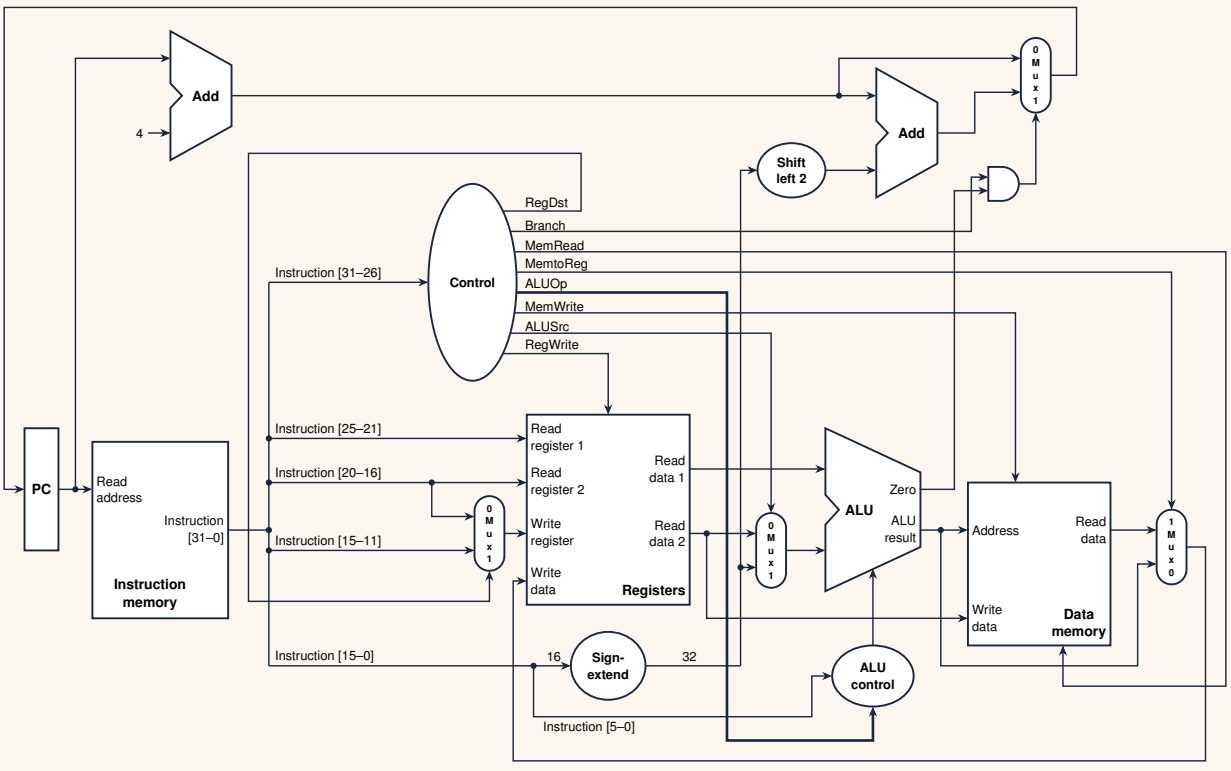
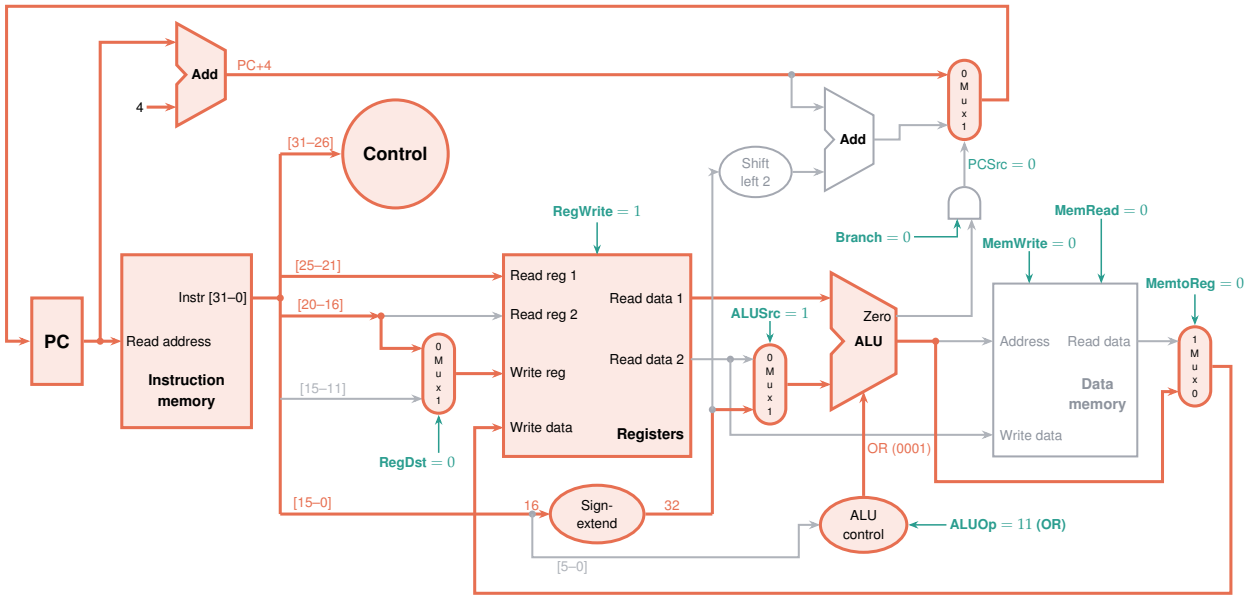


Figure 1 (missing from the scan; the standard single-cycle datapath with control unit is shown)

SOLUTION

NOTE Figure 1 is not in the scan. It is taken to be the standard single-cycle MIPS datapath with control (as in the slides), redrawn below. The operand syntax `100($t2)` is unusual for `ori`, which is a logical-immediate I-type instruction. It is read as `ori $t1, $t2, 100`, i.e. `$t1` ← `$t2` OR 100: rs = `$t2`, rt = `$t1`, immediate = 100.

What `ori` needs. Fetch and PC+4. Read rs (`$t2`). Feed the immediate into the ALU's second input (ALUSrc = 1). The ALU does *OR*. Write the ALU result (not memory data) into rt (`$t1`) (RegDst = 0, MemtoReg = 0, RegWrite = 1). Data memory and the branch path are *not* used.



Coral = active elements and paths for `ori`. Grey = idle (the data memory, the branch adder/shift path, the rt read port's data and the [15–11] field). Teal = control values.

Control bits

RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp	ALU control
0	1	0	**1**	0	0	0	11	0001 (OR)

Answer. Set (=1): **ALUSrc** and **RegWrite**. All the others are 0 (RegDst, MemtoReg, MemRead, MemWrite, Branch/PCSrc). The ALU must perform OR. The basic 2-bit ALUOp codes (00 add, 01 sub, 10 funct) cannot ask for OR without a funct field, so a new code, ALUOp = 11 → OR, is assumed.

NOTE Strictly, `ori` *zero*-extends its immediate. With the standard sign-extend unit the result is still correct here, because 100 is positive (bit 15 = 0). A full implementation adds a zero-extend option, chosen by an *ExtOp* control bit.

4. 2022-23 · Q4(a) · 16 marks · ★★★ High · Design · L5

2022–2023 • Q4(a) CSE 305 • 25/04/2024 • DP05 **MARKS 16**

Consider the following branch type instruction format given in Figure for Question 4(a) assuming 32 bit addressing. Construct the formula to calculate branch target address. Draw the single cycle datapath for branch-on-equal instruction assuming 32 bit addressing. [Include only the relevant components and control lines in your design. For example, you can exclude data memory from your design.]

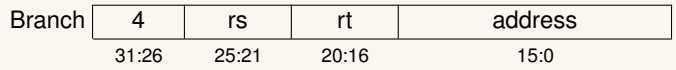


Figure for Question 4(a): Branch Instruction Format in MIPS

SOLUTION

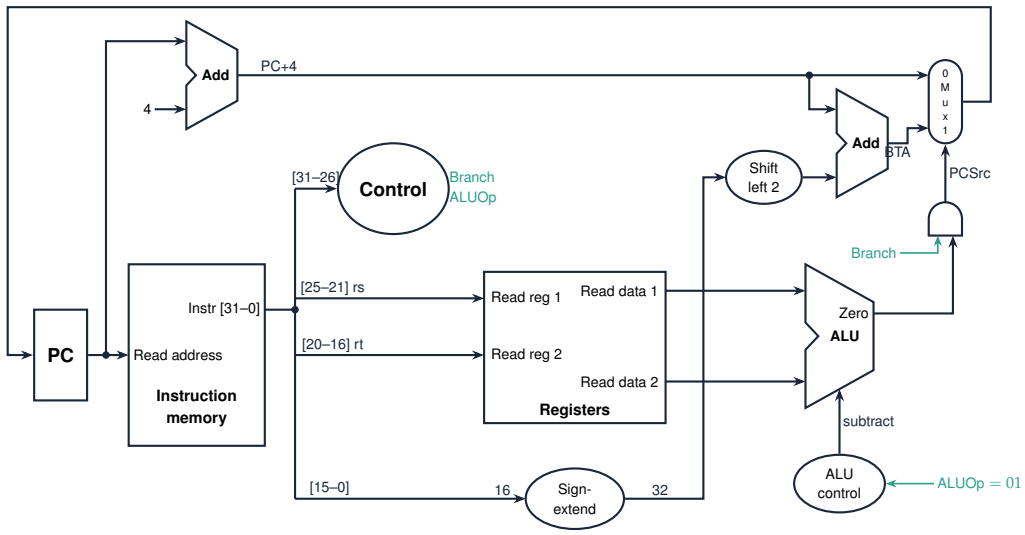
Branch target address

The 16-bit *address* field is a signed *word* offset relative to the next instruction:

$$\text{BTA} = (\text{PC} + 4) + \left(\text{SignExt}_{32}(\text{Instr}[15{:}0]) \ll 2\right) = \text{PC} + 4 + 4 \times \text{offset}.$$

The reach is $-2^{15} \ldots 2^{15} - 1$ words from PC+4.

Single-cycle datapath for beq (relevant parts only)



- **Fetch:** PC addresses the instruction memory, and an adder forms PC+4.
- **Compare:** rs and rt are read. The ALU subtracts them (Control: ALUOp $= 01 \rightarrow$ ALU control $=$ subtract), and *Zero* $= 1$ if they are equal.
- **Target:** Instr[15–0] is sign-extended to 32 bits, shifted left 2, and added to PC+4 by a separate adder (BTA).
- **Select:** PCSrc $=$ Branch $\cdot$ Zero picks BTA, otherwise PC+4. For `beq`: Branch $= 1$, RegWrite $=$ MemWrite $=$ MemRead $= 0$, ALUSrc $= 0$.

5. 2023-24 · Q8(a)(iii) · 5 marks · ★★★ High · Theory · L6

Same question - solution already given under #1.

6. 2023-24 · Q8(b) · 10 marks · ★★★ High · Design · L6

2023–2024 • Q8(b) CSE 209 • 29/04/2026 • DP03 **MARKS 10**

Identify the component(s) required to be added to implement the jump instruction in the Figure for Question 8(a). Draw the diagram in your answer script implementing jump instruction assuming 32 bit addressing. [Include only the relevant components and control lines to implement jump instruction in your drawing.]

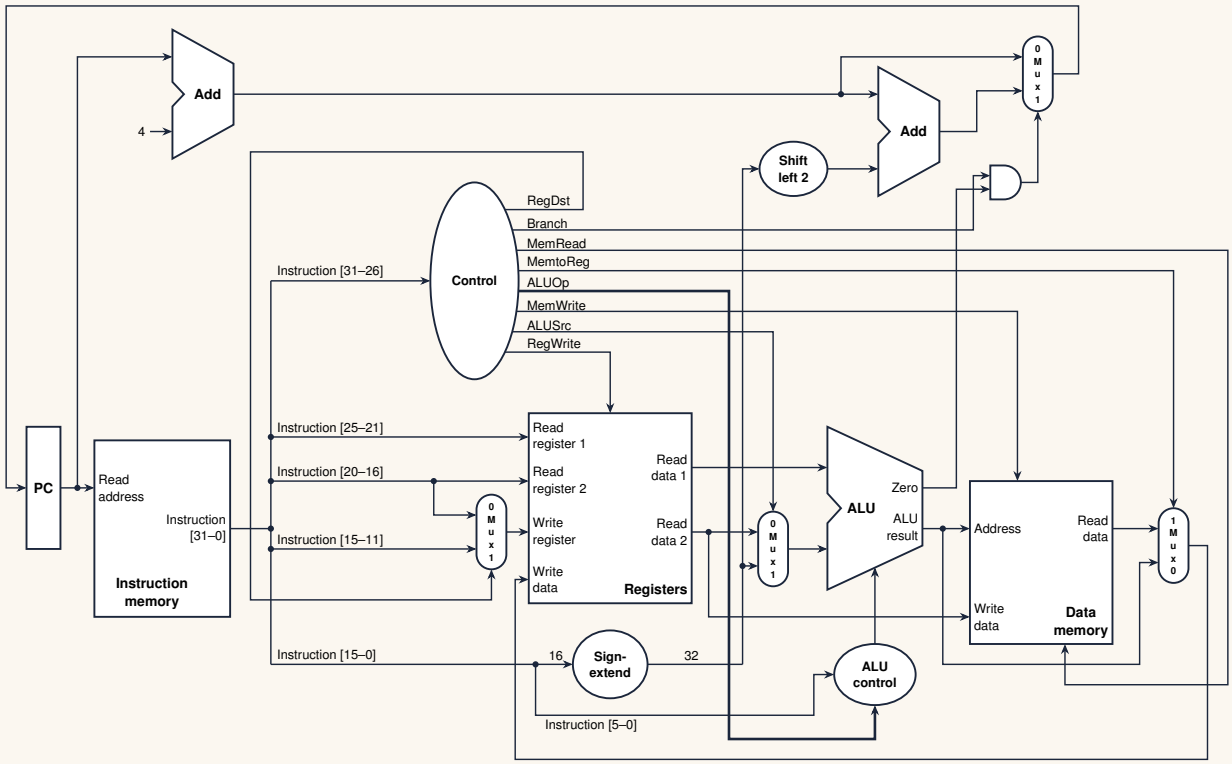


Figure for Question 8(a): Single cycle datapath with control signals

SOLUTION

Jump target (pseudo-direct addressing): JTA $= \{(\text{PC}+4)[31{:}28],\ \text{Instr}[25{:}0],\ 00\}$, i.e. a 26-bit word address shifted left by 2 and topped up with the 4 high bits of PC+4 to make 32 bits.

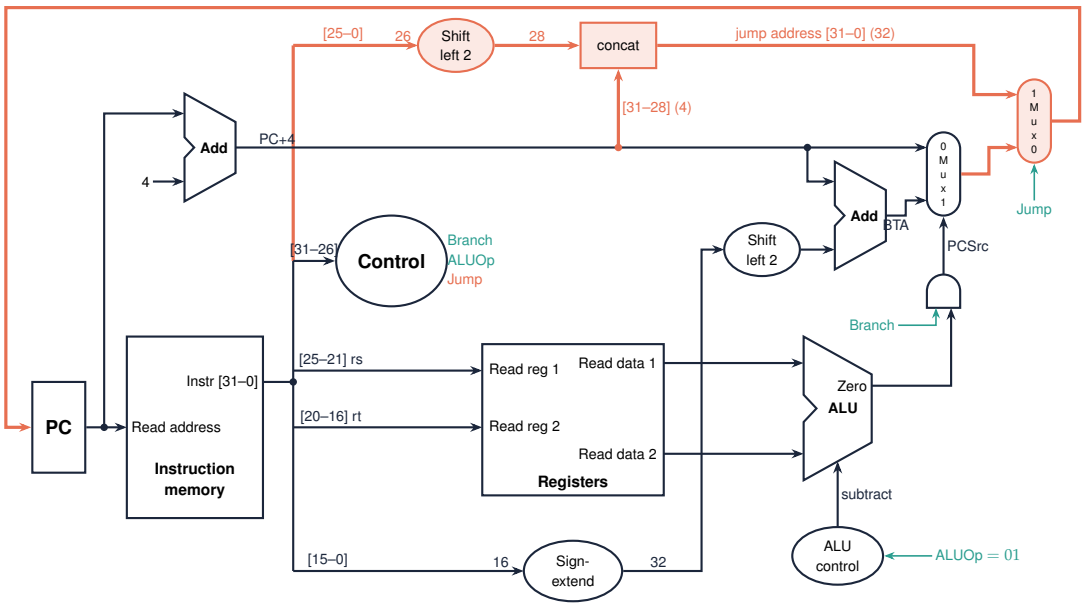
Components to add:

(i) a *shift-left-2* on Instr[25–0] (26 → 28 bits; it is only wiring);

(ii) a *concatenation* of PC+4[31–28] with those 28 bits, giving the 32-bit jump address;

(iii) one more *2-to-1 mux* in front of the PC, choosing between the existing branch-mux output (PC+4 or BTA) and the jump address;

(iv) a new main-control output **Jump**, which is 1 only for opcode 000010 (for `j`, RegWrite = MemWrite = 0).



Coral = the added jump hardware. Black = existing relevant parts (fetch, PC+4, branch mux).

7. 2022-23 · Q12(c)(ii) · 3 marks · ★★★ High · Numerical · L6

2022–2023 • Q12(c) CSE 209 • 13/01/2025 • IS08 **MARKS 10**

(i) Write the MIPS assembly code that creates the 32-bit constant `0010 0000 0000 0001 0100 1001 0010 0100`$_2$ and stores that value to register `$t1`.

(ii) If the current value of the PC is `0x00000000` can you use a single jump instruction to go to the address equal to the constant shown in question 12(c)(i)?

(iii) If the current value of the PC is `0x1FFFF000`, can you use a single branch instruction to go to the address equal to the constant shown in Question 12(c)(i)?

SOLUTION

Facts used

- 32-bit constant: `lui` loads the upper 16 bits (and clears the lower half). Then `ori` ORs in the lower 16 bits. `ori` zero-extends its immediate, so the upper half is not disturbed.
- `j` (pseudodirect): target = $(\text{PC}+4)_{31..28} : \text{addr}_{26} : 00$. It can only reach addresses in the same 256 MB region, i.e. with the same upper 4 bits.
- `beq`/`bne` (PC-relative): target = $\text{PC}+4+\text{SignExt}(\text{off}_{16})\times 4$ with $-32768 \le \text{off} \le 32767$.

(i) `0010 0000 0000 0001 | 0100 1001 0010 0100` = `0x2001 4924`. The upper half is `0x2001` = 8193 and the lower half is `0x4924` = 18724.

```
lui  $t1, 0x2001          # $t1 = 0x2001 0000
ori  $t1, $t1, 0x4924     # $t1 = 0x2001 4924
```

	upper 16 (from `lui`)	
`lui $t1, 0x2001`	`0010 0000 0000 0001`	`0000 0000 0000 0000`
`ori $t1, $t1, 0x4924`	`0010 0000 0000 0001`	`0100 1001 0010 0100`

(ii) PC = `0x00000000`, so PC+4 = `0x00000004`, whose upper 4 bits are `0000`. The target `0x20014924` has upper 4 bits `0010`. A `j` can only produce addresses whose top 4 bits are `0000`, so **no**. (The target is word-aligned; the region is the problem.) Instead use the `lui/ori` of (i) followed by `jr $t1`.
(iii) PC+4 = `0x1FFFF004`.

$$\text{offset} = \frac{\texttt{0x20014924} - \texttt{0x1FFFF004}}{4} = \frac{\texttt{0x15920}}{4} = \frac{88352}{4} = 22088\ (\texttt{0x5648}) \le 2^{15} - 1 = 32767.$$

So **yes**: a single `beq` with offset field 22088 reaches it. (A `j` would fail here, because the upper bits `0001` and `0010` differ.)

Answer. (i) `lui $t1,0x2001; ori $t1,$t1,0x4924`. (ii) No: the jump cannot change the upper 4 bits from 0000 to 0010. (iii) Yes: the offset is 22088, which is at most 32767.

8. 2021-22 · Q1(a) · 10 marks · ★★★ High · Design · L6

2021–2022 • Q1(a) CSE 305 • 06/04/2023 • DP06 **MARKS 10**

Complete the provided incomplete diagram of the MIPS single cycle datapath for the branch if equal (`beq`) and jump (`j`) instructions.

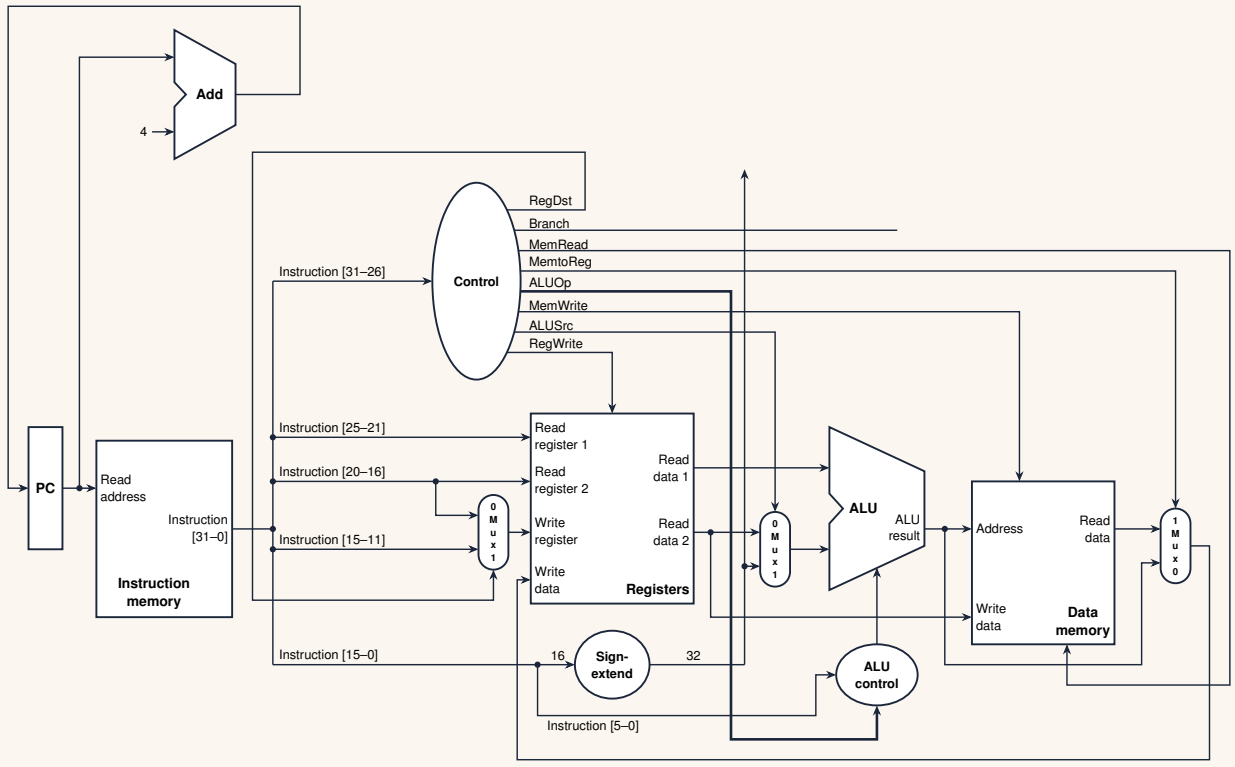
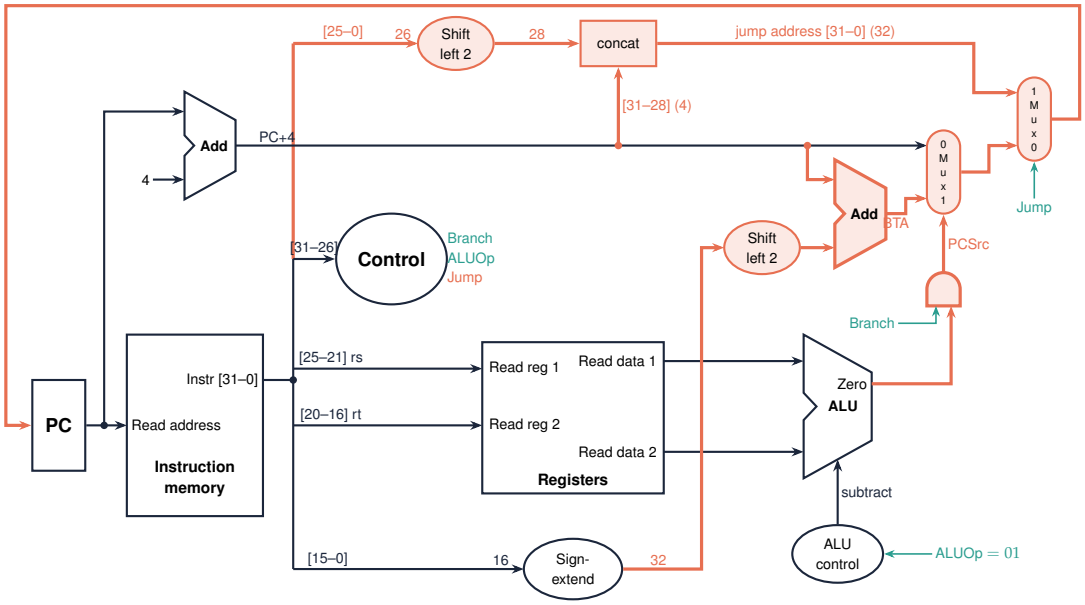


Figure for Question 1(a)

SOLUTION

Target addresses. BTA = (PC + 4) + (SignExt(Instr[15:0]) ≪ 2) and JTA = {(PC + 4)[31:28], Instr[25:0], 00}.
The completed PC-update part is drawn below (added hardware in coral). The rest of the given figure (data memory, write-back mux, RegDst and ALUSrc muxes) is unchanged and omitted for

clarity.



- **beq**: add a shift-left-2 on the sign-extended offset, an adder (PC+4 + offset×4), an AND gate (Branch · Zero = PCSrc) and a 2:1 mux (PC+4 / BTA). The ALU subtracts (ALUOp = 01) to produce Zero.
- **j**: add a shift-left-2 on Instr[25–0], a concatenation with PC+4[31–28], and a second 2:1 mux driven by a new control output *Jump*.

Instr.	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp	Jump
`beq`	X	0	X	0	0	0	1	01	0
`j`	X	X	X	0	0	0	0	XX	1

9. 2021-22 · Q6(b)(i) · 5 marks · ★★★ High · Numerical · L6

2021–2022 • Q6(b) CSE 305 • 06/04/2023 • IS17 **MARKS 5 + 5**

Suppose the program counter (PC) is set to `0x2000 0000`. Explain your answer for the following questions.

(i) Is it possible to use the jump (`j`) MIPS assembly instruction to set the PC to the address `0x4000 0000`?

(ii) Is it possible to use single branch (`beq/bneq`) instruction to set the PC to this same address?

SOLUTION

Facts used

- `j` (pseudodirect, J-format): target = $(\text{PC}+4)_{31..28} : \text{addr}_{26} : 00$. Only the low 28 bits come from the instruction; the upper 4 bits are copied from PC+4.
- `beq/bne` (PC-relative, I-format): target = $\text{PC}+4+\text{SignExt}(\text{off}_{16})\times 4$. The reach is $-2^{17} \ldots +2^{17} - 4$ bytes ($\approx \pm 128$ KB).

(i) PC+4 = `0x20000004`, upper 4 bits `0010`. The target `0x40000000` has upper 4 bits `0100`. These

differ, so `j` **cannot** reach it: the target is in another 256 MB region.
(ii) The required distance is `0x40000000`−`0x20000004` = `0x1FFFFFFC` = 536,870,908 bytes, i.e. $\approx 1.3 \times 10^8$ words. That is far beyond the 16-bit offset limit of 32767 words. So **no**.
How to do it: load the full address into a register and use `jr`:

```
lui  $t0, 0x4000          # $t0 = 0x4000 0000
jr   $t0                  # PC = 0x4000 0000
```

Answer. (i) No: `j` keeps $PC_{31..28} = 0010$, but the target needs 0100. (ii) No: the distance `0x1FFFFFFC` bytes far exceeds the $\pm 2^{17}$-byte branch range. Use `lui $t0,0x4000; jr $t0`.

10. 2023-24 · Q8(a)(ii) · 5 marks · ★★★ High · Theory · L7

Same question - solution already given under #1.

11. 2023-24 · Q8(a)(iv) · 5 marks · ★★★ High · Theory · L7

Same question - solution already given under #1.

12. 2023-24 · Q5(a)(i) · 10 marks · ★★★ High · Design · L8B

2023–2024 • Q5(a) CSE 209 • 29/04/2026 • DP01 **MARKS 20**

Consider a modified MIPS instruction set, called MIPS-v22, where each instruction is represented in 32-bits. Figure for Question 5(a) shows the format of R-type and I-type instruction of MIPS-v22.

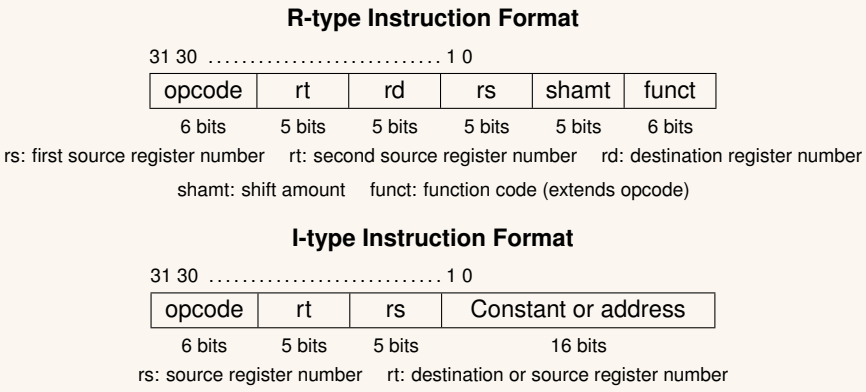


Figure for Question 5(a): R-type and I-type Instruction Formats for MIPS-v22

(i) Develop a block diagram of the partial single cycle datapath for R-type and I-type instructions of MIPS-v22 containing only an Instruction Memory, a Register file, and one or more MUXs (as required). Note that the output of the Instruction memory is a 32 bit MIPS-v22 instruction. Your task is to devise only connections from the Instruction Memory to the following 3 inputs of the Register File: Read register 1 number, Read register 2 number, Write register number. You may use as many MUXs as required. Clearly show the bit size of the connections.

(ii) Consider the numbers of the registers `$t0`, `$t1`, and `$t2` are 2, 3, 4 in decimal respectively. Translate the following 3 instructions in the binary format following the given MIPS-v22 instruction format. Assume shamt = 0_{10}, and funct = 0_{10}.

- `add $t0, $t1, $t2` Here `$t0`: destination, `$t1`: source 1, `$t2`: source 2, opcode = 0_{10}
- `addi $t2, $t0, 1` Here `$t2`: destination, `$t0`: source 1, opcode = 6_{10}

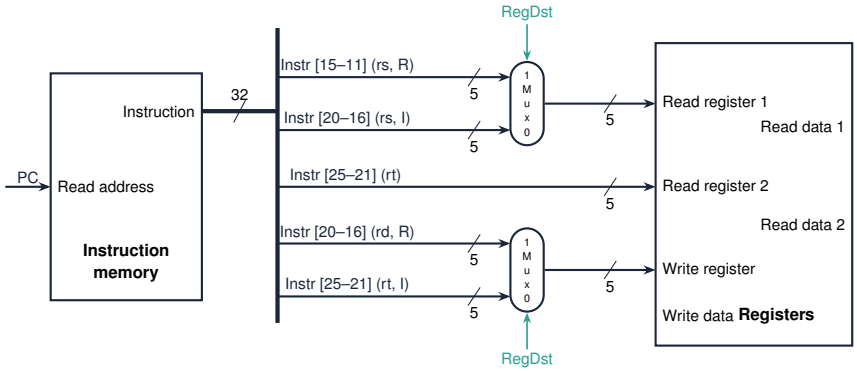
- `lw $t1, 4($t0)` Here `$t1`: destination, `$t0`: base address, opcode = 5_{10}

SOLUTION

(i) Where each register number sits

RF input	R-type field	I-type field	Hardware
Read register 1 (first source, rs)	[15–11]	[20–16]	2-to-1 mux
Read register 2 (second source, rt)	[25–21]	[25–21] (rt, for `sw`/`beq`)	direct wire
Write register (destination)	[20–16] (rd)	[25–21] (rt)	2-to-1 mux

Two 5-bit 2-to-1 muxes are needed. Both can share one control line, **RegDst** (1 for R-type, 0 for I-type). Read register 2 is always Instr[25–21], so it needs no mux. Every connection from the 32-bit instruction is a 5-bit field.



Opcode-based control: for R-type (opcode 0) RegDst = 1, so the muxes pass rs = [15–11] and rd = [20–16]. For I-type RegDst = 0, so they pass rs = [20–16] and rt = [25–21].

(ii) Encodings (`$t0=2, $t1=3, $t2=4`)

	opcode	rt	rd	rs	shamt	funct
`add $t0,$t1,$t2`	`000000`	`00100` (`$t2`)	`00010` (`$t0`)	`00011` (`$t1`)	`00000`	`000000`

	opcode	rt	rs	constant / address
`addi $t2,$t0,1`	`000110`	`00100` (`$t2`)	`00010` (`$t0`)	`0000 0000 0000 0001`
`lw $t1,4($t0)`	`000101`	`00011` (`$t1`)	`00010` (`$t0`)	`0000 0000 0000 0100`

Answer. add: `000000 00100 00010 00011 00000 000000` = `0x00821800`
addi: `000110 00100 00010 0000000000000001` = `0x18820001`
lw: `000101 00011 00010 0000000000000100` = `0x14620004`

13. 2022-23 · Q11(b) · 8 marks · ★★★ High · Theory · L8A

2022–2023 • Q11(b) CSE 209 • 13/01/2025 • IS06 **MARKS 8**

How does "`jal`" instruction differ from an unconditional jump instruction? How is the NOT operation performed in the MIPS assembly language?

SOLUTION

`jal` vs. `j`

	`j Label` (jump)	`jal Label` (jump and link)
Format / opcode	J-format, op = 2, 26-bit address	J-format, op = 3, 26-bit address
Target	PC ← $PC_{31..28}$: addr : 00	same target computation
Side effect	none	`$ra` (reg 31) ← PC+4, the return address
Purpose	`goto`, end of if-else, loop back-edge	**procedure call**. The callee returns with `jr $ra`

So `jal` is an unconditional jump that also *links*: it saves the address of the next instruction so control can come back to the caller. A non-leaf procedure must save `$ra` on the stack before its own `jal`, because the new `jal` overwrites it.

```
      jal  leaf        # $ra = address of next instr; PC = leaf
      ...              # execution resumes here after the return
leaf: ...
      jr   $ra         # PC = $ra  -> back to caller
```

NOT in MIPS

MIPS has no `not` instruction. It uses the 3-operand `nor`, since a NOR $0 = \overline{a + 0} = \bar{a}$ (slides: "NOT Operations"):

```
nor $t0, $t1, $zero     # $t0 = ~$t1
```

Example:

$$\begin{aligned} \texttt{\$t1} &= \quad \texttt{0000 0000 0000 0000 0011 1100 0000 0000} \\ \texttt{\$t0} = \texttt{nor(\$t1, 0)} &= \quad \texttt{1111 1111 1111 1111 1100 0011 1111 1111} \end{aligned}$$

The assembler pseudoinstruction `not $t0,$t1` expands to exactly this `nor`. `xori $t0,$t1,0xFFFF` would *not* work, because `xori` zero-extends its immediate and so inverts only the low 16 bits.

Answer. `jal` jumps like `j` but also stores PC+4 in `$ra`, so a procedure can return with `jr $ra`. `j` only changes the PC. NOT is done with `nor rd, rs, $zero`.

14. 2022-23 · Q3 · 15 marks · ★★★ High · Design · L8A

2022–2023 • Q3 CSE 209 • 13/01/2025 • DP04 **MARKS 15**

In the given incomplete MIPS single cycle data path shown in the Figure for Question 3, add new connections (no need to draw the control unit; if you need control signal(s) just naming them will suffice) to support `beq`, `bneq`, `j` and `jr` instructions using minimum numbers of MUX. You can only use 2×1 MUXs.

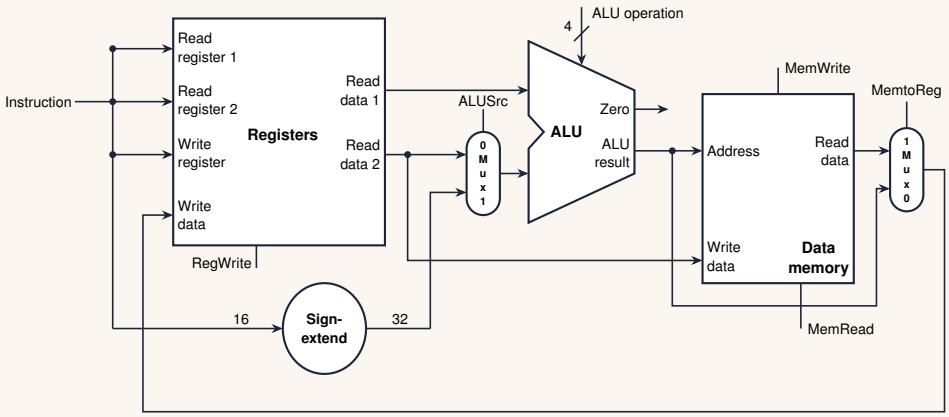


Figure for Question 3

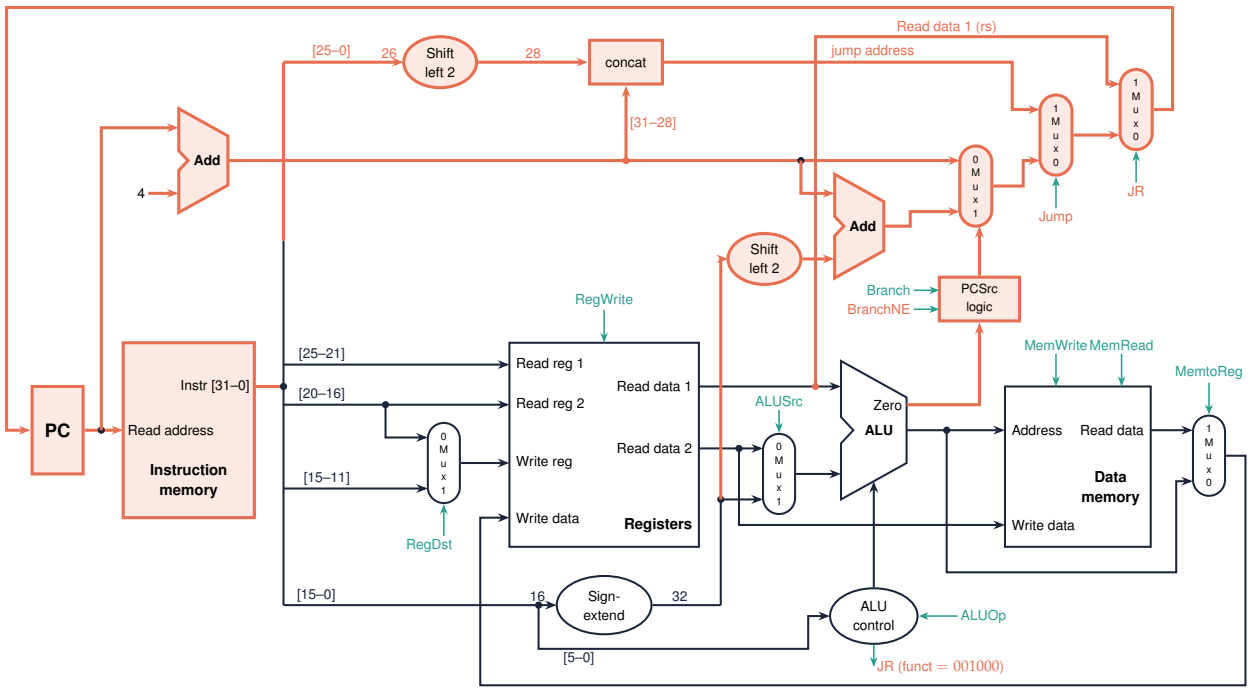
SOLUTION

What the four instructions need

- `beq rs,rt,L`: PC $\leftarrow$ BTA $=$ PC$+4+$SignExt(imm)$\times 4$ if rs $=$ rt. The ALU subtracts and gives Zero.
- `bneq rs,rt,L`: the same target, taken if rs $\neq$ rt. It shares the BTA mux; only the select logic differs.
- `j L`: PC $\leftarrow \{$PC$+4[31{:}28]$, Instr[25–0], $00\}$.

- `jr rs`: PC $\leftarrow$ Reg[rs] $=$ Read data 1.

So the PC has *four* possible sources: PC$+4$, BTA, jump address and Reg[rs]. Choosing among n sources with 2:1 muxes needs at least $n-1$ of them, so the minimum is **3 muxes**.



Black = the given partial datapath. Coral = added hardware: PC, instruction memory, PC+4 adder, shift-left-2 + branch adder, PCSrc logic, jump shift + concatenation, and the three 2:1 muxes (branch, Jump, JR).

PCSrc logic (two AND gates, one inverter, one OR gate):

$$\text{PCSrc} = \text{Branch} \cdot \text{Zero} \; + \; \text{BranchNE} \cdot \overline{\text{Zero}}.$$

The three muxes: (1) branch mux: PC+4 / BTA (PCSrc); (2) jump mux: previous / jump address (Jump); (3) JR mux: previous / Read data 1 (JR).

Instr.	Branch	BranchNE	Jump	JR	RegWrite	MemWrite	ALU op
`beq`	1	0	0	0	0	0	subtract
`bneq`	0	1	0	0	0	0	subtract
`j`	0	0	1	0	0	0	X
`jr`	0	0	X	1	0	0	X

`jr` is R-type (op = 0, funct = 8), so JR is decoded from the funct field and must force RegWrite = 0.

15. 2022-23 · Q6 · 15 marks · ★★★ High · Design · L8B

2022–2023 • Q6 CSE 209 • 13/01/2025 • IS04 **MARKS 15**

The current implementation of MIPS hardware doesn't support memory values as operand for R-type instructions. You can perform R-type instructions only if both of the operand values are stored in a register. Now you want to add a new instruction type (i.e. M-type instruction).
In M-type instruction you should perform only addition and subtraction, where one of the operand value would be stored in the memory (i.e. memory operand).
Now you need to design the M-type instruction format (i.e. which bits will represent which value) and the conversion protocol for transforming MIPS assembly to machine code for M-type instruction. You have the full freedom to design the new format and you can set any rules you wish; you can add new control signal(s) if needed. Explain your designed instruction format with proper justification. While designing your new protocol you **mustn't** hamper the working of the three original instruction formats.
Also, there will be major change(s) needed in hardware for the above-described M-type instruction. Identify the change(s) and explain them with proper justification.

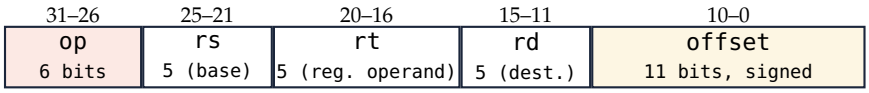
SOLUTION

1. Semantics and assembly syntax (one possible design)

$$\texttt{addm rd, rt, offset(rs)}: \quad \texttt{rd} \leftarrow \texttt{rt} + \text{Mem}[\texttt{rs} + \text{SignExt(offset)}]$$
$$\texttt{subm rd, rt, offset(rs)}: \quad \texttt{rd} \leftarrow \texttt{rt} - \text{Mem}[\texttt{rs} + \text{SignExt(offset)}]$$

This keeps the three-operand style (*simplicity favours regularity*) and uses the same `offset(base)` notation as `lw/sw`.

2. M-type format (32 bits)



Justification

- **rs and rt sit in bits 25–21 and 20–16**, as in R- and I-format, and **rd in 15–11**, as in R-format. The register file can therefore be read before decoding finishes, and the existing RegDst mux already selects rd (*good design demands good compromises*: keep formats as similar as possible).

- **add vs. sub is told apart by the opcode** (two new, currently unused opcodes, e.g. 24 = `addm`, 25 = `subm`) rather than by a funct field. This leaves $32 - 6 - 15 = 11$ bits for the offset (range $-1024 \ldots +1023$ bytes).
- **Original formats are untouched**: R (op=0), I and J instructions keep their opcodes and fields. Only the control unit learns two more opcodes.

3. Conversion protocol (assembler rules)

(i) Mnemonic → opcode: `addm`→`011000`, `subm`→`011001`.

(ii) The 1st register is rd, the 2nd is rt, and the register in parentheses is rs. Each is written as its 5-bit number.

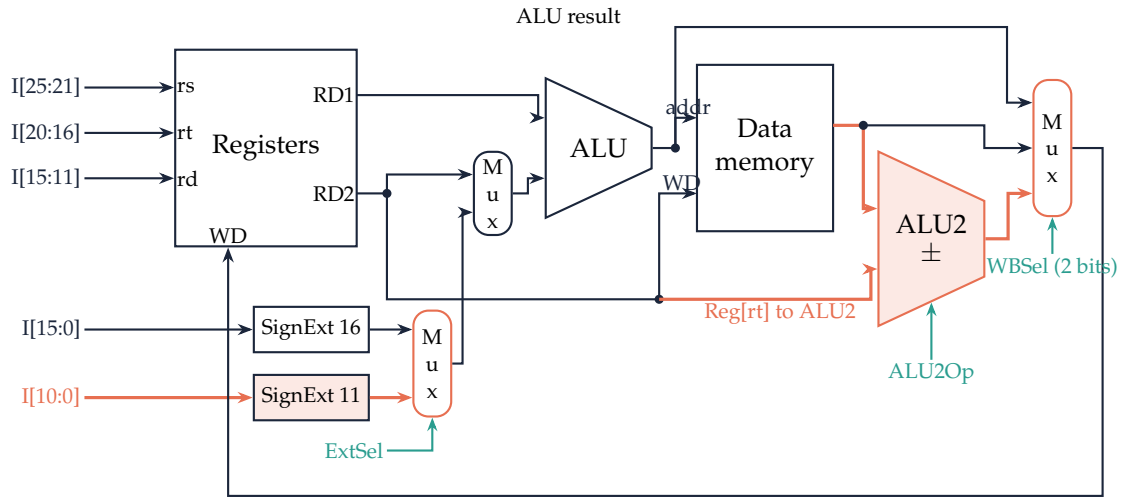
(iii) The offset is a byte offset written as 11-bit two's complement. It must be a multiple of 4 (word operand). If it lies outside $[-1024, 1023]$, the assembler expands the instruction into `lw $at,off(rs)` and `add/sub rd,rt,$at`, the same way pseudoinstructions are handled.

Example: `addm $t0, $s1, 8($s2)` → op= 24, rs= 18, rt= 17, rd= 8, offset= 8:

`011000 10010 10001 01000 00000001000` = `0x62514008`.

Similarly, `subm $t0,$s1,-4($s2)` has offset field `11111111100` and encodes as `0x665147FC`.

4. Hardware changes



(i) Second arithmetic unit (ALU2, add/sub) after data memory. In a single-cycle datapath the main ALU is busy computing the address rs+offset in the same cycle, so the memory operand needs its own adder/subtractor. Its inputs are Mem[EA] and Reg[rt] (Read data 2, which until now went only to memory for `sw`).

(ii) Write-back mux widened to 3 inputs (ALU result / memory data / ALU2 result). MemtoReg becomes a 2-bit signal, WBSel.

(iii) 11-bit sign extender plus a mux (ExtSel), so the address offset comes from bits 10–0 for M-type and from bits 15–0 for I-type.

(iv) Control unit: it decodes the two new opcodes and generates RegDst= 1, ALUSrc= 1, ALUOp=add, MemRead= 1, MemWrite= 0, RegWrite= 1, ExtSel= 1, WBSel=ALU2, ALU2Op= 0 (`addm`) or 1 (`subm`). All existing instructions get ExtSel= 0 and their old WBSel values, so their behaviour does not change.

(v) Timing: the critical path becomes IM $\rightarrow$ Reg $\rightarrow$ ALU $\rightarrow$ DM $\rightarrow$ ALU2 $\rightarrow$ Reg, longer than `lw`'s. In a single-cycle design *every* instruction pays for this longer clock. In a pipeline it would need an extra stage (or a memory-then-execute ordering, as in x86-style pipelines) and extra forwarding and stall logic. This is exactly why load/store RISC ISAs avoid memory operands.

Answer. Format: op(6) | rs(5, base) | rt(5, register operand) | rd(5, destination) | offset(11, signed), with opcodes 24/25 for `addm`/`subm`. Meaning: rd $=$ rt $\pm$ Mem[rs $+$ SignExt(offset)]. Hardware: a second add/sub unit after data memory, a 3-input write-back mux (WBSel), an 11-bit sign extender with its select mux (ExtSel), new control outputs, and a longer critical path.

2 Ch4 P2 Pipelining — Chapter 4 Part 2 — Pipelining

Notes: CSE209_Ch4_Part2_Pipelining_Notes.pdf

16. 2022-23 · Q6(a) · 5 marks · ★★★ High · Theory · L9

2022–2023 • Q6(a) CSE 305 • 25/04/2024 • PF02 **MARKS 5**

Joey claims that the number of cycles is identical to the number of instructions. Do you agree with Joey's statement? Justify your answer.

SOLUTION

Disagree. The two are related through the CPI:

$$\text{Clock cycles} = \sum_i \text{IC}_i \times \text{CPI}_i = \text{IC} \times \overline{\text{CPI}}.$$

- Cycles equal instructions only when the average CPI is exactly 1, i.e. every instruction takes exactly one cycle.
- In real processors, instruction classes take different numbers of cycles: multiply/divide, loads that miss in the cache, floating-point operations, and so on. A pipeline also loses cycles to stalls. So the average CPI is usually $\neq 1$. Superscalar (multiple-issue) machines can even reach CPI < 1 (IPC > 1).
- Slide example (sequence 1): IC $= 5$ instructions, but cycles $= 2 \times 1 + 1 \times 2 + 2 \times 3 = 10$, so CPI $= 2.0$.

Answer. No. Cycles $=$ IC $\times$ CPI, so they are equal only in the special case CPI $= 1$.

17. 2022-23 · Q1(a) · 12 marks · ★★★ High · Trace · L11

2022–2023 • Q1(a–c) CSE 305 • 25/04/2024 • PL10 **MARKS 12 + 16 + 7**

(a) Assume pipelined execution for the following sequence of instructions.

```
ins 1: sub $2, $1, $3
ins 2: and $2, $2, $5
ins 3: or  $6, $2, $7
```

Assume that **no data forwarding mechanism** is available to you. Construct the execution pipeline for the given sequence along with the stalls required to produce correct output. Evaluate the total number of clock cycles required to execute all three instructions. (12)

(b) Adapt **data forwarding mechanism** for the instruction set in Question 1(a) to enhance parallelism. Reconstruct the execution pipeline for the given sequence showing all pipelined registers and the forwarding lines. Revise the evaluation of the total number of clock cycles required to execute all three instructions. Explain double-data hazard with the help of your constructed pipeline. (16)

(c) Formulate the data forwarding condition between ins 1 and ins 2 in your constructed pipeline for 1 (b). (7)

SOLUTION

Dependences

`and` reads `$2` written by `sub` (RAW, distance 1). `or` reads `$2`, and the value it must get is the one

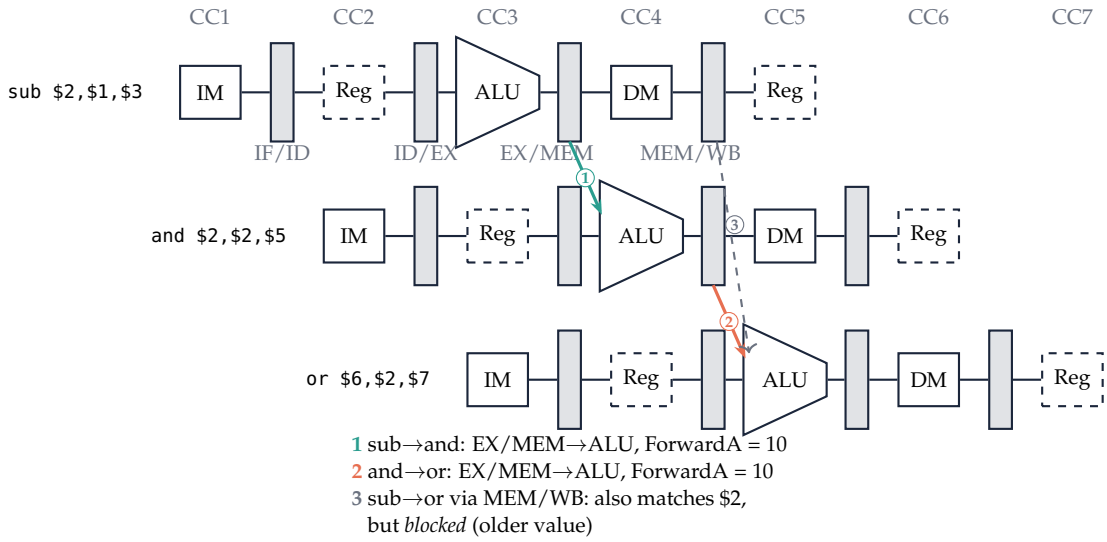
written by `and` (the most recent writer). `sub` also writes `$2`, two instructions earlier. Cycle count: total $= 5 + (n-1) +$ stalls.

(a) No forwarding. A value can be used only after it has been written to the register file. Writes happen in the first half of WB and reads in the second half of ID, so a consumer's (final) ID may coincide with the producer's WB. `and` must wait until `sub`'s WB in cycle 5, which costs 2 stalls. `or` must wait for `and`'s WB in cycle 8, another 2 stalls. (stall = the instruction repeats its previous stage.)

	1	2	3	4	5	6	7	8	9	10	11
`sub $2,$1,$3`	IF	ID	EX	MEM	WB						
`and $2,$2,$5`		IF	ID	stall	stall	EX	MEM	WB			
`or $6,$2,$7`			IF	stall	stall	ID	stall	stall	EX	MEM	WB

Answer. No forwarding: 4 stall cycles; total $= 5 + 2 + 4 =$ **11** clock cycles.

(b) With forwarding. Each result is taken from the pipeline register as soon as the ALU has produced it, so no stalls are needed:



- **CC4** (`and` in EX): EX/MEM holds `sub`'s result, and EX/MEM.Rd = `$2` = ID/EX.Rs, so ForwardA = 10 (path 1).
- **CC5** (`or` in EX): EX/MEM holds `and`'s result for `$2`, and MEM/WB holds `sub`'s result, also for `$2`. That is the **double data hazard**: both the EX-hazard and the MEM-hazard conditions are true for ID/EX.Rs = `$2`. The correct value is the most recent one (`and`), so ForwardA = 10 (path 2). The revised MEM-hazard condition ("forward from MEM/WB only if the EX hazard condition is not true") suppresses path 3. Forwarding `sub`'s value would give `or` the wrong `$2`.

	1	2	3	4	5	6	7
`sub $2,$1,$3`	IF	ID	EX	MEM	WB		
`and $2,$2,$5`		IF	ID	EX	MEM	WB	
`or $6,$2,$7`			IF	ID	EX	MEM	WB

Answer. With forwarding: 0 stalls; total $= 5 + 2 + 0 =$ **7** clock cycles (down from 11).

(c) Forwarding condition ins 1 → ins 2 (EX hazard on the first ALU operand, evaluated in CC4):

```
if (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
   and (EX/MEM.RegisterRd = ID/EX.RegisterRs)) ForwardA = 10
```

Here EX/MEM.RegWrite = 1 (`sub` is R-type), EX/MEM.RegisterRd = 2 ≠ 0, and ID/EX.RegisterRs

= 2. So ForwardA = 10 and the ALU's upper input takes sub's result from EX/MEM. ID/EX.RegisterRt = 5 $\neq$ 2, so ForwardB = 00.

Answer. ForwardA = 10 when EX/MEM.RegWrite $\wedge$ (EX/MEM.Rd $\neq$ 0) $\wedge$ (EX/MEM.Rd = ID/EX.Rs), true here with Rd = Rs = $2.

18. 2023-24 · Q1(a) · 10 marks · ★★★ High · Design · L12

2023–2024 • Q1(a) CSE 209 • 29/04/2026 • PL02 **MARKS 10**

Design a complete forwarding unit in the provided diagram in Figure for Question 1(a) with the necessary equations for the MIPS pipeline, considering double data hazard.
(The figure is not in the available scan. It is assumed to be the same simplified pipelined datapath as in the 2021–2022 paper, drawn below.)

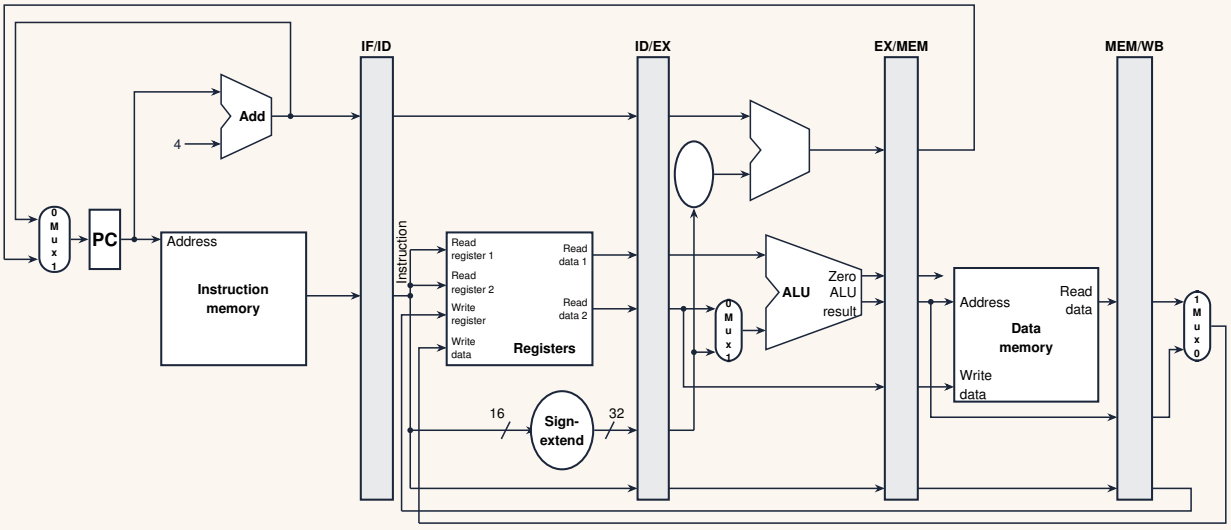


Figure for Question 1(a)

2021–2022 • Q2(a) CSE 305 • 06/04/2023 • PL02 **MARKS 15**

What do you mean by forwarding for data hazard? Design a complete forwarding unit in the provided diagram with necessary equations for the MIPS pipeline considering double data hazard.

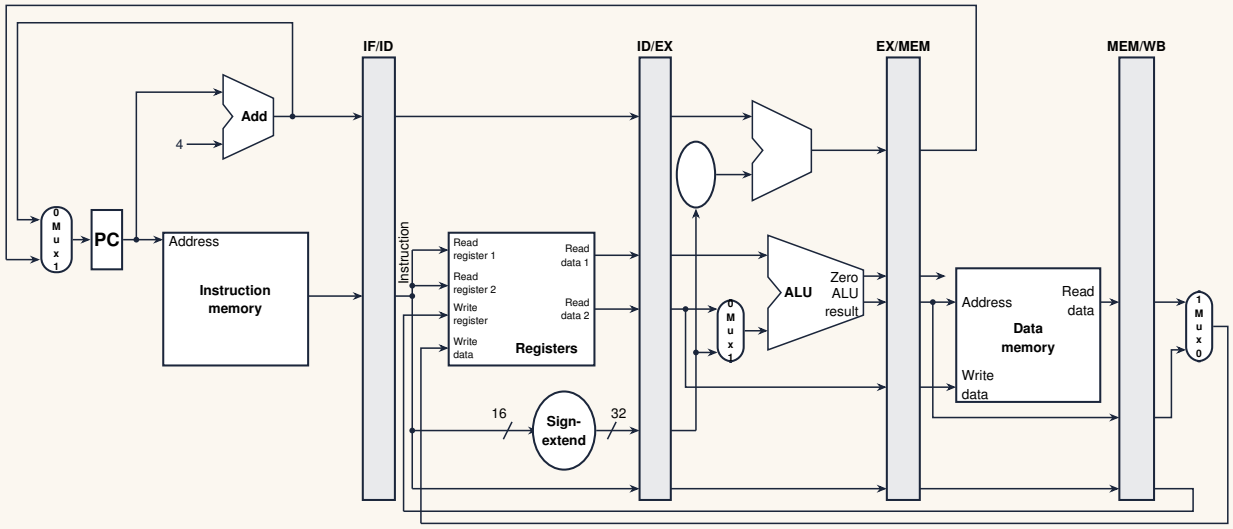


Figure for Question 2 (a)

2017–2018 • Q2(a–b) CSE 305 • 23/09/2018 • PL02 **MARKS 10 + 15**

(a) Briefly explain double data hazard with example. How MIPS detect hazard due to the use of load instruction and how MIPS stall the pipeline? (10)

(b) Design a complete forwarding unit with necessary equations for MIPS considering double data hazard. Draw the complete forwarding unit using the provided figure below. (15)

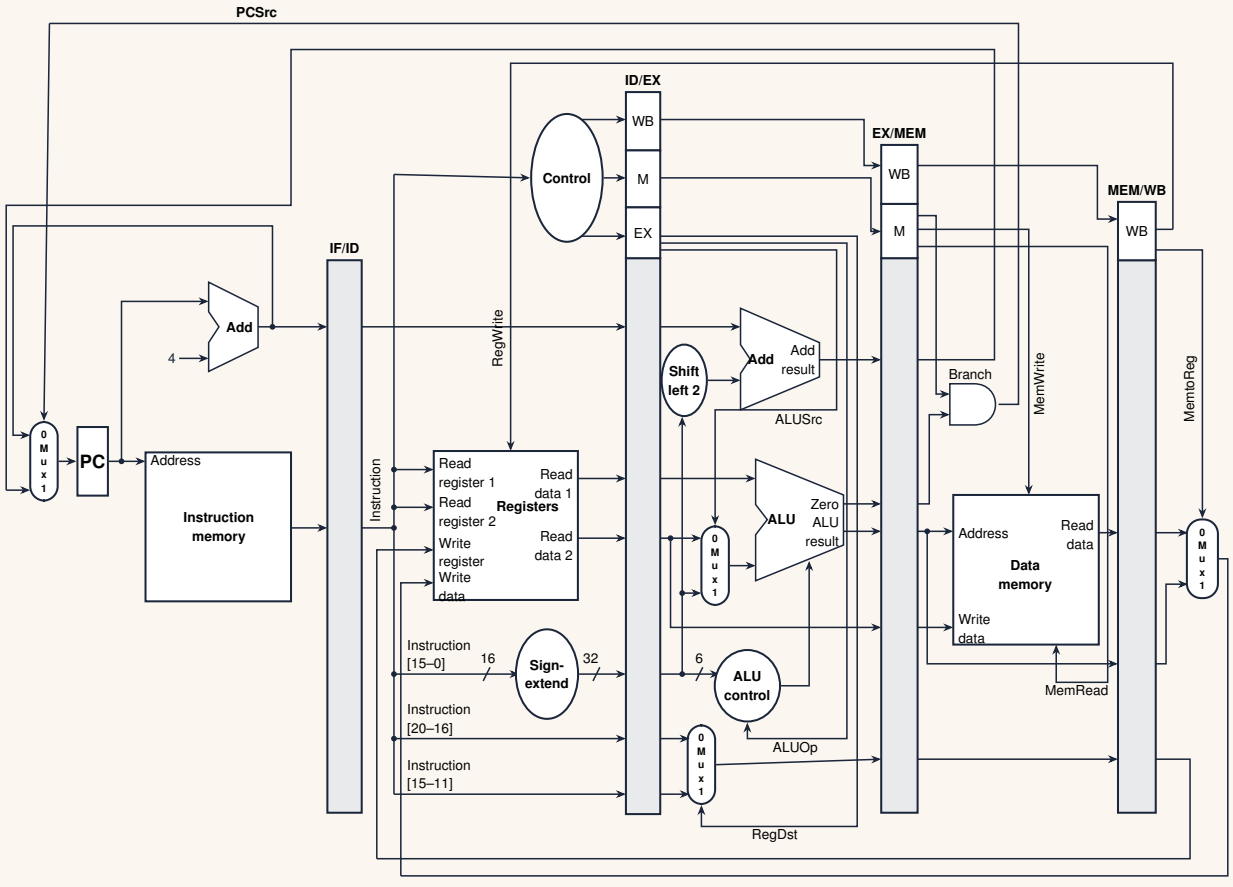


Figure for Question 2(b)

2015–2016 • Q3(a) CSE 305 • 18/01/2017 • PL02 **MARKS 10**

What do you mean by double data hazard? Write down the equations to detect forwarding in MIPS considering double data hazard? How MIPS detect hazard due to the use of load instruction?

2012–2013 • Q4(a) CSE 305 • 10/05/2014 • PL02 **MARKS 17**

Discuss how you can detect data hazards and solve those with the help of a forwarding unit.

SOLUTION

Forwarding (bypassing)

A data hazard occurs when an instruction needs a register value that an earlier instruction has not yet written back. With *forwarding*, the result is used as soon as it has been computed: it is taken from the EX/MEM or MEM/WB pipeline register and routed through extra paths and muxes straight to the ALU inputs, instead of waiting for WB. Example: in `sub $2,$1,$3; and $12,$2,$5`, the `sub` result sits in EX/MEM when `and` is in EX, so it is forwarded and no stall is needed.

Detecting the need to forward

Register numbers travel down the pipeline. The ALU operands in EX are named by `ID/EX.RegisterRs` and `ID/EX.RegisterRt`. A hazard exists when

1a.	EX/MEM.RegisterRd = ID/EX.RegisterRs	2a.	MEM/WB.RegisterRd = ID/EX.RegisterRs
1b.	EX/MEM.RegisterRd = ID/EX.RegisterRt	2b.	MEM/WB.RegisterRd = ID/EX.RegisterRt

Cases 1a/1b forward from EX/MEM, and cases 2a/2b forward from MEM/WB. A forward happens only if the earlier instruction really writes a register (`RegWrite`) and its destination is not `$zero`. Here "RegisterRd" is the destination after the RegDst mux, so it is `rt` for `lw` and `addi`.

Double data hazard

```
add $1,$1,$2      # $1 = 10+30 = 40
add $1,$1,$3      # $1 = 40+20 = 60
add $1,$1,$4      # needs 60, result 64
```

(The values follow the slides: initially `$1`=10, `$2`=30, `$3`=20, `$4`=4.) When the third `add` is in EX, EX/MEM holds 60 from the 2nd `add` and MEM/WB holds 40 from the 1st. *Both* conditions 1a and 2a are true. The most recent value (60) must be used. If the MEM condition were allowed to override, 40 would be forwarded, which is wrong. The fix is to **forward from MEM/WB only if the EX hazard condition is not true**.

Complete forwarding equations (with the revised MEM hazard)

EX hazard

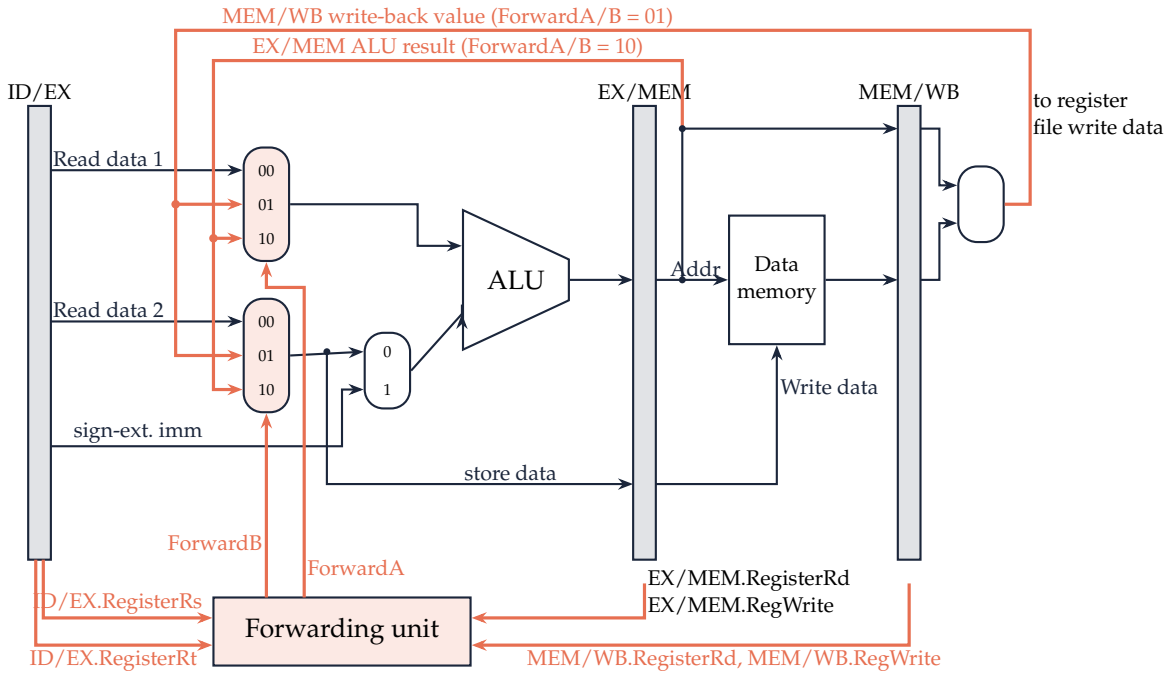
```
if (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
   and (EX/MEM.RegisterRd = ID/EX.RegisterRs)) ForwardA = 10
if (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
   and (EX/MEM.RegisterRd = ID/EX.RegisterRt)) ForwardB = 10
```

MEM hazard (revised for double data hazard)

```
if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
   and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
            and (EX/MEM.RegisterRd = ID/EX.RegisterRs))
```

```
   and (MEM/WB.RegisterRd = ID/EX.RegisterRs)) ForwardA = 01
if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
   and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
            and (EX/MEM.RegisterRd = ID/EX.RegisterRt))
   and (MEM/WB.RegisterRd = ID/EX.RegisterRt)) ForwardB = 01
otherwise ForwardA = 00, ForwardB = 00
```

Forwarding unit added to the datapath (new hardware in coral)



Mux control	Source	Meaning
ForwardA/B = 00	ID/EX	operand from the register file (no hazard)
ForwardA/B = 10	EX/MEM	forwarded result of the instruction 1 ahead (EX hazard)
ForwardA/B = 01	MEM/WB	ALU or load result of the instruction 2 ahead (MEM hazard)

Forwarding into ID is not needed. The register file is written in the first half of a cycle and read in the second half, so an instruction 3 ahead is handled by the register file itself.

Hazard that forwarding cannot remove: load-use (2017–18 (a), 2015–16)

A `lw` has its data only at the end of MEM, but the very next instruction needs it at the start of its EX, one cycle too early ("can't forward backward in time"). The hazard detection unit in ID checks

```
ID/EX.MemRead and ((ID/EX.RegisterRt = IF/ID.RegisterRs)
                or (ID/EX.RegisterRt = IF/ID.RegisterRt))
```

How the pipeline is stalled: (1) force the control fields written into ID/EX to 0, so a bubble (nop) goes through EX, MEM and WB; (2) set PCWrite = 0 and IF/IDWrite = 0, so the using instruction is decoded again and the following instruction is fetched again. After the 1-cycle stall the loaded value is in MEM/WB and is forwarded to EX (ForwardA/B = 01).

	1	2	3	4	5	6	7	8
`lw $2,20($1)`	IF	ID	EX	MEM	WB			
`and $4,$2,$5`		IF	ID	stall	EX	MEM	WB	
`or $8,$2,$6`			IF	stall	ID	EX	MEM	WB

Answer. Forwarding unit: ForwardA/ForwardB select 10 on an EX hazard and 01 on a MEM hazard only when no EX hazard exists for the same operand. Both cases need RegWrite = 1 and Rd $\neq 0$. A load-use hazard is detected in ID and costs one stall (bubble + PC/IF–ID hold).

19. 2022-23 · Q1(b) · 16 marks · ★★★ High · Trace · L12

Same question - solution already given under #17.

20. 2022-23 · Q1(c) · 7 marks · ★★★ High · Theory · L12

Same question - solution already given under #17.

21. 2021-22 · Q2(a) · 15 marks · ★★★ High · Design · L12

Same question - solution already given under #18.

22. 2020-21 · Q4 · 15 marks · ★★★ High · Trace · L12

2020–2021 • Q4 CSE 305 • 24/09/2022 • PL15 **MARKS 15**

Suppose you are maintaining a 5-stage pipeline in a MIPS datapath. You are given the following code snippet to execute. Identify all the data hazards that can occur using a multi-cycle pipeline diagram and mark where there is a need for forwarding. Also write down the forwarding conditions to mitigate the hazards. Can all of the hazards in this snippet be addressed using forwarding? Defend your position.

```
lw   $t0, 0($t1)
add  $t0, $t0, $t1
add  $t0, $t2, $t0
add  $t0, $t0, $t3
sub  $t1, $t2, $t3
and  $t4, $t5, $t6
sw   $t0, 0($t6)
```

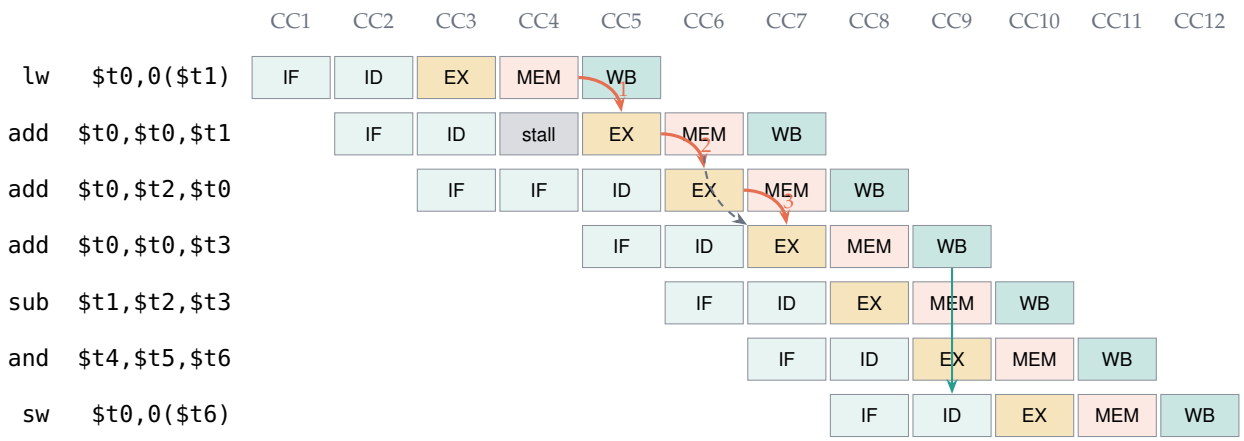
SOLUTION

Common rules

A **RAW** dependence is a hazard when the consumer is fewer than 3 instructions after the producer. The register file writes in the first half of a cycle and reads in the second half, so a distance of 3 is safe. WAR and WAW dependences cannot cause hazards in the in-order 5-stage MIPS pipeline.

- Distance 1 (ALU producer): forward from **EX/MEM** (Forward = 10).
- Distance 2: forward from **MEM/WB** (Forward = 01).
- Load followed directly by a use: **load-use hazard**. Forwarding cannot go backward in time, so the hazard detection unit inserts 1 stall, and then the value is forwarded from MEM/WB.

Dependences on `$t0`: I1→I2 (load-use), I2→I3, I3→I4 (I2→I4 as well, a *double data hazard*), and I4→I7 (distance 3, so no hazard). I5 (`sub`) overwrites `$t1`, which I1/I2 read earlier; that is WAR, so no hazard. I6 is independent.



Coral arrows are the forwarding needed. Dashed: the older MEM/WB value of `$t0` (from I2), which must *not* be used. Teal: I4 writes `$t0` in CC9 and I7 reads it in CC9, through the register file. The whole snippet takes 12 cycles (7 + 4 + 1 stall).

#	Hazard	Resolution	Forwarding condition (checked in EX)
1	I1→I2, `$t0` (load-use)	stall 1 + fwd	detect: `ID/EX.MemRead` ∧ `ID/EX.Rt = IF/ID.Rs` then `MEM/WB.Rd = ID/EX.Rs` ⇒ ForwardA = 01
2	I2→I3, `$t0` as Rt	EX/MEM fwd	`EX/MEM.Rd = ID/EX.Rt` ⇒ ForwardB = 10
3	I3→I4, `$t0` as Rs	EX/MEM fwd	`EX/MEM.Rd = ID/EX.Rs` ⇒ ForwardA = 10
	(I2→I4 double hazard)	MEM/WB blocked	MEM/WB fwd only if the EX-hazard test is false

The general conditions, including the RegWrite and ≠ $0 checks and the double-data-hazard rule:

```
EX : if (EX/MEM.RegWrite and EX/MEM.Rd != 0 and EX/MEM.Rd = ID/EX.Rs) ForwardA = 10
     if (EX/MEM.RegWrite and EX/MEM.Rd != 0 and EX/MEM.Rd = ID/EX.Rt) ForwardB = 10
MEM: if (MEM/WB.RegWrite and MEM/WB.Rd != 0
         and not (EX/MEM.RegWrite and EX/MEM.Rd != 0 and EX/MEM.Rd = ID/EX.Rs)
         and MEM/WB.Rd = ID/EX.Rs) ForwardA = 01
     (same with Rt for ForwardB = 01)
```

Answer. **No**, forwarding alone cannot remove every hazard. In the load-use pair I1→I2, `$t0` exists only at the end of I1's MEM stage, but I2 needs it at the start of its own EX, in the same cycle. Data cannot be forwarded backward in time, so one stall is unavoidable unless the code is rescheduled. All other hazards are removed by forwarding. Scheduling fix: move the independent `and $t4,$t5,$t6` between I1 and I2. (`sub $t1,...` cannot be moved there, because I2 reads `$t1`.) The stall disappears: `$t0` reaches the first `add` by MEM/WB forwarding, and `sw` (now 2 behind I4) also gets it from MEM/WB. The snippet then runs in 11 cycles.

2019–2020 • Q8(c) CSE 305 • 18/09/2021 • PL19 **MARKS 6×2 = 12**

Figure 8c shows a pipelined execution of three instructions **sub**, **and**, and **or**. The pipeline registers are denoted respectively as IF/ID, ID/EX, EX/MEM, and MEM/WB. The data forwarding path between **sub** and **and** has been shown in *green* color and the data forwarding path between **sub** and **or** has been shown in *yellow* color. For each of these two data forwarding cases, write down the data forwarding conditions.

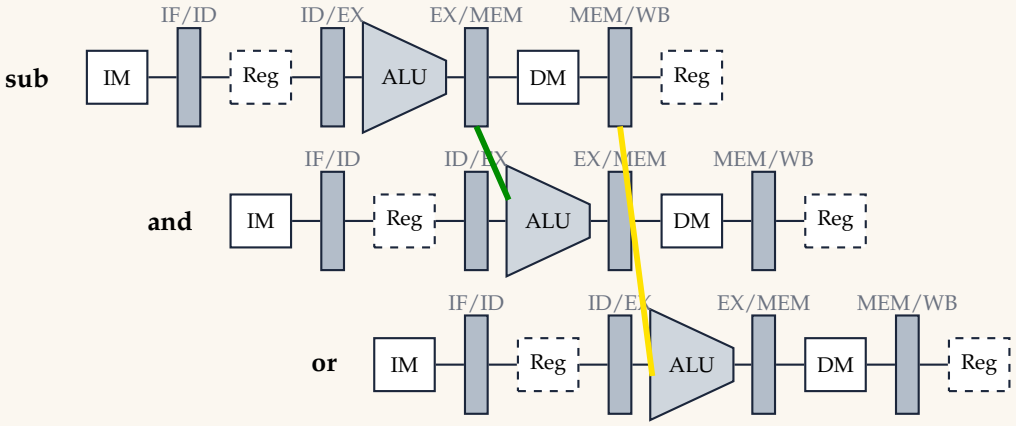


Figure 8c (redrawn): pipelined execution with data forwarding — green: sub→and, yellow: sub→or.

SOLUTION

NOTE Figure 8c does not print the operands. They are taken as in the course slides (P&H): `sub $2,$1,$3`, `and $12,$2,$5`, `or $13,$6,$2`. So `$2` is the *first* source (rs) of `and` and the *second* source (rt) of `or`. The general conditions below hold whichever operand is used: replace Rs/ForwardA by Rt/ForwardB as needed.

Green path: sub → and (EX hazard, forward from EX/MEM)

When `and` is in EX (CC4), `sub` is in MEM and its ALU result sits in EX/MEM.

```
if (EX/MEM.RegWrite                                          [sub writes a register]
   and (EX/MEM.RegisterRd ≠ 0)                          [destination is not $zero]
   and (EX/MEM.RegisterRd = ID/EX.RegisterRs))                        [$2 = rs of and]
      ForwardA = 10                                  [ALU input A ← EX/MEM.ALUResult]
```

Yellow path: sub → or (MEM hazard, forward from MEM/WB)

When `or` is in EX (CC5), `sub` is in WB and its result sits in MEM/WB. By the revised (double-data-hazard-safe) condition:

```
if (MEM/WB.RegWrite                                          [sub writes a register]
   and (MEM/WB.RegisterRd ≠ 0)                                          [not $zero]
   and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
            and (EX/MEM.RegisterRd = ID/EX.RegisterRt))   [and is not a newer writer]
   and (MEM/WB.RegisterRd = ID/EX.RegisterRt))                        [$2 = rt of or]
      ForwardB = 01                                  [ALU input B ← MEM/WB result]
```

Here `and` writes `$12` ≠ `$2`, so the "not EX hazard" term is true and the yellow forward happens.

Path	Cycle	Source register	Condition checks	Mux setting
green (sub→and)	CC4	EX/MEM	RegWrite, Rd≠0, EX/MEM.Rd = ID/EX.Rs	ForwardA = 10
yellow (sub→or)	CC5	MEM/WB	RegWrite, Rd≠0, no EX hazard, MEM/WB.Rd = ID/EX.Rt	ForwardB = 01

Answer. Green: EX/MEM.RegWrite $\wedge$ EX/MEM.Rd$\neq 0 \wedge$ EX/MEM.Rd = ID/EX.Rs $\Rightarrow$ ForwardA = 10. Yellow: MEM/WB.RegWrite $\wedge$ MEM/WB.Rd$\neq 0 \wedge \neg$(EX hazard on the same operand) $\wedge$ MEM/WB.Rd = ID/EX.Rt $\Rightarrow$ ForwardB = 01.

24. 2022-23 · Q2 · 15 marks · ★★★ High · Design · L13

2022–2023 • Q2 CSE 209 • 13/01/2025 • PL07 **MARKS 15**

Consider the following code snippet:

```
lw    $t0, 55($t1)
addi  $t1, $t0, 1
lw    $t0, 57($t1)
sw    $t0, 32($t4)
```

Identify the hazard(s) present in the given code snippet. Write the detection conditions for each identified hazard. Complete the *Hazard Detection Unit* by including all the necessary input and output signals in the provided reference figure (Figure for Question 2) to ensure that hazard(s) can be detected and stalls can be applied when necessary. Draw the given partial diagram in your script and complete only the Hazard Detection Unit.

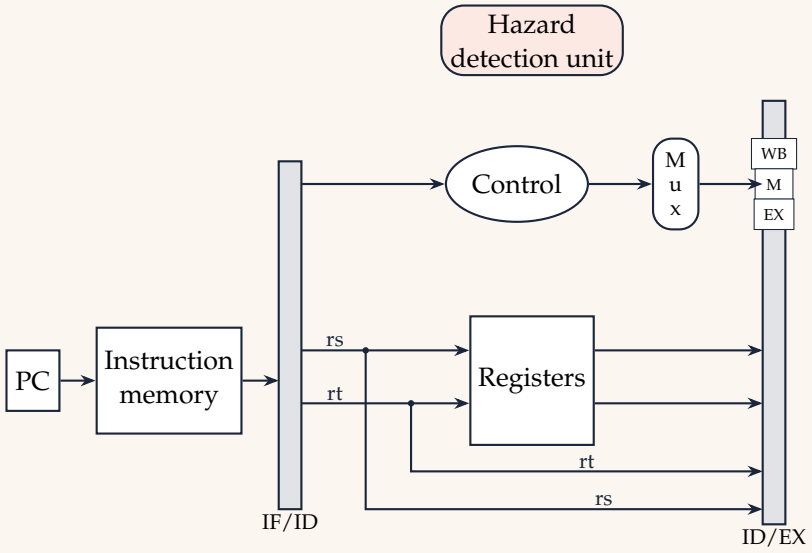


Figure for Question 2 (redrawn)

SOLUTION

Hazards in the snippet

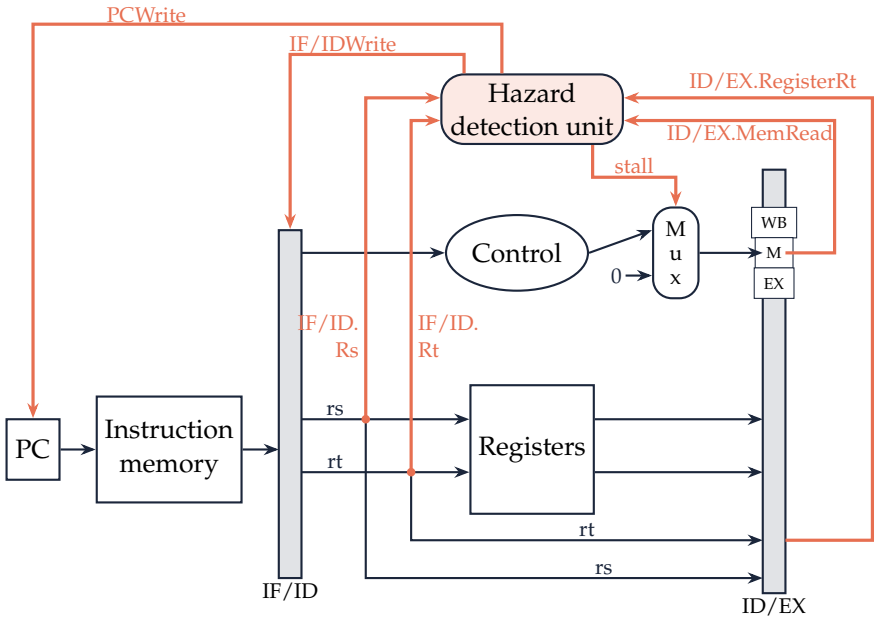
Pair	Register	Type and handling
`lw $t0,55($t1)` → `addi $t1,$t0,1`	`$t0` (rs)	**load-use** RAW: **1 stall** by the hazard detection unit, then MEM/WB→EX forward
`addi $t1,$t0,1` → `lw $t0,57($t1)`	`$t1` (rs = base)	ALU→address RAW: EX/MEM→EX forward (ForwardA = 10), no stall
`lw $t0,57($t1)` → `sw $t0,32($t4)`	`$t0` (rt = store data)	**load→store** RAW: caught by the load-use check on rt, so 1 stall, then forward
`lw/lw` on `$t0`; `lw/addi` on `$t1`	—	WAW / WAR: not hazards in the in-order 5-stage pipeline

Detection conditions

```
(1) load-use (lw→addi), checked when addi is in ID:
    ID/EX.MemRead and ((ID/EX.RegisterRt = IF/ID.RegisterRs)
                    or (ID/EX.RegisterRt = IF/ID.RegisterRt))
    here ID/EX.RegisterRt = $t0 = IF/ID.RegisterRs  ⇒ stall
(2) EX hazard (addi→lw), checked when lw is in EX:
    EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs)  ⇒ ForwardA = 10
    (RegisterRd = RegDst-selected destination = rt = $t1 for addi)
(3) load→store (lw→sw), checked when sw is in ID: same as (1),
    matching ID/EX.RegisterRt = $t0 = IF/ID.RegisterRt  ⇒ stall
```

NOTE The `sw` needs `$t0` only in its MEM stage, so this stall could be avoided by an extra MEM/WB→data-memory forward ("Load → Store" slide). The standard hazard detection unit of the slides compares rt as well, so it stalls one cycle and then forwards MEM/WB→EX. The timing below uses the standard unit.

Completed hazard detection unit (added signals in coral)



Signal	Equation
`stall`	ID/EX.MemRead ∧ [(ID/EX.Rt = IF/ID.Rs) ∨ (ID/EX.Rt = IF/ID.Rt)]
`PCWrite`	$\overline{\texttt{stall}}$ (PC holds; the next instruction is fetched again)
`IF/IDWrite`	$\overline{\texttt{stall}}$ (the using instruction stays in ID, decoded again)
control-mux select	`stall` (EX/M/WB control bits into ID/EX forced to 0 = bubble)

Resulting timing (standard unit, full forwarding)

	1	2	3	4	5	6	7	8	9	10
`lw $t0,55($t1)`	IF	ID	EX	MEM	WB					
`addi $t1,$t0,1`		IF	ID	stall	EX	MEM	WB			
`lw $t0,57($t1)`			IF	stall	ID	EX	MEM	WB		
`sw $t0,32($t4)`					IF	ID	stall	EX	MEM	WB

Answer. Hazards: load-use on `$t0` (stall + forward), ALU→base on `$t1` (forward 10, no stall), load→store on `$t0` (caught by the rt comparison, 1 stall). The hazard detection unit takes IF/ID.Rs, IF/ID.Rt, ID/EX.Rt and ID/EX.MemRead, and drives PCWrite, IF/IDWrite and the control-zeroing mux. Total $5 + 3 + 2 = 10$ cycles.

25. 2022-23 · Q1(c) · 10 marks · ★★★ High · Trace · L14

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2022–2023 • Q1(c) CSE 209 • 13/01/2025 • PL06 MARKS 10

Consider the following code

$$A = B + C; \qquad D = A + E;$$

All the above are memory instructions (i.e. the values of the variable are stored in a memory location). The base address is in register `$t0` and the offsets are 0, 4, 8, 12, 16 for A, B, C, D, and E, respectively. Write the equivalent MIPS assembly code to determine the number of cycles that are required to execute the code.
Can you reduce the number of cycles computed above using code scheduling? If yes, then rearrange the MIPS assembly code and recount the number of cycles required to execute the rearranged code.

2017–2018 • Q1(a) CSE 305 • 23/09/2018 • PL06 MARKS 10

Consider the following C code:

$$A = B + C;\ D = A + E;$$

All the above are memory instructions. The base address is in register t0 and the offsets are A (0), B (4), C (8), D (12), E (16). How many cycles are required to execute the above? Can you reduce the number of cycles using code scheduling? If yes, what will be resulting number of cycles? Show each step.

2015–2016 • Q2(d) CSE 305 • 18/01/2017 • PL06 MARKS 10

Consider the following C code

$$A = B + C; \quad D = A + E;$$

All the above are memory instructions. The base address is in register t0 and the offset are A(0), B(4), C(8), D(12), E(16). How many cycles are required to execute the above? Can you reduce the number of cycles using code scheduling? If yes, what will be the resulting number of cycles? Show each step.

SOLUTION

Method

The pipeline is the 5-stage MIPS pipeline with full forwarding (as in the slides). ALU→ALU and ALU→`sw` values are forwarded with no stall. The only stall is the **load-use hazard**: an instruction that uses a `lw` result right after the load waits **1 cycle**, then gets it from MEM/WB. Total

$= 5 + (n - 1) +$ stalls. stall = the instruction repeats its previous stage.

Step 1: straightforward code

```
lw   $t1, 4($t0)     # B
lw   $t2, 8($t0)     # C
add  $t3, $t1, $t2   # A=B+C  (stall)
sw   $t3, 0($t0)     # store A
lw   $t4, 16($t0)    # E
add  $t5, $t3, $t4   # D=A+E  (stall)
sw   $t5, 12($t0)    # store D
```

Step 2: scheduled code

```
lw   $t1, 4($t0)     # B
lw   $t2, 8($t0)     # C
lw   $t4, 16($t0)    # E  (moved up)
add  $t3, $t1, $t2   # A
sw   $t3, 0($t0)
add  $t5, $t3, $t4   # D
sw   $t5, 12($t0)
```

Step 1 count. `add $t3` uses `$t2` right after its load, which costs 1 stall. `add $t5` uses `$t4` right after its load, another stall. Both sws get their data by forwarding, and A is kept in `$t3`, so it is not re-loaded.

	1	2	3	4	5	6	7	8	9	10	11	12	13
`lw $t1,4($t0)`	IF	ID	EX	MEM	WB								
`lw $t2,8($t0)`		IF	ID	EX	MEM	WB							
`add $t3,$t1,$t2`			IF	ID	stall	EX	MEM	WB					
`sw $t3,0($t0)`				IF	stall	ID	EX	MEM	WB				
`lw $t4,16($t0)`						IF	ID	EX	MEM	WB			
`add $t5,$t3,$t4`							IF	ID	stall	EX	MEM	WB	
`sw $t5,12($t0)`								IF	stall	ID	EX	MEM	WB

Total $= 5 + (7 - 1) + 2 = 13$ cycles.

Step 2 count. The load of E depends on nothing above it, so it moves into the slot after `lw $t2`. Now `add $t3` is 2 instructions after `lw $t2` (MEM/WB forward), and `add $t5` is 3 after `lw $t4` (register file). There are no stalls.

	1	2	3	4	5	6	7	8	9	10	11
`lw $t1,4($t0)`	IF	ID	EX	MEM	WB						
`lw $t2,8($t0)`		IF	ID	EX	MEM	WB					
`lw $t4,16($t0)`			IF	ID	EX	MEM	WB				
`add $t3,$t1,$t2`				IF	ID	EX	MEM	WB			
`sw $t3,0($t0)`					IF	ID	EX	MEM	WB		
`add $t5,$t3,$t4`						IF	ID	EX	MEM	WB	
`sw $t5,12($t0)`							IF	ID	EX	MEM	WB

Answer. Original: 2 load-use stalls, $5 + 6 + 2 =$ **13 cycles**. Yes, scheduling helps: move `lw` of E above the first `add`, giving 0 stalls and $5 + 6 + 0 =$ **11 cycles**.

26. 2021-22 · Q1(b) · 10 marks · ★★★ High · Trace · L14

2021–2022 • Q1(b) CSE 305 • 06/04/2023 • PL12 **MARKS 10**

Consider the following MIPS code:

```
lw   $t1, 0($t0)
lw   $t2, 4($t0)
add  $t3, $t1, $t2
sw   $t3, 12($t0)
lw   $t4, 8($t0)
add  $t5, $t1, $t4
sw   $t5, 16($t0)
```

How many cycles are required to execute the above code? Can you reduce the number of cycles using code scheduling? If yes, what will be resulting number of cycles?

SOLUTION

Method

The pipeline is the 5-stage MIPS pipeline with full forwarding. ALU→ALU and ALU→`sw` dependences are forwarded. Only a **load-use hazard** (a `lw` result needed by the very next instruction) costs **1 stall**. Total $= 5 + (n - 1) +$ stalls. This is the slides' example $A = B + E;\ C = B + F$.
Original code. `lw $t2`→`add $t3` is a load-use hazard (1 stall), and so is `lw $t4`→`add $t5` (1 stall). `add`→`sw` values are forwarded from EX/MEM.

	1	2	3	4	5	6	7	8	9	10	11	12	13
`lw $t1,0($t0)`	IF	ID	EX	MEM	WB								
`lw $t2,4($t0)`		IF	ID	EX	MEM	WB							
`add $t3,$t1,$t2`			IF	ID	stall	EX	MEM	WB					
`sw $t3,12($t0)`				IF	stall	ID	EX	MEM	WB				
`lw $t4,8($t0)`						IF	ID	EX	MEM	WB			
`add $t5,$t1,$t4`							IF	ID	stall	EX	MEM	WB	
`sw $t5,16($t0)`								IF	stall	ID	EX	MEM	WB

Total $= 5 + 6 + 2 = 13$ cycles.
Scheduled code. `lw $t4` is independent of the instructions above it, so it moves up to third place. Every load is then at least 2 instructions ahead of its use:

```
lw   $t1, 0($t0)
lw   $t2, 4($t0)
lw   $t4, 8($t0)      # moved up
add  $t3, $t1, $t2    # $t2 forwarded from MEM/WB
sw   $t3, 12($t0)     # $t3 forwarded from EX/MEM
add  $t5, $t1, $t4    # $t4 via register file
sw   $t5, 16($t0)
```

	1	2	3	4	5	6	7	8	9	10	11
`lw $t1,0($t0)`	IF	ID	EX	MEM	WB						
`lw $t2,4($t0)`		IF	ID	EX	MEM	WB					
`lw $t4,8($t0)`			IF	ID	EX	MEM	WB				
`add $t3,$t1,$t2`				IF	ID	EX	MEM	WB			
`sw $t3,12($t0)`					IF	ID	EX	MEM	WB		
`add $t5,$t1,$t4`						IF	ID	EX	MEM	WB	
`sw $t5,16($t0)`							IF	ID	EX	MEM	WB

Answer. 13 cycles (2 load-use stalls). Yes: moving `lw $t4,8($t0)` up removes both stalls, giving $5+6+0=$ **11 cycles**.

27. 2019-20 · Q5(a) · 15 marks · ★★★ High · Trace · L14

2019–2020 • Q5(a–c) CSE 305 • 18/09/2021 • PL17 **MARKS 15 + 5 + 15**

(a) Assume pipelined execution for the following sequence of instructions. The corresponding C code is $A = B + C;\ D = A + D$.

```
lw  $t1, 0($t0)      // load B
lw  $t2, 4($t0)      // load C
add $t3, $t1, $t2    // add B & C
sw  $t3, 12($t0)     // store A
lw  $t4, 8($t0)      // load D
add $t4, $t3, $t4    // add A & D
sw  $t4, 8($t0)      // store D
```

Draw the execution pipeline for the given sequence of instructions and clearly show the stalls and the forwarding paths. Note that data forwarding is available if needed. (15)

(b) How many stalls and clock cycles are required in question 5(a)? (5)

(c) Is it possible to rearrange the code in question 5(a) to reduce the required number of stalls? If not, then explain why it is not possible. If yes, then write down the rearranged code block and indicate the number of stalls and the total number of clock cycles required after the rearrangement. (15)

SOLUTION

Method

The pipeline is the 5-stage MIPS pipeline with full forwarding. ALU→ALU and ALU→`sw` values are forwarded with no stall. A **load-use hazard** (a `lw` result used by the next instruction) costs **1 stall**, then MEM/WB→EX forwarding. The register file is written in the first half of a cycle and read in the second. Total $= 5 + (n-1) +$ stalls. stall = the instruction repeats its previous stage.

(a) Pipeline with stalls and forwarding paths

	1	2	3	4	5	6	7	8	9	10	11	12	13
`lw $t1,0($t0)`	IF	ID	EX	MEM	WB								
`lw $t2,4($t0)`		IF	ID	EX	MEM	WB							
`add $t3,$t1,$t2`			IF	ID	stall	EX	MEM	WB					
`sw $t3,12($t0)`				IF	stall	ID	EX	MEM	WB				
`lw $t4,8($t0)`						IF	ID	EX	MEM	WB			
`add $t4,$t3,$t4`							IF	ID	stall	EX	MEM	WB	
`sw $t4,8($t0)`								IF	stall	ID	EX	MEM	WB

From → to	Reg	Path (cycle)	Note
`lw $t1` → `add $t3`	`$t1`	register file (WB c5, read 2nd half)	read in the repeated ID (c5)
`lw $t2` → `add $t3`	`$t2`	MEM/WB → ALU in B (c6)	after 1 load-use stall
`add $t3` → `sw $t3`	`$t3`	EX/MEM → ALU in B / store data (c7)	no stall
`add $t3` → `add $t4`	`$t3`	register file (WB c8, read in ID c8)	no forwarding needed
`lw $t4` → `add $t4`	`$t4`	MEM/WB → ALU in B (c10)	after 1 load-use stall
`add $t4` → `sw $t4`	`$t4`	EX/MEM → ALU in B / store data (c11)	no stall

(b)

Answer. 2 stalls (both load-use: `lw $t2`→`add $t3` and `lw $t4`→`add $t4`); clock cycles $= 5 + (7-1) + 2 = \mathbf{13}$.

(c) Rearrangement — yes. `lw $t4,8($t0)` (load D) depends on nothing before it. Address 8 is not written by `sw $t3,12($t0)`, so it can move up behind `lw $t2`:

```
lw  $t1, 0($t0)      # B
lw  $t2, 4($t0)      # C
```

```
lw  $t4, 8($t0)      # D  (moved up)
add $t3, $t1, $t2    # A = B + C   ($t2 via MEM/WB)
sw  $t3, 12($t0)     # store A     ($t3 via EX/MEM)
add $t4, $t3, $t4    # D = A + D   ($t3 via MEM/WB)
sw  $t4, 8($t0)      # store D     ($t4 via EX/MEM)
```

	1	2	3	4	5	6	7	8	9	10	11
`lw $t1,0($t0)`	IF	ID	EX	MEM	WB						
`lw $t2,4($t0)`		IF	ID	EX	MEM	WB					
`lw $t4,8($t0)`			IF	ID	EX	MEM	WB				
`add $t3,$t1,$t2`				IF	ID	EX	MEM	WB			
`sw $t3,12($t0)`					IF	ID	EX	MEM	WB		
`add $t4,$t3,$t4`						IF	ID	EX	MEM	WB	
`sw $t4,8($t0)`							IF	ID	EX	MEM	WB

Answer. After rearrangement: **0 stalls**, $5 + 6 + 0 = \mathbf{11}$ clock cycles.

28. 2019-20 · Q5(b) · 5 marks · ★★★ High · Numerical · L14

Same question - solution already given under #27.

29. 2019-20 · Q5(c) · 15 marks · ★★★ High · Trace · L14

Same question - solution already given under #27.

30. 2023-24 · Q1(b) · 15 marks · ★★★ High · Trace · L15

2023–2024 • Q1(b) CSE 209 • 29/04/2026 • PL03 MARKS 15

You are given the following MIPS code:

```
      lw   $t0, 0($s0)
      add  $t1, $t0, $s1
      beq  $t1, $zero, L1
      add  $t2, $t1, $s2
      bne  $t2, $zero, L2
      sub  $t3, $t2, $t1
L1:   and  $t4, $t1, $t2
L2:   or   $t5, $t3, $t4
```

You can assume the following:

- 5 stage MIPS: IF – ID – EX – MEM – WB
- Code rescheduling is not available
- All stage forwarding available
- Branch resolved in ID

- Taken branch penalty = 1 cycle
- Load-use hazard = 1 stall

Now, identify all the data and control hazards and evaluate the following:

(i) Total stall cycles

(ii) Total cycles if both branches are not taken

(iii) Total cycles if the first branch is taken

(iv) Total cycles if the first branch is not taken, but the second branch is taken

SOLUTION

Rules used (course slides)

Load-use costs 1 stall, then MEM/WB→EX forwarding. ALU→ALU dependences are forwarded with no stall. The branch is resolved in ID, so if it compares a register written by the *immediately preceding ALU* instruction it waits **1 stall**, then gets the value forwarded from EX/MEM into the comparator (slides: "Data hazards for branches"). Predict not taken: a taken branch flushes 1 instruction. Total $= 5 + (n-1) +$ stalls + flushes, with n = instructions executed. A stall cell means the instruction repeats its previous stage.

> **NOTE** The assumptions list only the load-use stall. With the branch resolved in ID, the slides' rule adds 1 stall for each branch that follows the ALU instruction producing its operand (`add $t1`→`beq`, `add $t2`→`bne`). These stalls are counted below.

Data hazards: `lw`→`add $t1` (`$t0`, load-use, **1 stall**); `add $t1`→`beq` (`$t1`, ALU→branch in ID, **1 stall**); `add $t2`→`bne` (`$t2`, ALU→branch in ID, **1 stall**). These pairs are covered by forwarding with no stall: `$t1` into `add $t2`, `sub`, `and`; `$t2` into `sub`, `and`; `$t3` from `sub` into `or` (MEM/WB); `$t4` from `and` into `or` (EX/MEM).

Control hazards: `beq` and `bne`, each costing a 1-cycle flush when taken.

(i) Total stall cycles.

Path	data stalls	taken-branch flushes	total stall cycles
both not taken	3	0	3
first taken (`bne` never executed)	2	1	3
first not taken, second taken	3	1	4

If every instruction executes, there are 3 data-hazard stalls.

(ii) Both not taken: $n = 8$, 3 stalls.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
`lw $t0,0($s0)`	IF	ID	EX	MEM	WB										
`add $t1,$t0,$s1`		IF	ID	stall	EX	MEM	WB								
`beq $t1,$zero,L1`			IF	stall	ID	stall	EX	MEM	WB						
`add $t2,$t1,$s2`					IF	stall	ID	EX	MEM	WB					
`bne $t2,$zero,L2`							IF	ID	stall	EX	MEM	WB			
`sub $t3,$t2,$t1`								IF	stall	ID	EX	MEM	WB		
`and $t4,$t1,$t2`										IF	ID	EX	MEM	WB	
`or $t5,$t3,$t4`											IF	ID	EX	MEM	WB

Answer. $5 + (8-1) + 3 =$ **15** cycles.

(iii) First branch taken: executes `lw, add, beq, and, or`, so $n = 5$, with 2 stalls + 1 flush.

	1	2	3	4	5	6	7	8	9	10	11	12
`lw $t0,0($s0)`	IF	ID	EX	MEM	WB							
`add $t1,$t0,$s1`		IF	ID	stall	EX	MEM	WB					
`beq $t1,$zero,L1`			IF	stall	ID	stall	EX	MEM	WB			
`add $t2,$t1,$s2`					IF	stall	flush					
`and $t4,$t1,$t2`							IF	ID	EX	MEM	WB	
`or $t5,$t3,$t4`								IF	ID	EX	MEM	WB

Answer. $5 + (5 - 1) + 2 + 1 = \mathbf{12}$ cycles.

(iv) First not taken, second taken: executes `lw, add, beq, add, bne, or`, so $n = 6$, with 3 stalls + 1 flush.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
`lw $t0,0($s0)`	IF	ID	EX	MEM	WB									
`add $t1,$t0,$s1`		IF	ID	stall	EX	MEM	WB							
`beq $t1,$zero,L1`			IF	stall	ID	stall	EX	MEM	WB					
`add $t2,$t1,$s2`					IF	stall	ID	EX	MEM	WB				
`bne $t2,$zero,L2`							IF	ID	stall	EX	MEM	WB		
`sub $t3,$t2,$t1`								IF	stall	flush				
`or $t5,$t3,$t4`										IF	ID	EX	MEM	WB

Answer. $5 + (6 - 1) + 3 + 1 = \mathbf{14}$ cycles.

31. 2022-23 · Q4(b) · 12 marks · ★★★ High · Theory · L15

2022–2023 • Q4(b) CSE 305 • 25/04/2024 • PL11 **MARKS 12**

Four hardware components have been added in the ID stage of the pipelined datapath design as shown in Figure for Question 4(b) to reduce branch stalls. Write down the names of these four newly added components. Explain how the branch decision and the target address becomes available by the end of the ID stage of the *beq* instruction.

44: and $12, $2, $5	**40:** beq $1, $3, 7	**36:** sub $10, $4, $8	before<1>	before<2>

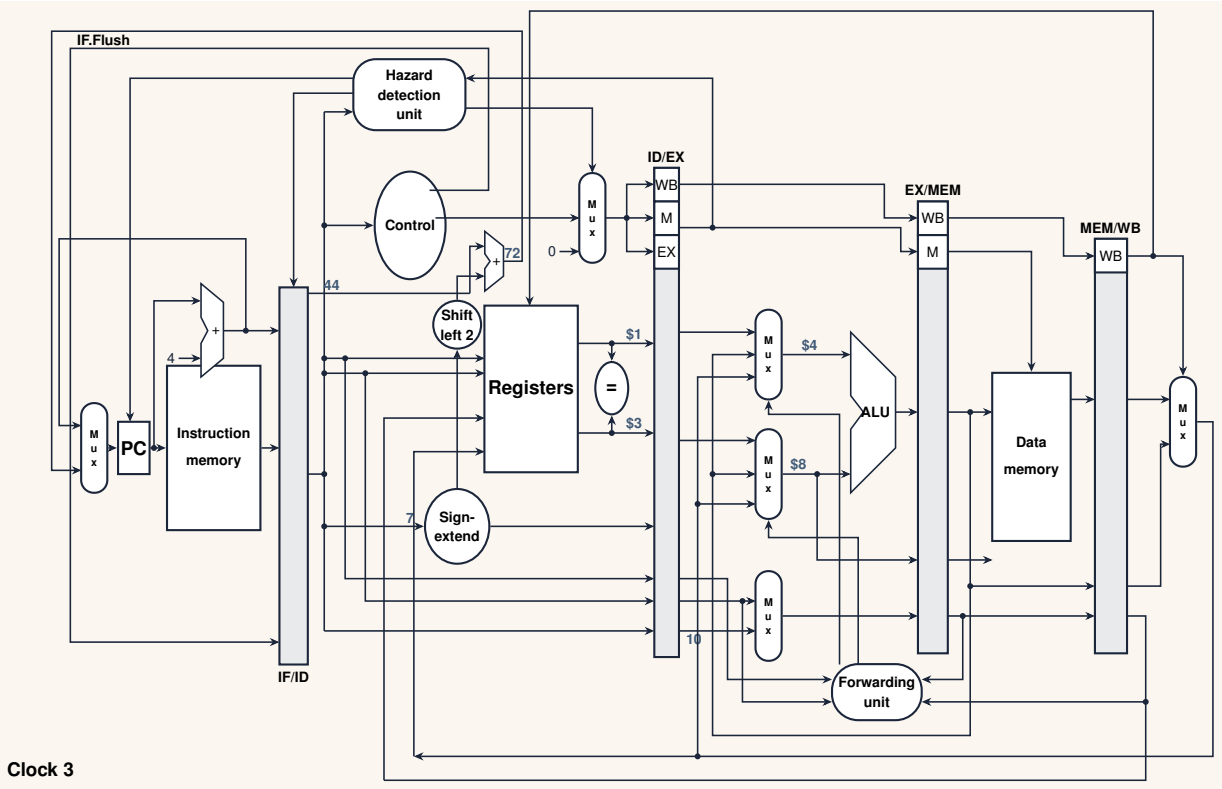


Figure for Question 4(b): Branch Stall Reduction Mechanism

SOLUTION

NOTE The figure is too blurry to read every label. It shows the slides' "Reducing Branch Delay" example (36: `sub`, 40: `beq $1,$3,7`, 44: `and`, ..., 72: `lw $4,50($7)`), i.e. P&H Fig. 4.62. The four components named below are those of that standard figure.

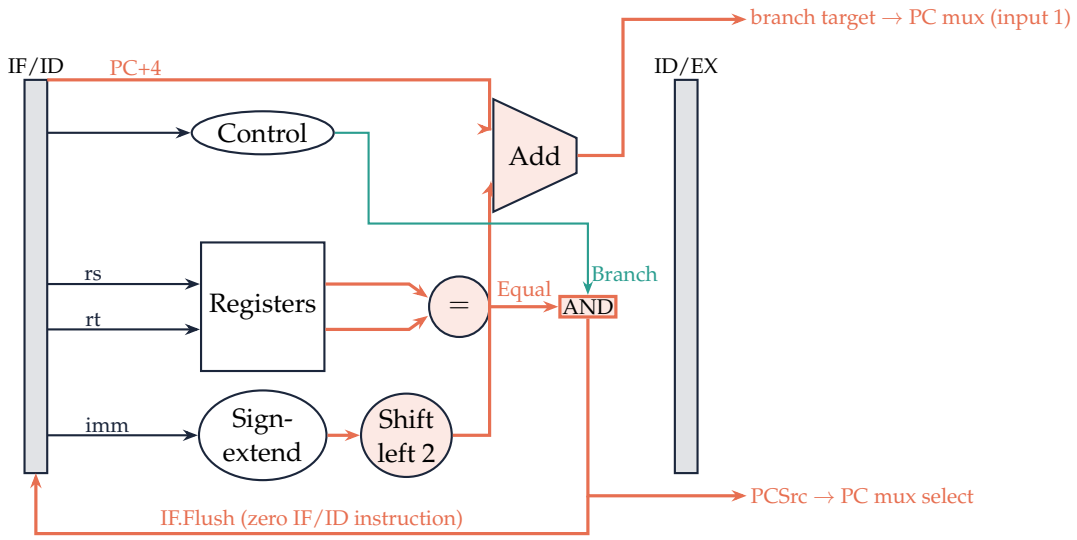
The four components added in ID

(i) **Shift-left-2 unit** for the sign-extended branch offset, moved from EX into ID.

(ii) **Branch target adder**, moved into ID: it adds IF/ID.PC+4 and (offset $\ll$ 2).

(iii) **Register equality comparator** ("=", a bank of XORs feeding a NOR) on the two register-file outputs. It replaces the ALU `Zero` test done in EX.

(iv) **IF.Flush control**: a control line from the branch logic that zeroes the instruction field of IF/ID (turning the wrongly fetched instruction into a nop), together with the AND gate that forms PCSrc = Branch $\wedge$ Equal and drives the PC mux.



How the decision and the target are ready at the end of ID (`beq $1,$3,7` at address 40, clock cycle 3)

- **Target:** IF/ID carries PC+4 = 44. The 16-bit offset 7 is sign-extended and shifted left 2 ($7 \times 4 = 28$). The ID-stage adder gives $44 + 28 = \mathbf{72}$ within cycle 3.
- **Decision:** the register file reads `$1` and `$3` in the first part of ID, and the equality comparator compares them directly. No subtraction in the ALU is needed, which is why it fits in ID. Control decodes `beq` (Branch = 1), and the AND gate produces PCSrc = Branch $\wedge$ Equal.
- **End of cycle 3 (taken):** the PC mux loads 72 instead of 48. IF.Flush zeroes IF/ID, so the `and` at 44 already fetched becomes a nop. In cycle 4 `lw $4,50($7)` at 72 is fetched. A taken branch costs only **1 cycle** instead of 3 (decision in MEM).
- The comparator needs its operands earlier, so forwarding paths into ID (from EX/MEM, MEM/WB) and stalls for a just-computed or just-loaded operand are required. These are the "data hazards for branches" cases.

Answer. Added in ID: shift-left-2, branch-target adder, register equality comparator, and the IF.Flush control (with the Branch $\wedge$ Equal gate driving the PC mux). The target PC+4+(imm$\ll$2) and the equality test both finish in ID, so the PC is redirected at the end of ID and only the one instruction in IF is flushed (1-cycle penalty).

32. 2022-23 · Q5 · 15 marks · ★★★ High · Theory · L15

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2022–2023 • Q5 CSE 209 • 13/01/2025 • PL08 **MARKS 15**

Consider the following code snippet:

```
beq $1, $2, TARGET   # branch is/will be taken for this one
lw  $3, 40($4)
add $2, $3, $4
sw  $2, 40($4)

TARGET:
or $1,$1,$2
```

Assume that branch decision is checked using the ALU zero signal (i.e., no separate hardware is

placed to decide early). Now can an attempt to flush and stall the pipeline occur simultaneously? If yes, explain with the help of an illustration.
Will there be an issue if flush and stall signals occur simultaneously? If there is an issue, you need to provide a solution. If there is no issue, you need to clearly explain why not.

2018–2019 • Q2(a) CSE 305 • 29/10/2019 • PL08 **MARKS 10**

Consider the following code snippet:

```
        beq $1, $2, TARGET   # branch is/will be taken for this one
        lw  $3, 40($4)
        add $2, $3, $4
        sw  $2, 40($4)
TARGET: or  $1, $1, $2
```

Assume that branch decision is checked using the ALU zero signal (i.e., no separate hardware is placed to decide early). Now can an attempt to flush and stall the pipeline occur simultaneously? If yes, then will there be an issue? Explain with the help of a single cycle illustration. If there is an issue, you need to provide a solution. If there is no issue, you need to clearly explain why not.

SOLUTION

Setting

There is no early-branch hardware, so the pipeline is the standard one of the slides ("Branch Hazards: if branch outcome determined in MEM"). The ALU produces `Zero` in EX, it is latched in EX/MEM, and `PCSrc` = `Branch` ∧ `Zero` is formed in **MEM**. When the branch is taken, the three younger instructions (in IF, ID, EX) are flushed by zeroing their control values. Predict-not-taken: `lw`, `add`, `sw` are fetched behind the `beq`.

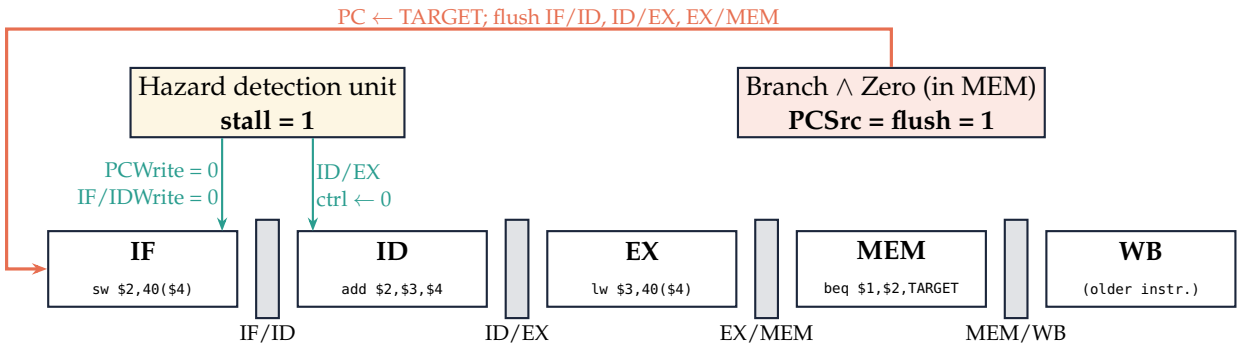
Yes — both happen in clock cycle 4

	1	2	3	**4**	5	6
`beq $1,$2,TARGET`	IF	ID	EX	MEM	WB	
`lw $3,40($4)`		IF	ID	EX		
`add $2,$3,$4`			IF	ID		
`sw $2,40($4)`				IF		

In cycle 4:

- `lw $3` is in EX (ID/EX.MemRead = 1, ID/EX.Rt = `$3`) and `add $2,$3,$4` is in ID (IF/ID.Rs = `$3`). The **hazard detection unit** sees a load-use hazard, so it asserts **stall**: PCWrite = 0, IF/IDWrite = 0, and a bubble goes into ID/EX.
- `beq` is in MEM with Zero = 1, so PCSrc = 1. The **branch logic** asserts **flush**: load PC ← TARGET, and zero IF/ID, the ID/EX controls and the EX/MEM controls (kill `sw`, `add`, `lw`).

Single-cycle illustration (cycle 4):



conflict: stall says "hold PC and IF/ID", flush says "load PC with TARGET and clear IF/ID"

Is there an issue? Yes

If the stall wins (PCWrite = 0, IF/IDWrite = 0), then:

- the PC is *not* loaded with TARGET. The `beq` leaves MEM in this cycle, so the taken branch is lost for good;
- IF/ID is held, not cleared, so the wrong-path `add` (and then `sw`) is decoded again and executes after the bubble. It writes `$2` and memory, which the program never intended.

The result is incorrect execution: `or` at TARGET is reached late or not at all, and wrong-path instructions change the state.

Solution: give the flush (the older instruction) priority over the stall

The stall exists only to help `add`, which is being squashed anyway. So the hazard detection unit's stall is suppressed whenever a taken branch is flushing:

$$\texttt{stall}_{\text{eff}} = \texttt{stall} \wedge \overline{\texttt{PCSrc}}, \qquad \texttt{PCWrite} = \texttt{IF/IDWrite} = \overline{\texttt{stall}_{\text{eff}}} = \overline{\texttt{stall}} \vee \texttt{PCSrc},$$
$$\texttt{IF.Flush} = \texttt{ID.Flush} = \texttt{EX.Flush} = \texttt{PCSrc}.$$

In hardware this is one AND gate (with PCSrc inverted) on the hazard unit's output, or equivalently a priority rule in the control: flush > stall. Zeroing the ID/EX controls is needed by both actions, so there is no conflict there.

	1	2	3	4	5	6	7	8	9
`beq $1,$2,TARGET`	IF	ID	EX	MEM	WB				
`lw $3,40($4)`		IF	ID	EX	flush				
`add $2,$3,$4`			IF	ID	flush				
`sw $2,40($4)`				IF	flush				
`or $1,$1,$2`					IF	ID	EX	MEM	WB

Answer. Yes. In cycle 4 the load-use hazard (`lw $3`→`add`) asks for a stall while the taken `beq` in MEM asks for a flush. If the stall wins, the branch target is never loaded and wrong-path instructions execute. Fix: let the flush override the stall ($\texttt{stall} \wedge \overline{\texttt{PCSrc}}$), so the PC loads TARGET and the three younger instructions are squashed. `or` is fetched in cycle 5 (3-cycle branch penalty).

33. 2022-23 · Q7 · 15 marks · ★★★ High · Theory · L15

2022–2023 • Q7 CSE 209 • 13/01/2025 • PL09 **MARKS 15**

Read the following conversation between three friends.
Chandler: In the pipelined version of MIPS implementation, every jump instruction would need a 1-cycle stall.
Joey: If we just move the control unit in IF stage, we won't need any stall for jump instruction.

Ross: That would cause other problems. I have a better solution to avoid stalls (most of the time) which wouldn't cause any side effect problem.
Now answer the following:

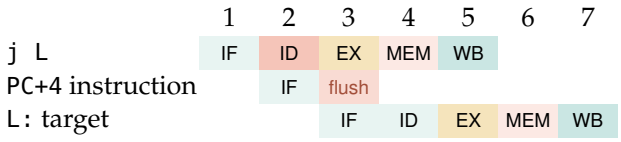
(i) Do you agree with Chandler's statement? Justify your answer.

(ii) If you implement Joey's solution what would be the problem? Explain your answer.

(iii) Can you rediscover Ross's solution, which avoids the problem present in Joey's solution? Provide a brief explanation of your solution.

SOLUTION

(i) Chandler — yes, for the standard 5-stage pipeline. A `j` is recognised only when the control unit decodes it in **ID**. Its target $\{\text{PC+4}[31{:}28],\ \text{address} \times 4\}$ is also formed there. While `j` was being fetched, IF had no way of knowing that it was a jump, so it fetched PC+4. That instruction has to be flushed (IF.Flush), which loses one cycle on *every* jump, since a jump is always taken. (Strictly it is a flush/bubble, not a data-hazard stall, but the cost is the same: 1 cycle.) The only exception is a delayed-branch ISA, where a useful instruction placed in the delay slot hides the cycle.

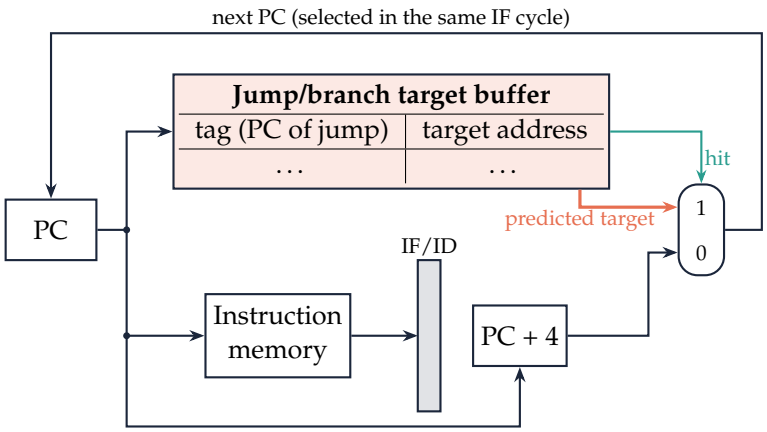


(ii) Joey's idea — decode in IF. The instruction word only comes out of instruction memory at the *end* of IF. Decoding it, forming the jump target and steering the PC mux in the same cycle puts IM access + control decode + target mux + PC set-up all on one combinational path:

- IF becomes by far the longest stage. The clock period is set by the slowest stage, so **every instruction** (not just the 2–3% that are jumps) runs slower, and the pipeline is no longer balanced. The loss is much larger than the one cycle saved per jump.
- The control signals produced in IF must now also be carried in IF/ID (a wider pipeline register). The hazard logic that zeroes control on a stall or flush (which lives in ID) has to reach back into IF, which complicates stalling: a stalled or flushed instruction may already have changed the PC.

(iii) Ross's solution — predict the jump target in IF with a Branch/Jump Target Buffer.

- Add a small cache (BTB) indexed by the *PC of the instruction being fetched*. Each entry holds a tag (the PC of a jump/taken branch) and its target address.
- In IF, look up the BTB *in parallel* with the instruction-memory access. This needs no decoding and does not lengthen the IF path much. On a hit, the next PC is the stored target, so the target is fetched in the very next cycle with **0 penalty**.
- The first time a jump is seen (or after it is evicted) the BTB misses. The jump then costs the normal 1-cycle flush and its target is written into the BTB. This is why it avoids the stall "most of the time".
- No side effect: the clock period is unchanged, and the decode in ID still checks the target. A wrong BTB prediction (e.g. an aliased entry) is simply flushed like a misprediction.



Answer. (i) Agree: a jump is identified and its target formed only in ID, so the fetch behind it is always flushed (1 cycle per jump, unless a delay slot is used). (ii) Decoding in IF puts IM access + decode + target selection in one stage, which lengthens the clock for every instruction and complicates stalling and flushing. (iii) A jump (branch) target buffer looked up with the PC in IF supplies the target in the same cycle: 0 penalty on a hit, the normal 1 cycle on a miss, and no change to the cycle time.

34. 2021-22 · Q1(c) · 10 marks · ★★★ High · Theory · L15

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2021–2022 • Q1(c) CSE 305 • 06/04/2023 • PL13 **MARKS 10**

What do you mean by data hazard for branches? Give MIPS code examples with explanations for each of the following cases of data hazard for branches:

(i) Needs no stall to resolve

(ii) Needs one cycle stall to resolve

(iii) Needs two cycle stalls to resolve

2017–2018 • Q2(c) CSE 305 • 23/09/2018 • PL13 **MARKS 5**

Give the MIPS codes for each of the following cases of data hazard for branches:

(i) Needs one cycle stall to resolve

(ii) Needs two cycle stalls to resolve

2013–2014 • Q8(c) CSE 305 • 05/07/2015 • PL13 **MARKS 9**

What do you mean by data hazard for branches? Give examples with MIPS code for each of the following cases of data hazards for branches:

(i) Needs no stall to resolve

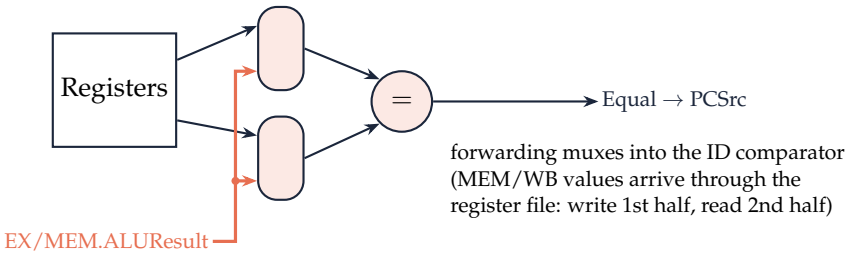
(ii) Needs one cycle stall to resolve

(iii) Needs two cycle stalls to resolve

SOLUTION

Data hazard for branches

To cut the branch penalty to 1 cycle, MIPS moves the branch decision into **ID**: a register comparator there tests `rs` = `rt`. The comparator therefore needs its operands at ID, one stage *earlier* than an ALU instruction needs them (EX). If a compared register is the destination of a recent instruction whose result is not yet available, that is a **data hazard for branches**. It is handled by forwarding into the ID comparator (from EX/MEM or MEM/WB) and, when the value does not exist yet, by stalling the branch in ID.



(stall cells below mean the `beq` repeats ID while bubbles go ahead.)

(i) No stall — the producer is the 2nd or 3rd preceding ALU instruction

```
add $4, $5, $6
add $1, $2, $3
sub $7, $8, $9       # independent instruction
beq $1, $4, target
```

When `beq` is in ID (cycle 5), `add $1` is in MEM, so `$1` is forwarded from EX/MEM into the comparator. `add $4` is in WB, so `$4` is written in the first half and read in the second. No stall.

	1	2	3	4	5	6	7	8
`add $4,$5,$6`	IF	ID	EX	MEM	WB			
`add $1,$2,$3`		IF	ID	EX	MEM	WB		
`sub $7,$8,$9`			IF	ID	EX	MEM	WB	
`beq $1,$4,target`				IF	ID	EX	MEM	WB

(ii) One stall — the producer is the immediately preceding ALU instruction, or a load 2 instructions back

```
lw  $1, addr
add $4, $5, $6
beq $1, $4, target   # $4 from the add just before; $1 from the lw 2 back
```

In cycle 4 `beq` is in ID, but `add $4` is still in EX and `lw $1` is in MEM, so neither value exists at the start of ID. After 1 stall (cycle 5), `$4` is forwarded from EX/MEM and `$1` comes through the register file (lw's WB).

	1	2	3	4	5	6	7	8
`lw $1,addr`	IF	ID	EX	MEM	WB			
`add $4,$5,$6`		IF	ID	EX	MEM	WB		
`beq $1,$4,target`			IF	ID	stall	EX	MEM	WB

(iii) Two stalls — the producer is the immediately preceding load

```
lw  $1, addr
beq $1, $0, target
```

The loaded value exists only at the end of `lw`'s MEM (cycle 4). `beq` (ID in cycle 3) must wait until cycle 5, when `$1` is written in the first half of WB and read by the comparator. That is 2 stalls.

	1	2	3	4	5	6	7	8
`lw $1,addr`	IF	ID	EX	MEM	WB			
`beq $1,$0,target`		IF	ID	stall	stall	EX	MEM	WB

Answer. A branch compared in ID needs its registers one stage early. 0 stalls if the producer is an ALU instruction 2–3 instructions back (forwarding / register file). 1 stall if it is the immediately preceding ALU instruction or a load 2 back. 2 stalls if it is the immediately preceding load.

35. 2023-24 · Q1(c) · 10 marks · ★★★ High · Theory · L16

REPEATED ×6 Same question in 6 papers — stacked below, solved once.

2023–2024 • Q1(c) CSE 209 • 29/04/2026 • PL04 — **MARKS 10**

Consider the following code:

```
outer:
        // some code
inner:
        // some code
beq $t0, $zero, inner
// some code
beq $t1, $zero, outer
```

Explain the problem with the above code if we use 1 bit branch predictor? How can you solve that?
"Even with a perfect branch predictor, 1-cycle penalty for a taken branch" – why does this penalty occur, and how can it be eliminated?

2021–2022 • Q1(d) CSE 305 • 06/04/2023 • PL04 — **MARKS 5**

Consider the following code:

```
outer:
        // some code
inner:
        // some code
        beq $t0, $zero, inner
        // some code
        beq $t1, $zero, outer
```

What is the problem with the above code if we use 1 bit branch predictor? How can you solve that?

2017–2018 • Q1(d) CSE 305 • 23/09/2018 • PL04 — **MARKS 5**

"Even with a perfect branch predictor, 1-cycle penalty for a taken branch". Why is that? How to solve this?

2017–2018 • Q2(d) CSE 305 • 23/09/2018 • PL04 — **MARKS 5**

Consider the following code:

```
outer:
        // some code
inner:
        // some code
        beq $t0, $zero, inner
        // some code
        beq $t1, $zero, outer
```

What is the problem with the above code if we use 1 bit branch predictor? How can you solve that?

2015–2016 • Q2(e) CSE 305 • 18/01/2017 • PL04 **MARKS 5**

"Even with a perfect branch predictor, 1-cycle penalty for a taken branch". Why is that? How to solve this?

2013–2014 • Q8(d) CSE 305 • 05/07/2015 • PL04 **MARKS 4**

Consider the following code:

```
outer:
        // some code
inner:
        // some code
beq $t0, $zero, inner
// some code
beq $t1, $zero, outer
```

What is the problem with the above code if we use 1 bit branch predictor? How you can solve that? Explain

SOLUTION

Part 1 — the 1-bit predictor problem (2023–24, 2021–22, 2017–18 Q2(d), 2013–14)

A 1-bit predictor (branch history table) stores only the *last* outcome of each branch and predicts the same outcome next time. On a misprediction it flushes and flips the bit.

The inner-loop branch `beq $t0,$zero,inner` is taken on every iteration except the last. Say the inner loop runs N times per outer iteration: the branch goes T,T,...,T,NT.

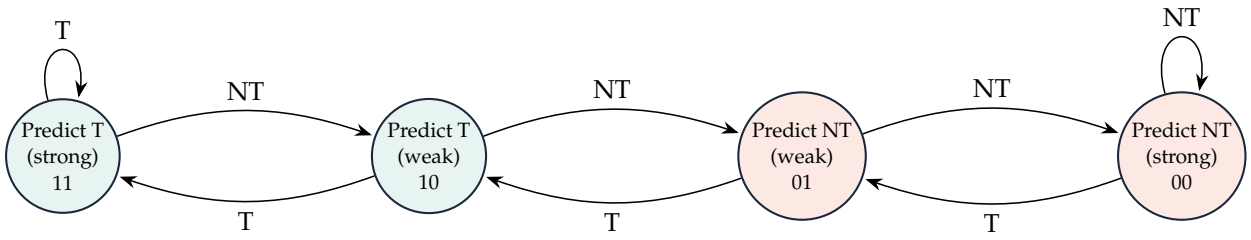
- **Last inner iteration:** the bit says T (from the previous iteration), but the branch falls through. That is misprediction 1, and the bit becomes NT.
- The outer loop then branches back and the inner loop starts again. **First inner iteration:** the bit still says NT, but the branch is taken. That is misprediction 2.

So the inner-loop branch is **mispredicted twice per pass** even though it is "almost always taken". Accuracy is $(N-2)/N$; for example, $N = 10$ gives only 80%.

Inner iteration	1	2	…	$N-1$	N	(next pass) 1
Actual	T	T	…	T	NT	T
1-bit prediction	**NT**	T	…	T	**T**	**NT**
Correct?	×	✓	…	✓	×	×

Solution: 2-bit predictor. The prediction changes only after *two successive* mispredictions. The single NT at loop exit moves the state from "strongly taken" to "weakly taken", and the prediction

is still T. So the first iteration of the next pass is predicted correctly: only **1 misprediction per pass**, giving accuracy $(N-1)/N$ (90% for $N = 10$).



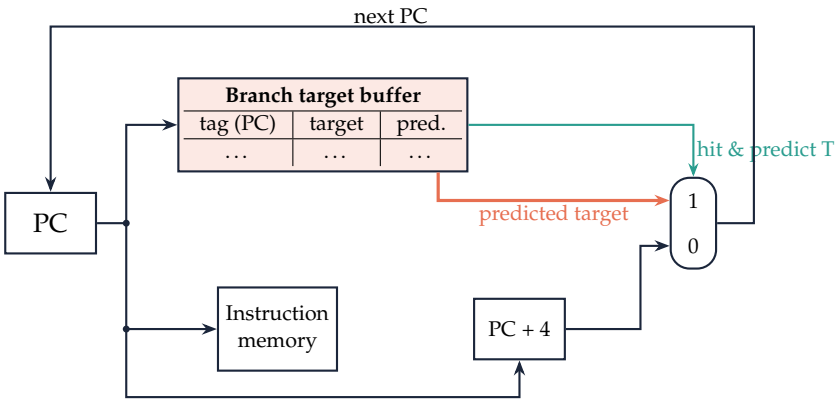
The outer branch behaves the same way, but it runs far less often. (Further improvements are correlating / tournament predictors.)

Part 2 — why a perfect predictor still costs 1 cycle on a taken branch (2023–24, 2017–18 Q1(d), 2015–16)

A predictor only answers *whether* the branch is taken. To fetch the target, the processor also needs *where* it is: $\text{PC} + 4 + (\text{sign-ext(imm)} \ll 2)$. In the MIPS pipeline that target is computed by the branch adder in the **ID** stage, once the instruction has been fetched and decoded. During the cycle in which the branch itself is fetched (IF), nobody yet knows it is a branch or what its offset is. So the next fetch can only go to PC + 4, and that slot is lost when the branch is taken:

	1	2	3	4	5	6
`beq` (predicted T, correct)	IF	ID	EX	MEM	WB	
`PC+4` instruction		IF	flush			
target instruction			IF	ID	EX	MEM

Elimination — branch target buffer (BTB): a small cache of target addresses, indexed by the PC *at fetch time*. In IF, the PC is looked up in the BTB in parallel with the instruction-memory access. On a hit for a branch that is predicted taken, the stored target becomes the next PC immediately, so the target is fetched in cycle 2 and the penalty is 0. The real target and outcome are still checked in ID; on a mismatch the pipeline flushes and the BTB entry is updated. (A compiler-filled *delayed branch* slot is the other way to hide this cycle.)



Answer. 1-bit predictor: the inner-loop branch is mispredicted twice per outer iteration (at loop exit and at loop re-entry). A 2-bit (hysteresis) predictor cuts this to one. Perfect predictor: the target address is known only after decode (ID), so one fetch slot is lost on every taken branch. A branch target buffer looked up in IF supplies the target at once and removes that penalty.

36. 2021-22 · Q1(d) · 5 marks · ★★★ High · Theory · L16

Same question - solution already given under #35.

37. 2019-20 · Q8(b) · 8 marks · ★★★ High · Design · L16

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2019–2020 • Q8(b) CSE 305 • 18/09/2021 • PL18 MARKS 8

Draw the finite state machine for a 2-bit dynamic branch prediction scheme.

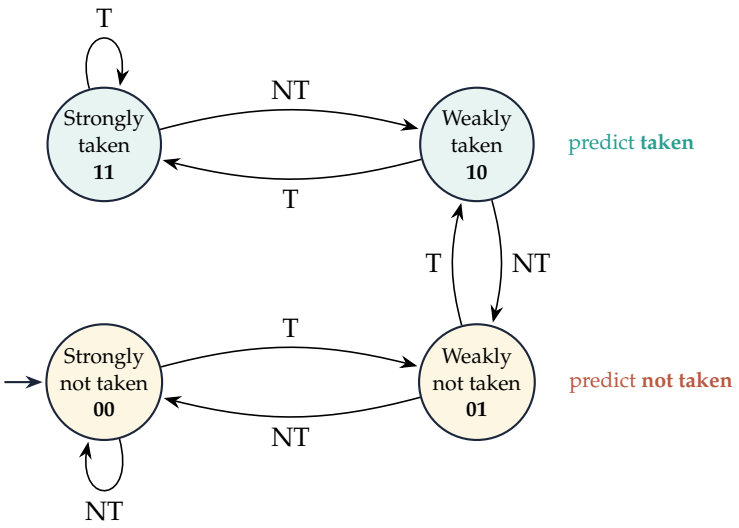
2014–2015 • Q3(b) CSE 305 • 26/01/2016 • PL18 MARKS 10

Briefly describe different dynamic branch prediction techniques for dealing with control hazards.

SOLUTION

2-bit predictor FSM (2019–2020 Q8(b))

Each branch-history-table entry is a 2-bit saturating counter. The prediction changes only after **two successive mispredictions**. A loop branch is therefore mispredicted only once per loop (at the exit), not twice as with a 1-bit predictor.



The top two states predict taken, the bottom two predict not taken. T = actual outcome taken

(counter $+1$), NT = not taken (counter -1), saturating at 11 and 00.

2014–2015 Q3(b): dynamic branch prediction techniques

- **Branch prediction buffer / branch history table (1-bit)**: a small memory indexed by the low bits of the branch address. It stores the last outcome; fetch follows that outcome, and on a misprediction the pipeline is flushed and the bit flipped. Weakness: a loop branch is mispredicted twice per loop (last iteration, and the first iteration the next time).
- **2-bit saturating counter**: the FSM above. The prediction flips only after two consecutive mispredictions, so a loop branch is mispredicted once per loop.
- **Branch target buffer**: a cache of target addresses indexed by the PC at fetch time. On a hit for a branch predicted taken, the target is fetched in the next cycle. This removes the 1-cycle penalty of computing the target, which remains even with a perfect predictor.
- **Correlating (2-level) predictors**: use the outcomes of the last few branches (global history) together with the branch address to pick a counter, so they capture branches that depend on each other.
- **Tournament predictors**: keep a local and a global predictor for each branch plus a selector counter that chooses the one that has been more accurate.

On a wrong guess the CPU stalls while it re-fetches, flushes the wrongly fetched instructions, and updates the history.

38. 2023-24 · Q4(b) · 15 marks · ★★★ High · Trace · L17

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2023–2024 • Q4(b) CSE 209 • 29/04/2026 • PL05 MARKS 15

Consider the following program:

```
Loop: lw   $t0, 0($s1)
      addu $t0, $t0, $s2
      sw   $t0, 0($s1)
      addi $s1, $s1, -4
      bne  $s1, $zero, Loop
```

Schedule the above program for a dual-issue MIPS with the following configuration: Two-issue packets, one ALU/branch instruction, one load/store instruction, ALU/branch then load/store. What will be the IPC for this scheduling? Which technique can further increase the IPC, and How?

2021–2022 • Q2(c) CSE 305 • 06/04/2023 • PL05 **MARKS 10**

Consider the following program:

```
Loop: lw   $t0, 0($s1)
      addu $t0, $t0, $s2
      sw   $t0, 0($s1)
      addi $s1, $s1, -4
      bne  $s1, $zero, Loop
```

Schedule the above program for a dual-issue MIPS with the following configuration: Two-issue packets, one ALU/branch instruction, one load/store instruction, ALU/branch then load/store. What will be the Instructions per Cycle (IPC) for this scheduling? Which technique can increase the IPC even more?

SOLUTION

Scheduling rules of the static dual-issue MIPS (slides)

- A packet = one ALU/branch slot + one load/store slot; an empty slot holds a `nop`.
- No true dependence inside a packet: an ALU result can be used by a load/store only in a *later* packet (EX data hazard). A register may be read and overwritten in the same packet (the read happens in ID, the write in WB).
- Load-use latency is still one cycle: a load in cycle n feeds an ALU instruction in cycle $n + 2$.
- If `addi` moves above a load/store that uses the same base register, adjust the offset (e.g. `sw $t0,0($s1)` becomes `sw $t0,4($s1)`).

Schedule (one ALU/branch + one load/store slot per packet)

	ALU/branch	Load/store	cycle
`Loop:`	`nop`	`lw $t0, 0($s1)`	1
	`addi $s1, $s1, -4`	`nop`	2
	`addu $t0, $t0, $s2`	`nop`	3
	`bne $s1, $zero, Loop`	`sw $t0, 4($s1)`	4

`addu` has to wait for the load (cycle 3). `addi` is moved up into the free cycle 2, so the store now uses offset 4. The store and the branch share the last packet.

Answer. 5 instructions in 4 cycles: IPC $= 5/4 =$ **1.25** (the peak is 2).

How to increase the IPC: loop unrolling with register renaming

Copy the loop body 4 times, so there is one `addi`/`bne` pair for every 4 elements (less loop-control overhead). Give each copy its own temporary register (`$t0`–`$t3`). This is register renaming: it removes the name (anti-)dependences on `$t0`, so the copies can be interleaved freely. (Assume the element count is a multiple of 4.)

	ALU/branch	Load/store	cycle
`Loop:`	`addi $s1, $s1, -16`	`lw $t0, 0($s1)`	1
	`nop`	`lw $t1, 12($s1)`	2
	`addu $t0, $t0, $s2`	`lw $t2, 8($s1)`	3
	`addu $t1, $t1, $s2`	`lw $t3, 4($s1)`	4
	`addu $t2, $t2, $s2`	`sw $t0, 16($s1)`	5
	`addu $t3, $t3, $s2`	`sw $t1, 12($s1)`	6
	`nop`	`sw $t2, 8($s1)`	7
	`bne $s1, $zero, Loop`	`sw $t3, 4($s1)`	8

Answer. Unrolled: 14 instructions in 8 cycles, IPC $= 14/8 = \mathbf{1.75}$. This is closer to 2, but it costs more registers and larger code. Dynamic multiple issue (out-of-order superscalar) can also raise IPC.

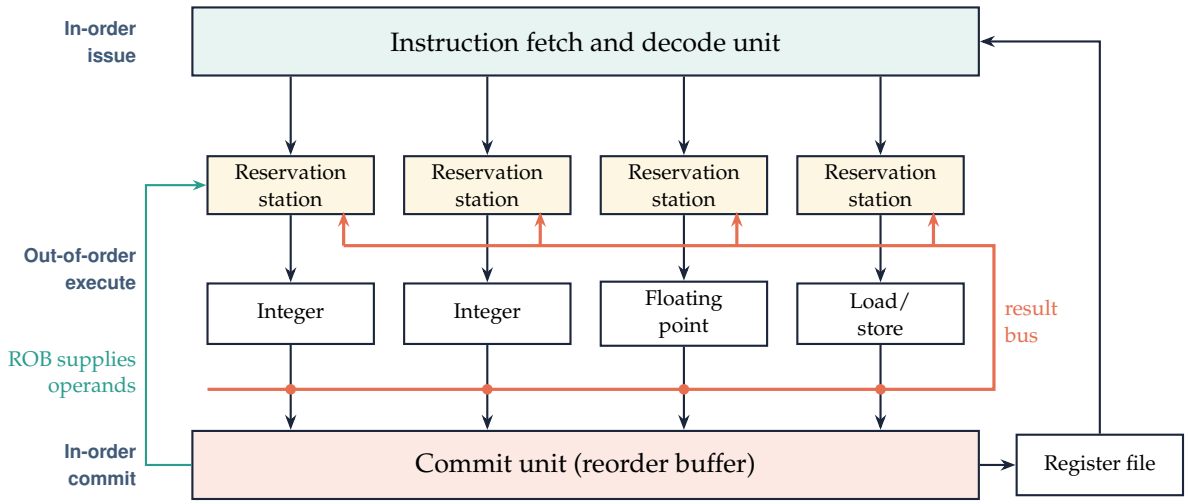
NOTE The slides print the 6th packet as `addu $t3, $t4, $s2`. That is a typo; it should be `addu $t3, $t3, $s2`, as used above.

39. 2021-22 · Q2(b) · 10 marks · ★★★ High · Design · L17

2021–2022 • Q2(b) CSE 305 • 06/04/2023 • PL14 **MARKS 10**

Draw the block diagram of a dynamically scheduled multiple-issue super-scalar CPU.

SOLUTION



- **Fetch & decode unit** fetches several instructions per cycle. It decodes them, renames their registers, and *issues* them in program order to the reservation station of the right unit. Each instruction also gets a reorder-buffer entry.

- **Reservation stations** hold an issued instruction and its operands. They keep the pending operands (or the tag of the unit that will produce them). An instruction is dispatched as soon as its operands are ready, so independent instructions run *out of order* around a stalled one. Example: `sub $s4,$s4,$t3` runs while `addu $t1,$t0,$t2` waits for `lw $t0,20($s2)`.
- **Functional units** (integer, FP, load/store) execute. Each result is broadcast to every reservation station waiting for it (forwarding) and to the commit unit.
- **Commit unit / reorder buffer** buffers the results and writes them to the register file and memory *in program order*. It also supplies operands of instructions that are done but not yet committed, which preserves dependences. On a mispredicted (speculated) branch, the buffered results after the branch are discarded.

40. 2021-22 · Q2(c) · 10 marks · ★★★ High · Trace · L17

Same question - solution already given under #38.

41. 2020-21 · Q5 · 15 marks · ★★★ High · Trace · L17

2020–2021 • Q5 CSE 305 • 24/09/2022 • PL16 **MARKS 15**

You are given the following code snippet and have been tasked to run multiple issues in parallel for faster execution. Refactor the code so that IPC is maximized. Also compute the effective IPC. [Hint: you do not have to limit the code to two issues.]

```
Loop: lw   $t0, 0($s0)
      lw   $t1, 0($s1)
      addu $t0, $t0, $t1
      sw   $t0, 0($s1)
      addi $s0, $s0, -4
      addi $s1, $s1, -4
      bne  $s1, $zero, Loop
```

SOLUTION

Scheduling rules of the static dual-issue MIPS (slides)

- A packet = one ALU/branch slot + one load/store slot; an empty slot holds a `nop`.
- No true dependence inside a packet: an ALU result can be used by a load/store only in a *later* packet (EX data hazard). A register may be read and overwritten in the same packet (the read happens in ID, the write in WB).
- Load-use latency is still one cycle: a load in cycle n feeds an ALU instruction in cycle $n+2$.
- If `addi` moves above a load/store that uses the same base register, adjust the offset (e.g. `sw $t0,0($s1)` becomes `sw $t0,4($s1)`).

Step 1 – plain dual issue (baseline).

	ALU/branch	Load/store	cycle
Loop:	`addi $s0, $s0, -4`	`lw $t0, 0($s0)`	1
	`addi $s1, $s1, -4`	`lw $t1, 0($s1)`	2
	`nop`	`nop`	3
	`addu $t0, $t0, $t1`	`nop`	4
	`bne $s1, $zero, Loop`	`sw $t0, 4($s1)`	5

The 7 instructions take 5 cycles, so IPC $= 1.4$. Unrolling alone does not help much here. Unrolled $\times 4$ the loop has 19 instructions, and 12 of them (8 `lw` + 4 `sw`) need the single load/store slot. That forces ≥ 12 cycles, so IPC $\leq 19/12 \approx 1.58$. As the hint says, we must issue more than two instructions per cycle.

Step 2 – unroll $\times 4$, rename, and use a 4-issue static machine (2 ALU/branch + 2 load/store slots, same latencies as the dual-issue MIPS). Element k of an iteration uses `$t2`k (from `$s0`) and `$t2`$k{+}1$ (from `$s1`). Both pointers are decremented by 16 in cycle 1, so later accesses use offsets $+16, +12, +8, +4$.

ALU/br 1	ALU/br 2	Ld/St 1	Ld/St 2	cyc
`addi $s0,$s0,-16`	`addi $s1,$s1,-16`	`lw $t0,0($s0)`	`lw $t1,0($s1)`	1
`nop`	`nop`	`lw $t2,12($s0)`	`lw $t3,12($s1)`	2
`addu $t0,$t0,$t1`	`nop`	`lw $t4,8($s0)`	`lw $t5,8($s1)`	3
`addu $t2,$t2,$t3`	`nop`	`lw $t6,4($s0)`	`lw $t7,4($s1)`	4
`addu $t4,$t4,$t5`	`nop`	`sw $t0,16($s1)`	`sw $t2,12($s1)`	5
`addu $t6,$t6,$t7`	`nop`	`sw $t4,8($s1)`	`nop`	6
`bne $s1,$zero,Loop`	`nop`	`sw $t6,4($s1)`	`nop`	7

Why 7 cycles is the minimum here: the 8 loads need at least 4 cycles in 2 slots. The last load (cycle 4) gives its `addu` in cycle 6 (load-use), and its `sw` then goes in cycle 7.

Answer. Refactored loop (unroll $\times 4$, renamed `$t0`–`$t7`, 4-wide issue): 19 instructions in 7 cycles per 4 elements. $\text{IPC}_{\text{eff}} = 19/7 \approx \mathbf{2.71}$, compared with 1.4 for the plain dual-issue schedule. Per element, time falls from 5 to $7/4 = 1.75$ cycles.

> **NOTE** The 2020–21 question does not fix the issue width. We assumed a 4-issue static machine (2 ALU/branch + 2 load/store slots), an element count that is a multiple of 4, and arrays at `$s0` and `$s1` that do not overlap.

42. 2022-23 · Q8 · 15 marks · ★★★ High · Design · L18

2022–2023 • Q8 NOT IN SLIDES CSE 209 • 13/01/2025 • CU01 MARKS 15

Make the necessary changes to incorporate precise exception handling in pipelined version of MIPS hardware. You can make the changes in the Figure given in Figure for Question 8 and have that page stapled securely with your answer script. The question page containing the figure must then be signed by the invigilator.
You do not need to draw any input signal for *Hazard detection unit*. If any output signal is needed for incorporating precise exception handling you must draw that. For any other output signal which isn't necessary for exception handling you do not need to draw it.

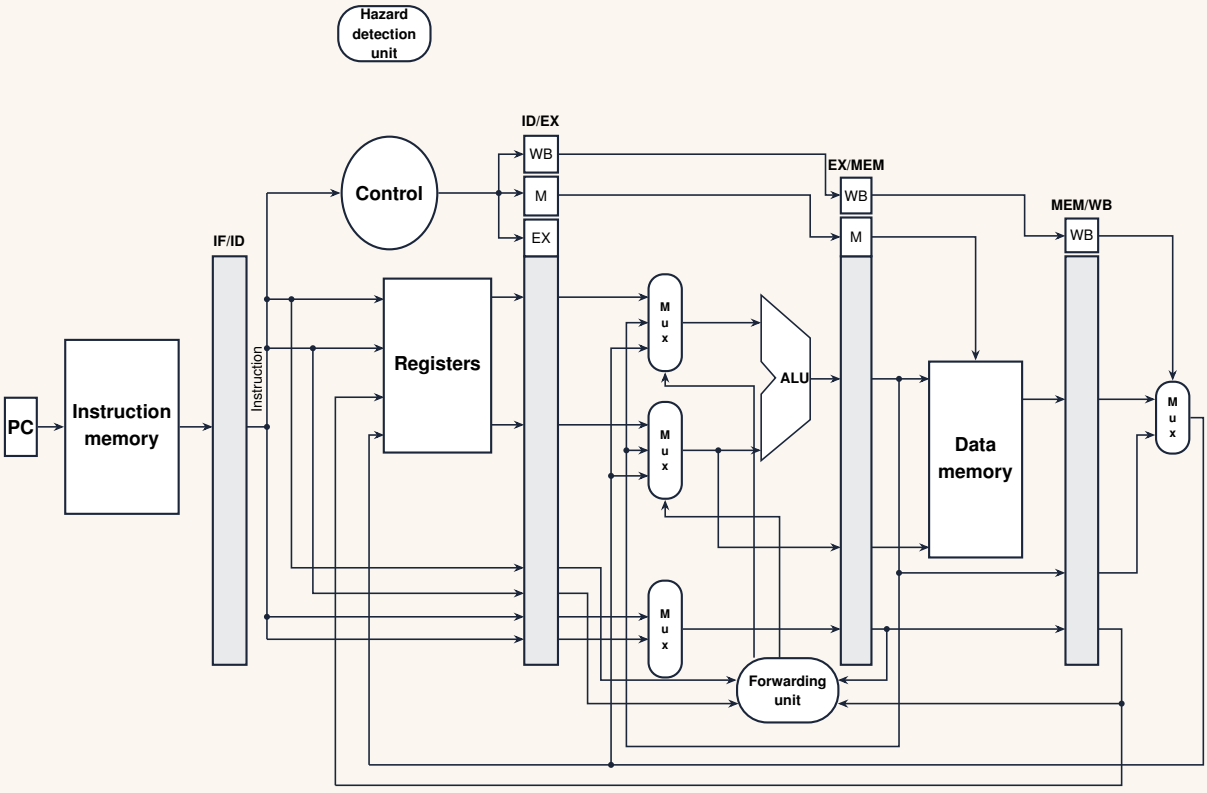


Figure for Question 8

SOLUTION

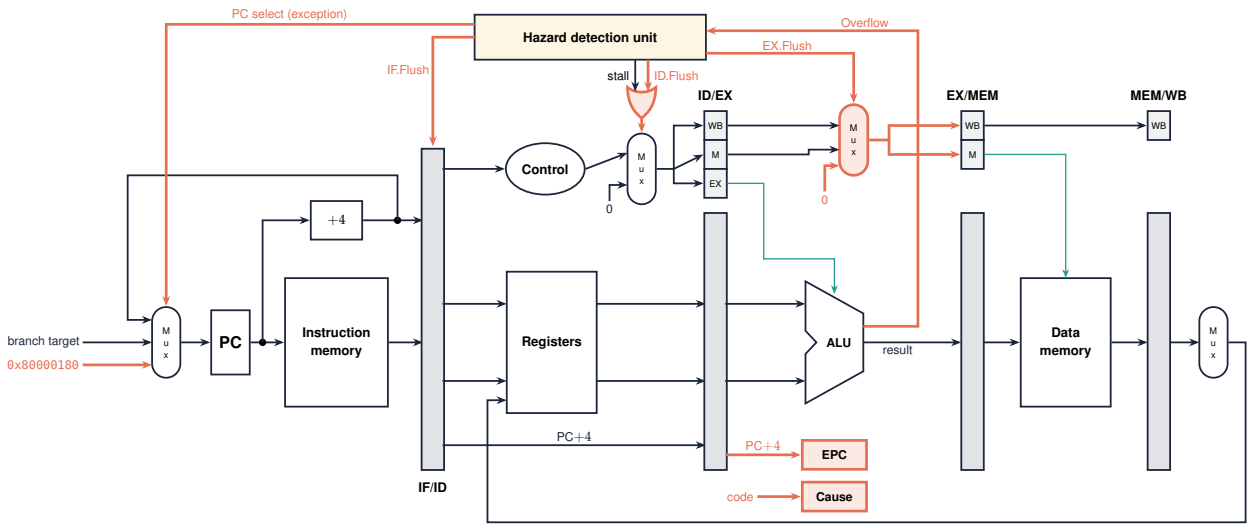
Precise exception: when instruction I raises an exception (e.g. arithmetic overflow of `add` in EX, or an undefined opcode in ID), every instruction *before* I completes, while I and every instruction *after* it leave no trace (no register or memory write). The address of I is saved so that the handler

can restart or report it. Three things are needed:

(i) **Flush** *I* and the younger instructions behind it: **IF.Flush** turns the instruction in IF/ID into a `nop`. **ID.Flush** (ORed with the stall signal) zeros the control bits entering ID/EX. **EX.Flush** zeros the WB and M control bits of *I* itself on its way into EX/MEM, so the overflowing `add` cannot write $rd.

(ii) **Record** the cause and the address: **Cause** register ← exception code (overflow = 12, undefined instruction = 10). **EPC** ← PC+4 of *I*, carried down the pipeline (the handler subtracts 4).

(iii) **Transfer control**: an extra input on the PC mux loads the handler address `0x80000180`.



Coral = hardware and outputs added for exceptions. The forwarding unit and its muxes are unchanged and omitted. Only the hazard-detection outputs needed for exceptions are drawn (plus the existing *stall*).

Output (from the hazard detection / exception logic)	Effect when an exception is detected in EX
IF.Flush	clears IF/ID, so the instruction just fetched becomes a `nop`
ID.Flush (OR stall)	control mux injects 0s into ID/EX, so the instruction in ID becomes a bubble
EX.Flush	muxes zero the WB and M bits of the faulting instruction going to EX/MEM, so no register or memory write happens
EPCWrite, CauseWrite	EPC ← PC+4 of the faulting instruction; Cause ← code
PC select	PC ← `0x80000180` (OS exception handler) on the next clock

Instructions already in MEM and WB are older than the faulting one, so they finish normally. That is what makes the exception *precise*.

3 Ch5 Memory — Chapter 5 — Memory Hierarchy

Notes: CSE209_Ch5_Memory_Hierarchy_Notes.pdf

43. 2023-24 · Q2(b) · 15 marks · ★★★ High · Trace · L20

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2023–2024 • Q2(b) CSE 209 • 29/04/2026 • MH04 MARKS 15

For a direct-mapped cache design with a 32-bit address, the following bits of the address are used to access the cache.

Tag	Index	Offset
31–10	9–5	4–0

Starting from power on, the following byte-addressed cache references are recorded.

0	4	16	132	232	160	1024	30	140	3100	180	2180

(i) What is the cache block size (in words)?

(ii) How many entries does the cache have?

(iii) How many blocks are replaced?

(iv) What is the hit ratio?

(v) List the final state of the cache, with each valid entry represented as ⟨index, tag, data⟩.

2015–2016 • Q4(b) CSE 305 • 18/01/2017 • MH04 MARKS 10

For a direct-mapped cache design with a 32-bit address, the following bits of the address are used to access the cache.

Tag	Index	Offset
31–10	9–5	4–0

What is the cache block size (in words) and how many entries does the cache have? Starting from power on, the following byte-addressed cache references are recorded.

0, 4, 16, 132, 232, 160, 1024, 30, 140, 3100, 180, 2180

How many blocks are replaced and what is the hit ratio?

SOLUTION

Both cards use the same cache and the same trace; the 2023–2024 card additionally asks for the final state (v).



(i), (ii) Block size and number of entries

Offset = 5 bits ⇒ block = $2^5 = 32$ bytes = 8 words. Index = 5 bits ⇒ $2^5 = 32$ entries (lines).

Answer. Block size = 8 words (32 bytes); the cache has 32 entries.

Trace

Block address = $\lfloor A/32 \rfloor$, index = block address mod 32, tag = $\lfloor A/1024 \rfloor$.

Address	Bits 11–10 | 9–5 | 4–0	Block	Index	Tag	H/M	Action
0	00 00000 00000	0	0	0	×	load block 0 (bytes 0–31)
4	00 00000 00100	0	0	0	✓	
16	00 00000 10000	0	0	0	✓	
132	00 00100 00100	4	4	0	×	load bytes 128–159
232	00 00111 01000	7	7	0	×	load bytes 224–255
160	00 00101 00000	5	5	0	×	load bytes 160–191
1024	01 00000 00000	32	0	1	×	**replace** index 0 (tag 0 → 1)
30	00 00000 11110	0	0	0	×	**replace** index 0 (tag 1 → 0)
140	00 00100 01100	4	4	0	✓	
3100	11 00000 11100	96	0	3	×	**replace** index 0 (tag 0 → 3)
180	00 00101 10100	5	5	0	✓	
2180	10 00100 00100	68	4	2	×	**replace** index 4 (tag 0 → 2)

(iii), (iv) Replacements and hit ratio

Answer. 4 blocks are replaced (by 1024, 30, 3100 and 2180). Hits = 4 (4, 16, 140, 180) out of 12 references ⇒ hit ratio = $4/12 = 33.3\%$.

(v) Final state (2023–2024 card)

Index	Valid	Tag	Data
0	1	3	Mem[3072 … 3103]
4	1	2	Mem[2176 … 2207]
5	1	0	Mem[160 … 191]
7	1	0	Mem[224 … 255]
all other indices invalid			

Answer. $\langle 0, 3, \text{Mem}[3072..3103]\rangle$, $\langle 4, 2, \text{Mem}[2176..2207]\rangle$, $\langle 5, 0, \text{Mem}[160..191]\rangle$, $\langle 7, 0, \text{Mem}[224..255]\rangle$.

44. 2022-23 · Q2(b) · 15 marks · ★★★ High · Numerical · L21

2022–2023 • Q2(b) CSE 305 • 25/04/2024 • MH15 **MARKS 15**

How many total bits are required for an **eight-way set associative** cache of 4K blocks, a 4-word block size (i.e., 16 bytes), assuming a 32-bit address? [Show set number and tag bit calculations.]

SOLUTION

Sets and tag

Offset = $\log_2 16 = 4$ bits. Sets = $\dfrac{4096}{8} = 512 = 2^9 \Rightarrow$ index = 9 bits. Tag = $32 - 9 - 4 = 19$ bits.



Total bits

Per block: 128 data +19 tag +1 valid = 148 bits.

$$\text{Total} = 4096 \times 148 = 606{,}208 \text{ bits}; \qquad \text{tag bits alone} = 19 \times 4096 = 77{,}824$$

Answer. 512 sets, 19-bit tag; total = 606,208 bits (≈ 74 KB, of which 64 KB is data).

45. 2022-23 · Q9(b) · 10 marks · ★★★ High · Numerical · L21

2022–2023 • Q9(b) CSE 209 • 13/01/2025 • MH09 **MARKS 10**

Determine the total size of a 4-way set-associative cache with the following specifications. Assume the cache has tag bits, a valid bit, a dirty bit, a protection bit, and the data bits.

- Address length: 48 bits
- Cache (Data only) size: 64 KB
- Block size: 16 words
- The cache has hardware support for the instruction `lb` (load byte).

SOLUTION

Method

Block offset $= \log_2(\text{block bytes})$; sets $=$ blocks / ways; index $= \log_2(\text{sets})$; tag $=$ address $-$ index $-$ offset; total $=$ blocks $\times$ (data $+$ tag $+$ status bits).

Fields

Block $= 16 \times 4 = 64$ B $\Rightarrow$ offset $= 6$ bits (`lb` support means single bytes are addressed, so the 2-bit byte offset is kept). Blocks $= 2^{16}/2^6 = 1024$; sets $= 1024/4 = 256 \Rightarrow$ index $= 8$; tag $= 48 - 8 - 6 = 34$ bits.



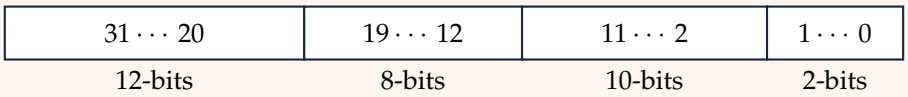
$$\text{bits per line} = 34\,(\text{tag}) + 1\,(\text{V}) + 1\,(\text{D}) + 1\,(\text{prot.}) + 512\,(\text{data}) = 549$$

Answer. Total $= 1024 \times 549 = 562{,}176$ bits $= 70{,}272$ bytes ≈ 68.6 KB (of which 64 KB is data).

46. 2020-21 · Q6 · 15 marks · ★★★ High · Numerical · L21

2020–2021 • Q6 CSE 305 • 24/09/2022 • MH23 **MARKS 15**

Answer the questions below for the following memory address subdivision and a 2-way set associative LRU cache:



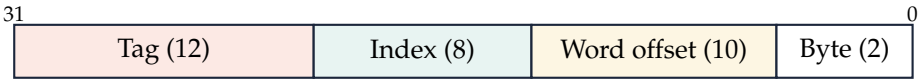
(a) How many distinct blocks of data can there be in the cache?

(b) In which cache index will the decimal address 24092022 map to?

(c) If a cache index is valid, what is the probability that one of its elements will have to be replaced in the next request? Assume each memory address has equal likelihood of being requested and each element in a cache index has equal likelihood of being filled.

SOLUTION

Reading the fields from the left as **tag**, **index (set)**, **block (word) offset**, **byte offset**:



So a block holds 2^{10} words $= 2^{12}$ bytes $= 4\,\text{KB}$, there are $2^8 = 256$ sets, and each set has 2 ways.

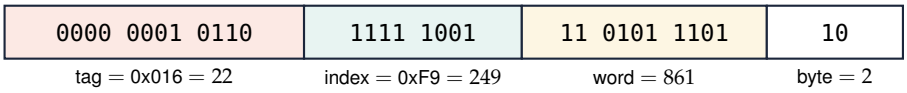
(a) Number of distinct blocks in the cache

$$\#\text{blocks} = \#\text{sets} \times \text{associativity} = 2^8 \times 2 = 512.$$

Answer. 512 blocks can be resident at once (256 sets $\times$ 2 ways; data capacity $512 \times 4\,\text{KB} = 2\,\text{MB}$).

(b) Index of address 24092022

$24092022_{10} = \texttt{0x016F9D76}$. Splitting the 32-bit pattern:



Equivalently, index $= \lfloor 24092022/2^{12} \rfloor \bmod 2^8 = 5881 \bmod 256 = 249$.

Answer. The address maps to cache index (set) **249** (11111001_2), with tag 22.

(c) Probability that a valid set must replace an element on the next request

A replacement is needed only if both conditions hold (the set is full because it is "valid", and the two ways hold two different tags):

- the request maps to this set: $P = 1/2^8 = 1/256$ (every address equally likely, index bits uniform);
- it misses, i.e. its tag is neither of the 2 stored tags: $P = (2^{12} - 2)/2^{12} = 4094/4096$.

$$P(\text{replace}) = \frac{1}{256} \times \frac{4094}{4096} = \frac{4094}{1\,048\,576} \approx 3.90 \times 10^{-3}.$$

The victim is the LRU way of the two; since each element is equally likely to have been filled, each way is the victim with probability $\frac{1}{2}$ ($\approx 1.95 \times 10^{-3}$ each).

Answer. $P \approx 0.0039$ (about 0.39%). Given that the request falls in this set, the replacement probability is $4094/4096 \approx 0.9995$.

NOTE "Valid index" is taken to mean that both ways of the set hold valid blocks. If it means only that at least one way is filled, each way filled independently with probability $\frac{1}{2}$, then the set is full with probability $\frac{1/4}{3/4} = \frac{1}{3}$, and the answer becomes $\frac{1}{3} \times 3.90 \times 10^{-3} \approx 1.30 \times 10^{-3}$.

47. 2019-20 · Q6(b) · 15 marks · ★★★ High · Numerical · L21

2019–2020 • Q6(b) CSE 305 • 18/09/2021 • MH27 **MARKS 15**

Assuming a cache of 4K blocks, a 4-word block size, and a 32-bit address, find the *total* number of tag bits for a cache that is **eight-way set associative**.

SOLUTION

Block $= 4$ words $= 16$ B $\Rightarrow$ offset $= 4$ bits. Sets $= 4096/8 = 512 = 2^9 \Rightarrow$ index $= 9$ bits. Tag $= 32 - 9 - 4 = 19$ bits.



Every one of the 4096 blocks stores its own tag:

Answer. Total tag bits $= 19 \times 4096 = 77{,}824$ bits ($= 76\,\text{Kbit}$).

48. 2019-20 · Q7(a) · 20 marks · ★★★ High · Numerical · L21

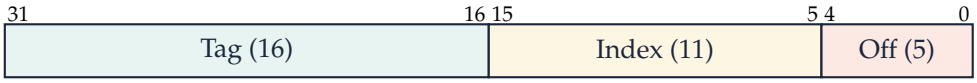
2019–2020 • Q7(a) CSE 305 • 18/09/2021 • MH28 **MARKS 20**

How many *total* bits are required for a **direct-mapped** cache with 64 KB of data and 8-word blocks (i.e., 32 bytes), assuming a 32-bit address?

SOLUTION

Fields

Offset $= \log_2 32 = 5$ bits. Blocks $= 2^{16}/2^5 = 2048 = 2^{11} \Rightarrow$ index $= 11$ bits. Tag $= 32 - 11 - 5 = 16$ bits.



Total

Each block: $8 \times 32 = 256$ data bits $+16$ tag $+1$ valid $= 273$ bits.

$$\text{Total} = 2^{11} \times 273 = 2048 \times 273 = 559{,}104 \text{ bits}$$

Answer. 559,104 bits $= 546$ Kbit ≈ 68.25 KB, i.e. about $1.07\times$ the 64 KB of data.

49. 2022-23 · Q2(a) · 20 marks · ★★★ High · Trace · L22

2022–2023 • Q2(a) CSE 305 • 25/04/2024 • MH14 **MARKS 20**

Assume that there are two small caches, each consisting of eight (8) blocks. One cache is direct-mapped, and the other is a two-way set-associative cache. For each cache organization.

(i) Construct the following table for each cache organization for the following sequence of memory block addresses: 32, 36, 35, 32, 48, 32, 51 and 35. Note that you are allowed to subdivide the column "Cache content after access" as needed. Also, when it is necessary to replace an existing cache entry on a cache miss, follow the least recently used (LRU) replacement policy.

Address of Memory Block Accessed	Cache Index	Hit or Miss	Cache content after access

Table 1: Table for Question 2(a)

(ii) Compute the hit ratio from the constructed tables in (i) for each cache organization.

SOLUTION

Method (slides: "Associativity Example")

Direct-mapped: index = block address mod 8. Two-way: 4 sets, set = block address mod 4; on a miss fill an empty way, else replace the **LRU** block of the set. Cells give the block number held in each slot after the access (✓ hit, × miss; Sk = a way of set k).

Direct-mapped

Blk	Idx	H/M	0	1	2	3	4	5	6	7
32	0	×	32							
36	4	×	32				36			
35	3	×	32			35	36			
32	0	✓	32			35	36			
48	0	×	48			35	36			
32	0	×	32			35	36			
51	3	×	32			51	36			
35	3	×	32			35	36			

misses 7, hits 1

Two-way set associative

Blk	Set	H/M	S0	S0	S1	S1	S2	S2	S3	S3
32	0	×	32							
36	0	×	32	36						
35	3	×	32	36					35	
32	0	✓	32	36					35	
48	0	×	32	48					35	
32	0	✓	32	48					35	
51	3	×	32	48					35	51
35	3	✓	32	48					35	51

misses 5, hits 3

Answer. (ii) Hit ratio: direct-mapped = $1/8 = 12.5\%$; two-way set associative = $3/8 = 37.5\%$ (48 replaces the LRU block 36, so 32 and 35 survive).

50. 2021-22 · Q4(a) · 15 marks · ★★★ High · Trace · L22

2021–2022 • Q4(a) CSE 305 • 06/04/2023 • MH22 **MARKS 15**

Assume there are three small caches, each consisting of four one-word blocks and uses LRU replacement policy. One cache is fully associative, a second is two-way set-associative, and the third is direct-mapped. Find the number of misses for each cache organization given the following sequence of block addresses:

1, 3, 2, 4, 5, 1, 3, 1, 3, 5, 3, 2

SOLUTION

Method (slides: "Associativity Example")

Direct-mapped: index = block mod 4. Two-way: 2 sets, set = block mod 2. Fully associative: any of 4 ways. On a miss fill an empty way, else replace the **LRU** block of the set. Cells give the block held in each slot after the access (Wk = way k; Sk = a way of set k).

Fully associative

Blk	H/M	W0	W1	W2	W3

Two-way set associative

Blk	Set	H/M	S0	S0	S1	S1

Direct-mapped

Blk	Idx	H/M	0	1	2	3

1	×	1			
3	×	1	3		
2	×	1	3	2	
4	×	1	3	2	4
5	×	5	3	2	4
1	×	5	1	2	4
3	×	5	1	3	4
1	✓	5	1	3	4
3	✓	5	1	3	4
5	✓	5	1	3	4
3	✓	5	1	3	4
2	×	5	1	3	2

misses 8, hits 4

1	1	×			1	
3	1	×			1	3
2	0	×	2		1	3
4	0	×	2	4	1	3
5	1	×	2	4	5	3
1	1	×	2	4	5	1
3	1	×	2	4	3	1
1	1	✓	2	4	3	1
3	1	✓	2	4	3	1
5	1	×	2	4	3	5
3	1	✓	2	4	3	5
2	0	✓	2	4	3	5

misses 8, hits 4

1	1	×				1	
3	3	×				1	3
2	2	×			1	2	3
4	0	×	4	1	2	3	
5	1	×	4	5	2	3	
1	1	×	4	1	2	3	
3	3	✓	4	1	2	3	
1	1	✓	4	1	2	3	
3	3	✓	4	1	2	3	
5	1	×	4	5	2	3	
3	3	✓	4	5	2	3	
2	2	✓	4	5	2	3	

misses 7, hits 5

Answer. Misses: fully associative 8, two-way 8, direct-mapped 7. (Here LRU in the fully associative cache evicts exactly the blocks about to be reused, so higher associativity does not help.)

51. 2019-20 · Q6(a) · 20 marks · ★★★ High · Trace · L22

2019–2020 • Q6(a) CSE 305 • 18/09/2021 • MH26 **MARKS 20**

Assume there are *three* small caches, each consisting of eight (8) one-word blocks. One cache is **direct-mapped**, a second is **two-way set-associative**, and the third is **four-way set-associative**. For each cache organization, complete the following table for the following sequence of memory block addresses: **0**, **12**, **16**, **18**, and **0**. You are allowed to subdivide the third column "Contents of cache blocks after reference" as needed.
Note that when it is necessary to replace an existing cache entry on a cache miss, follow the least recently used (LRU) replacement rule to determine which cache entry to replace and show the cache content after the replacement.

Address of memory block accessed	Hit or Miss	Contents of cache blocks after reference
0		
12		
16		
18		
0		

SOLUTION

Method (slides: "Associativity Example")

DM: index $= b \bmod 8$ $(0,4,0,2,0)$. 2-way: 4 sets, set $= b \bmod 4$ $(0,0,0,2,0)$. 4-way: 2 sets, set $= b \bmod 2$ (all set 0). On a miss fill an empty way, else replace the **LRU** block of the set. Cells give the block held in each slot (Sk = a way of set k).

Direct-mapped

Blk	Idx	H/M	0	1	2	3	4	5	6	7
0	0	×	0							
12	4	×	0				12			
16	0	×	16				12			
18	2	×	16		18		12			
0	0	×	0		18		12			

misses 5, hits 0

Two-way set associative

Blk	Set	H/M	S0	S0	S1	S1	S2	S2	S3	S3
0	0	×	0							
12	0	×	0	12						
16	0	×	16	12						
18	2	×	16	12			18			
0	0	×	16	0			18			

misses 5, hits 0

Four-way set associative

Blk	Set	H/M	S0	S0	S0	S0	S1	S1	S1	S1
0	0	×	0							
12	0	×	0	12						
16	0	×	0	12	16					
18	0	×	0	12	16	18				
0	0	✓	0	12	16	18				

misses 4, hits 1

Answer. Direct-mapped: 5 misses; two-way: 5 misses (16 evicts LRU block 0); four-way: 4 misses, 1 hit (the final 0).

52. 2023-24 · Q2(c) · 10 marks · ★★★ High · Theory · L23

2023–2024 • Q2(c) CSE 209 • 29/04/2026 • MH02 **MARKS 10**

What are the advantages and disadvantages of larger cache blocks? What do you mean by early restart and critical-word-first? What are the uses of the write buffer in the write-through and write-back schemes?

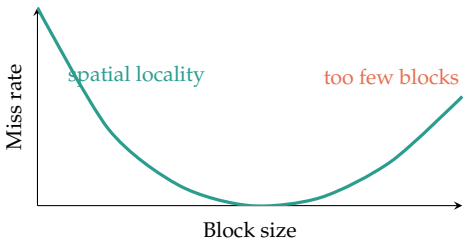
SOLUTION

Advantages of larger blocks

- **Exploit spatial locality:** one miss brings in many neighbouring words, so later accesses to them hit ⇒ lower miss rate (fewer compulsory misses).
- **Less tag overhead:** for the same data capacity there are fewer blocks, so fewer tag/valid bits per byte of data.
- **Efficient transfers:** DRAM delivers a burst of consecutive words cheaply once the row is open.

Disadvantages

- **Fewer blocks** in a fixed-size cache ⇒ more competition for entries; beyond a point the miss rate *rises* again.
- **Pollution:** words that are never used are brought in and displace useful data.
- **Larger miss penalty:** more words must be transferred on each miss; this can cancel the gain from the lower miss rate.



Addressing the disadvantages (CT-4)

- **Early restart:** resume execution as soon as the *requested* word of the block arrives, instead of waiting for the whole block.
- **Critical-word-first (requested word first):** memory sends the requested word first, then the rest of the block (wrapping around); the CPU restarts immediately.
- Reduce the transfer time: wider memory/bus, interleaved banks, DRAM burst mode.
- Keep the number of blocks reasonable: use a larger cache or higher associativity together with the larger block size.

Write buffer (2023–2024 Q2(c))

- **Write-through:** every store must also update memory. The write buffer holds data waiting to be written to memory, so the CPU continues immediately; it stalls only if the buffer is full.
- **Write-back:** when a *dirty* block is replaced it must be written back. The victim block is copied into the write buffer, so the new block can be read from memory first; the buffer writes the old block back in the background.

53. 2023-24 · Q2(a) · 10 marks · ★★★ High · Numerical · L24

2023–2024 • Q2(a) CSE 209 • 29/04/2026 • MH03 **MARKS 10**

Suppose we have a processor with a base CPI of 1.0, assuming all references hit in the primary cache, and a clock rate of 4 GHz. The main memory access time is 200 ns, including all misshandling overhead. The miss rate per instruction in the primary cache is 5%.

(i) How much faster will the processor be if we add a second-level cache with an access time of 10 ns (for both hits and misses), assuming it reduces the miss rate to main memory to 2%?

(ii) How much faster will the processor be if we further add a third-level cache with an access time of 40 ns (for both hits and misses), assuming it reduces the miss rate to main memory to 1%?

SOLUTION

Method (slides' multilevel-cache example)

Penalty in cycles = time / cycle time. Every L1 miss pays the L2 access; only references that also miss L2 (global miss rate) pay the next level: $\text{CPI} = \text{CPI}_{\text{base}} + m_{L1}t_{L2} + m_{L2,\text{global}}t_{L3} + m_{\text{to mem}}t_{\text{mem}}$; speed-up $= \text{CPI}_{\text{old}}/\text{CPI}_{\text{new}}$.

Penalties

Cycle $= 1/4\,\text{GHz} = 0.25\,\text{ns}$: memory $= 200/0.25 = 800$ cycles, L2 $= 10/0.25 = 40$ cycles, L3 $= 40/0.25 = 160$ cycles.

$$\begin{aligned} \text{L1 only:} &\quad 1 + 0.05 \times 800 = 41 \\ \text{L1+L2:} &\quad 1 + 0.05 \times 40 + 0.02 \times 800 = 1 + 2 + 16 = 19 \\ \text{L1+L2+L3:} &\quad 1 + 0.05 \times 40 + 0.02 \times 160 + 0.01 \times 800 = 1 + 2 + 3.2 + 8 = 14.2 \end{aligned}$$

Answer. (i) $41/19 = 2.16\times$ faster with L2. (ii) $19/14.2 = 1.34\times$ faster again with L3 ($41/14.2 = 2.89\times$ faster than L1 only).

54. 2022-23 · Q3(a) · 25 marks · ★★★ High · Numerical · L24

2022–2023 • Q3(a) CSE 305 • 25/04/2024 • MH16 **MARKS 25**

Suppose we have a processor with a base CPI of 2.0, assuming all references hit in the primary cache, and a clock cycle length of 0.5 ns. Assume a main memory access time of 150 ns, including all the miss handling. Suppose the miss rate per instruction at the primary cache is 5%.

(i) Calculate the total CPI for one-level of caching. How much faster the ideal CPU is?

(ii) Now assume that we add a secondary cache that has a 15 ns access time for either a hit or a miss and is large enough to reduce the miss rate to 0.5%. The miss rate for primary cache remains 5%. Calculate the total CPI for two-level of caching. [Note that this secondary cache is accessed when there is a miss in the primary cache]

(iii) How much faster will the processor be with a two-level of caching compared to the processor with just one-level of caching?

SOLUTION

Method (slides' multilevel-cache example)

Penalty in cycles = time / cycle time. Every L1 miss pays the L2 access time; only the misses to main memory (0.5% of instructions) also pay the memory time: $\text{CPI} = \text{CPI}_{\text{base}} + m_{L1}\, t_{L2} + m_{\text{to mem}}\, t_{\text{mem}}$.

Penalties

Memory $= 150/0.5 = 300$ cycles; L2 $= 15/0.5 = 30$ cycles.

(i) One level: $\text{CPI} = 2 + 0.05 \times 300 = 2 + 15 = 17$. The ideal CPU (CPI = 2) is $17/2 = 8.5\times$ faster.

(ii) Two levels: $\text{CPI} = 2 + 0.05 \times 30 + 0.005 \times 300 = 2 + 1.5 + 1.5 = 5.0$.

(iii) Speed-up $= 17/5.0 = 3.4$.

Answer. $\text{CPI}_{\text{1-level}} = 17$ (the ideal CPU is $8.5\times$ faster); $\text{CPI}_{\text{2-level}} = 5.0$; the two-level system is $3.4\times$ faster.

55. 2022-23 · Q9(a) · 15 marks · ★★★ High · Numerical · L24

2022–2023 • Q9(a) CSE 209 • 13/01/2025 • MH08 **MARKS 15**

Assume there are two processors, P1 and P2 where P1 has two levels of cache (L1 and L2) and P2 has three levels of cache (L1, L2 and L3). The DRAM memory access time is 100 ns, and memory accesses constitute 30% of all instructions. Additionally, the L2 hit time is twenty times the cycle times for both P1 and P2. The base CPI for both P1 and P2 is 1.

L1 Cache Data:

Processor	L1 Size	L1 Miss Rate	L1 Hit Time
P1	16 KB	4%	1.16 ns
P2	32 KB	3%	2.04 ns

L2 Cache Data:

Processor	L2 Size	L2 Miss Rate	L2 Hit Time
P1	16 MB	65%	27.5 ns
P2	8 MB	50%	30.5 ns

L3 Cache Data:

L3 Size	L3 Global Miss Rate	L3 Hit Time
64 MB	1%	50.7 ns

(i) Determine the faster processor between P1 and P2. Justify your answer by determining the effective CPI for each processor.

(ii) If P1 is faster, determine the miss rate P2 would need in its L2 cache to match P1's performance. If P2 is faster, determine the miss rate P1 would need in its L2 cache to match P2's performance.

SOLUTION

NOTE Assumptions: "memory accesses constitute 30% of instructions" is taken as 0.3 cache accesses per instruction that can miss (the base CPI of 1 already contains L1 hits). The L1/L2 miss rates in the tables are *local* rates; the L3 rate is stated to be *global* (per memory access).

Cycle times and penalties in cycles

L2 hit time = 20 cycles $\Rightarrow t_{c,P1} = 27.5/20 = 1.375$ ns, $t_{c,P2} = 30.5/20 = 1.525$ ns.

	P1	P2
L2 access	20 cycles	20 cycles
L3 access	–	$50.7/1.525 = 33.25$ cycles
DRAM access	$100/1.375 = 72.73$ cycles	$100/1.525 = 65.57$ cycles

(i) Effective CPI

Stall cycles per memory access:

$$\text{P1: } 0.04 \times \left(20 + 0.65 \times 72.73\right) = 0.04 \times 67.27 = 2.691$$

$$\text{P2: } 0.03 \times 20 + \underbrace{0.03 \times 0.50}_{\text{L2 global}=1.5\%} \times 33.25 + 0.01 \times 65.57$$

$$= 0.600 + 0.499 + 0.656 = 1.754$$

$$\text{CPI}_{P1} = 1 + 0.3 \times 2.691 = 1.807, \qquad \text{CPI}_{P2} = 1 + 0.3 \times 1.754 = 1.526$$

Time per instruction: P1 $= 1.807 \times 1.375 = 2.485$ ns; P2 $= 1.526 \times 1.525 = 2.328$ ns.

Answer. P2 is faster: effective CPI 1.53 vs 1.81, and 2.328 ns vs 2.485 ns per instruction ($\approx 1.07\times$ faster) even though its clock is slower.

(ii) L2 miss rate P1 needs to match P2

Require $T_{P1} = T_{P2}$:

$$1.375\left[1 + 0.3 \times 0.04\,(20 + 72.73\,m)\right] = 2.328 \;\Rightarrow\; 1 + 0.012\,(20 + 72.73\,m) = 1.693$$

$$\Rightarrow\; m = \frac{0.693/0.012 - 20}{72.73} = 0.519$$

Answer. P1's L2 (local) miss rate must drop from 65% to about **51.9%**. (If only the CPIs were matched, $1 + 0.012(20 + 72.73m) = 1.526$ gives $m \approx 32.8\%$; execution time is the proper measure because the clock periods differ.)

56. 2021-22 · Q3(a) · 10 marks · ★★★ High · Numerical · L24

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2021–2022 • Q3(a) CSE 305 • 06/04/2023 • MH19 MARKS 10

Suppose we have a processor with a base CPI of 1.0, assuming all references hit in the primary cache and a clock rate of 4 GHz. Assume a main memory access time of 200 ns, including all the miss handling. Suppose the miss rate per instruction at the primary cache is 4%. How much faster will the processor be if we add a second level cache that has a 10 ns access time for either a hit or a miss and is large enough to reduce the miss rate to main memory to 1%? How much faster will the processor be if we add a third level cache that has a 40 ns access time for either a hit or a miss and is large enough to reduce the miss rate to main memory to 0.5%?

2017–2018 • Q4(a) CSE 305 • 23/09/2018 • MH19 MARKS 10

Suppose we have a processor with a base CPI of 1.0, assuming all references hit in the primary cache and a clock rate of 4 GHz. Assume a main memory that has an access time of 200 ns, including all the miss handling. Suppose the miss rate per instruction at the primary cache is 4%. How much faster will the processor be if we add a second level cache that has a 10 ns access time for either a hit or a miss and is large enough to reduce the miss rate to main memory to 1%? How much faster will the processor be if we add a third level cache that has a 40 ns access time for either a hit or a miss and is large enough to reduce the miss rate to main memory to 0.5%?

SOLUTION

Method (slides' multilevel-cache example)

Penalty in cycles = time / cycle time. Every L1 miss pays the L2 access; only references that also

miss L2 (global miss rate) pay the next level: $\text{CPI} = \text{CPI}_{\text{base}} + m_{L1}t_{L2} + m_{L2,\text{global}}t_{L3} + m_{\text{to mem}}t_{\text{mem}}$; speed-up $= \text{CPI}_{\text{old}}/\text{CPI}_{\text{new}}$.

Penalties

Cycle $= 0.25$ ns: memory $= 200/0.25 = 800$ cycles, L2 $= 10/0.25 = 40$ cycles, L3 $= 40/0.25 = 160$ cycles.

$$\begin{aligned} \text{L1 only:} &\quad 1 + 0.04 \times 800 = 33 \\ \text{L1+L2:} &\quad 1 + 0.04 \times 40 + 0.01 \times 800 = 1 + 1.6 + 8 = 10.6 \\ \text{L1+L2+L3:} &\quad 1 + 0.04 \times 40 + 0.01 \times 160 + 0.005 \times 800 = 1 + 1.6 + 1.6 + 4 = 8.2 \end{aligned}$$

Answer. Adding L2: $33/10.6 = 3.11\times$ faster. Adding L3 as well: $10.6/8.2 = 1.29\times$ faster than L1+L2 (i.e. $33/8.2 = 4.02\times$ faster than L1 only).

57. 2020-21 · Q7 · 15 marks · ★★★ High · Numerical · L24

2020–2021 • Q7 CSE 305 • 24/09/2022 • MH24 **MARKS 15**

Suppose in a memory hierarchy there are four types of memories: Cache → RAM → SSD → HDD. Their local miss rates are 5%, 15%, 25%, and 0% respectively. Their access times are 1, 10, 100, and 1,000 cycles respectively. Now answer the following questions:

(a) What is the probability that a data will be fetched from the SSD?

(b) What is the expected time of a fetch?

(c) If the cache is separated into an I-cache and a D-cache, what will the miss rate of the I-cache be for a program that accesses data elements in 10% of its instructions if the overall miss rate remains constant (i.e., 5%) and the D-cache miss rate is 10%?

SOLUTION

A **local** miss rate is the fraction of the requests *reaching* a level that miss there. So the fraction of all requests reaching a level is the product of the local miss rates of the levels above it.



(a) Probability that the data is fetched from the SSD

The request must miss in the cache and in RAM, and then hit in the SSD:

$$P(\text{SSD supplies}) = 0.05 \times 0.15 \times (1 - 0.25) = 0.0075 \times 0.75 = 0.005625.$$

Answer. $P = 0.005625 \approx 0.56\%$. (The SSD is *accessed* on $0.05 \times 0.15 = 0.75\%$ of requests; on a quarter of those the data comes from the HDD.)

(b) Expected time of a fetch

On a miss each level is accessed in turn, so the AMAT formula is applied recursively from the bottom:

$$\begin{aligned} T_{\text{SSD}} &= 100 + 0.25 \times 1000 = 350 \text{ cycles} \\ T_{\text{RAM}} &= 10 + 0.15 \times 350 = 62.5 \text{ cycles} \\ T_{\text{fetch}} &= 1 + 0.05 \times 62.5 = 4.125 \text{ cycles} \end{aligned}$$

Check by summing over levels: $1 \cdot 1 + 0.05 \cdot 10 + 0.0075 \cdot 100 + 0.001875 \cdot 1000 = 1 + 0.5 + 0.75 + 1.875 = 4.125$.

Answer. Expected fetch time $= 4.125$ cycles.

NOTE This assumes that a miss pays the access time of every level it passes through. If instead each request pays only the access time of the level that finally supplies the data, the result is $0.95(1) + 0.0425(10) + 0.005625(100) + 0.001875(1000) = 3.8125$ cycles.

(c) I-cache miss rate after splitting

Per instruction there is 1 instruction fetch and 0.1 data access, so 1.1 cache accesses in total. To keep the overall miss rate at 5% per access:

$$\frac{1 \cdot m_I + 0.1 \cdot 0.10}{1 + 0.1} = 0.05 \;\Rightarrow\; m_I = 0.05 \times 1.1 - 0.01 = 0.045.$$

Answer. I-cache miss rate $m_I = 4.5\%$.

58. 2022-23 · Q12(d) · 10 marks · ★★★ High · Theory · L25

2022–2023 • Q12(d) CSE 209 • 13/01/2025 • MH13 **MARKS 10**

Consider Figure 12.1 and Figure 12.2, which show the comparison of Quicksort and Radix Sort in terms of instructions and clock cycles. As expected, for large arrays, Radix Sort has an algorithmic advantage over Quicksort regarding the number of operations, as shown in Figure 12.1. Figure 12.2 shows clock cycles taken instead of instructions executed. We see that the lines start on the same trajectory as in Figure 12.1, but then the Radix Sort line diverges as the data to sort increases. Explain briefly why Figure 12.2 is not following Figure 12.1.

Figure 12.1: Instructions executed: Radix Sort vs Quicksort

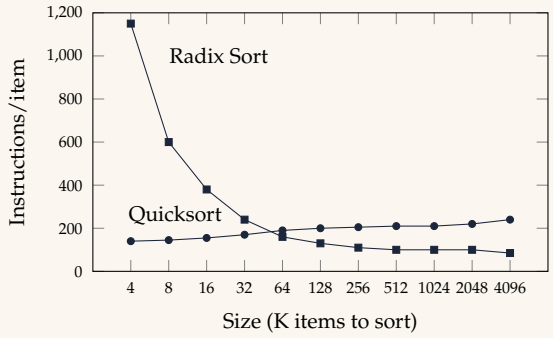
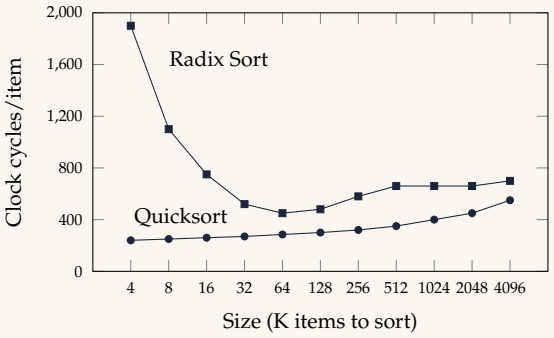


Figure 12.2: Clock Cycles: Radix Sort vs Quicksort



(plots redrawn approximately from the question paper)

SOLUTION

Reason: memory-hierarchy (cache) behaviour

- Clock cycles = instructions × CPI, and CPI = base CPI + memory-stall cycles per instruction. Figure 12.1 counts only instructions; Figure 12.2 also includes the time lost to cache misses.
- For small arrays the data fit in the cache, stalls are rare, so both figures have the same shape.

- **Radix sort has poor locality:** each pass reads the input and scatters every key into one of many buckets/counters spread over memory, so successive accesses jump to unrelated blocks. Once the array (and buckets) no longer fit in the cache, almost every access misses ⇒ cycles per item stop falling and rise (to ≈ 600–700), although instructions per item keep falling.
- **Quicksort has good locality:** partitioning sweeps sequentially through a sub-array (spatial locality) and then recursively works on ever smaller sub-arrays that fit in the cache (temporal locality), so its miss rate stays low and its cycle curve follows its instruction curve.
- Lesson (slides, "Interactions with Software"): misses depend on the memory-access pattern of the algorithm; an algorithm with fewer instructions can be slower if it ignores the memory hierarchy.

59. 2022-23 · Q10(a) · 15 marks · ★★★ High · Numerical · L26

2022–2023 • Q10(a) CSE 209 • 13/01/2025 • MH10 **MARKS 15**

Consider a memory system where the virtual address is 38 bits long, the physical DRAM installed is 8 GB, the page size is 16 KB, and the PTE size is 4 bytes. The computer system where this memory system is used has 75 processes running. The PID of each process is 8 bits long.

(i) For a single-level page table, calculate how much physical memory is needed for storing the page tables.

(ii) For a three-level page table where each level has an equal length of bits, calculate how much physical memory is needed for storing the page tables in the best case, average case, and worst case scenario.

(iii) For an inverted page table, calculate how much physical memory is needed for storing the page table. An extra 4 bytes of information besides the required ones is stored in the PTE.

SOLUTION

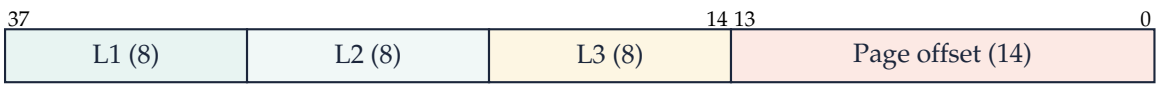
Address fields

Page offset $= \log_2 16\,\text{K} = 14$ bits ⇒ VPN $= 38 - 14 = 24$ bits. PA $= \log_2 8\,\text{GB} = 33$ bits ⇒ $2^{33}/2^{14} = 2^{19}$ physical pages. A forward page table has one PTE per *virtual* page of each process; an inverted table has one entry per *physical* page.

(i) Single level: $2^{24} \times 4\,\text{B} = 64\,\text{MB}$ per process; $75 \times 64 = 4800\,\text{MB}$.

Answer. 4800 MB ≈ 4.69 GB for the 75 page tables.

(ii) Three levels, 8 bits each:



Each table has 2^8 entries $\times 4\,\text{B} = 1\,\text{KB}$.

Case	Tables per process (L1 + L2 + L3)	Per process	75 processes
Best (one page path used)	$1+1+1=3$	3 KB	225 KB
Average (half the VA space mapped)	$1+128+32768=32{,}897$	≈ 32.1 MB	≈ 2.35 GB
Worst (whole VA space mapped)	$1+256+65536=65{,}793$	≈ 64.25 MB	≈ 4.71 GB

NOTE "Average case" is not defined in the question; it is taken here as a process that uses half of its virtual address space.

(iii) Inverted page table: one entry per physical page $= 2^{19}$ entries. Each entry must hold the VPN (24 bits) + PID (8 bits) = 4 B, plus the extra 4 B $\Rightarrow$ 8 B per entry.

Answer. $2^{19} \times 8\,\text{B} = 4\,\text{MB}$, a single table shared by all 75 processes.

60. 2022-23 · Q3(b) · 5 marks · ★★★ High · Theory · L26

2022–2023 • Q3(b) CSE 305 • 25/04/2024 • MH17 MARKS 5

What is difference between a virtual address and a physical address of instructions of a program? What is the purpose of the virtual addressing of instructions of a program?

SOLUTION

	Virtual address	Physical address
Generated by	The program / CPU (PC, load/store effective address)	The address translation hardware (TLB / page table)
Space	Private to each process, can be larger than DRAM (e.g. 2^{32} or 2^{64} bytes)	One shared space, limited to the installed DRAM
Form	Virtual page number | page offset	Physical page number | page offset (same offset)
Used by	Compiler, linker, the running program	Caches (PIPT) and main memory

Purpose of virtual addressing

- **Relocation:** a program is compiled/linked for a fixed virtual layout (code at the same virtual address every run) and can be loaded into *any* free physical pages, not necessarily contiguous.
- **Sharing and protection:** several programs share main memory, each in its own protected address space; a program cannot touch another's pages (protection bits, supervisor-only page tables).
- **Larger-than-memory programs:** main memory acts as a cache of the disk; only the pages in use need to be resident (page faults bring in the rest).

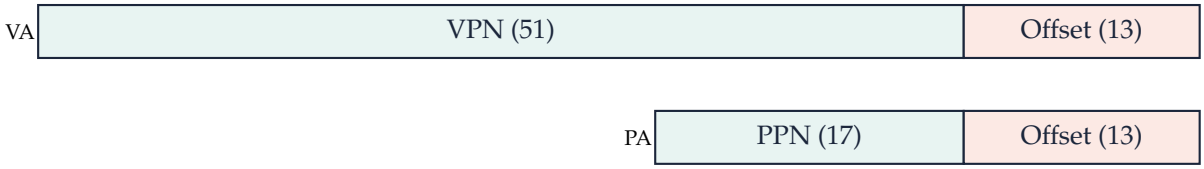
61. 2022-23 · Q3(c) · 5 marks · ★★★ High · Numerical · L26

2022–2023 • Q3(c) CSE 305 • 25/04/2024 • MH18 **MARKS 5**

Consider a mapping of a 64-bit virtual address and to a 30-bit physical address. How many entries are there in the page table if the page size is 8 KB.

SOLUTION

The page table has one entry per virtual page. Page offset $= \log_2 8\,\text{K} = 13$ bits $\Rightarrow$ VPN $= 64 - 13 = 51$ bits.



Answer. $2^{51} \approx 2.25 \times 10^{15}$ page-table entries (the 30-bit PA only fixes the PPN width, $30 - 13 = 17$ bits per entry).

62. 2021-22 · Q3(b) · 10 marks · ★★★ High · Theory · L26

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2021–2022 • Q3(b) CSE 305 • 06/04/2023 • MH20 **MARKS 10**

Explain with necessary diagrams how the page table works with respect to the virtual address, physical address, main memory, and disk.

2017–2018 • Q3(c) CSE 305 • 23/09/2018 • MH20 **MARKS 5**

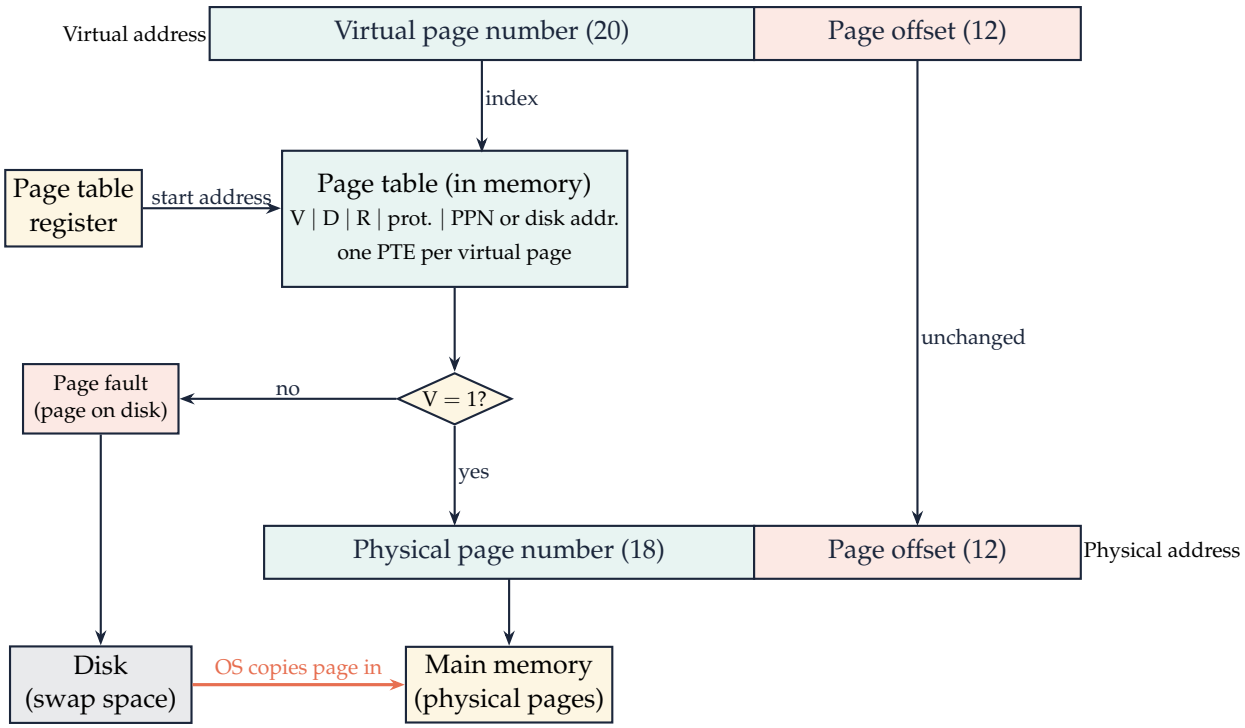
Briefly explain how page table works with respect to virtual address, physical address, main memory and disk.

2015–2016 • Q3(c) CSE 305 • 18/01/2017 • MH20 **MARKS 5**

Briefly explain how page table works with respect to virtual address, physical address, main memory and disk.

SOLUTION

Translation using the page table (32-bit VA, 4 KB pages, as in the slides)



- The **virtual address** is split into a virtual page number (VPN) and a page offset.
- The **page table register** holds the physical start address of the running process's page table; the VPN indexes it to fetch one page table entry (PTE).
- **Valid $= 1$:** the page is in **main memory**; the PTE's physical page number is concatenated with the unchanged offset to give the **physical address**, which accesses memory (via the cache). Reference/dirty bits are updated.
- **Valid $= 0$:** the page is on **disk** (the PTE or an OS structure holds its swap location) $\Rightarrow$ page fault: the OS chooses a victim page (LRU approximation; written back if dirty), reads the page from disk into a free physical page, updates the PTE (valid $= 1$, new PPN) and restarts the instruction.
- Placement is fully associative (any virtual page in any physical page), writes are always write-back, and a TLB caches recent PTEs so most translations need no extra memory access.

63. 2020-21 · Q8 · 15 marks · ★★★ High · Trace · L26

2020–2021 • Q8 CSE 305 • 24/09/2022 • MH25 **MARKS 15**

Suppose you have implemented an approximate LRU page replacement scheme using a reference bit for each page table entry. Your virtual memory has eight [pages] and page table has four entries. After every four time-steps, the reference bits are cleared. Assuming a fully set associative page table, simulate your page replacement scheme for the virtual memory reference below:

0, 0, 2, 7, 0, 6, 1, 2, 6, 7, 1, 1, 7, 1, 4, 1

You have to print the page table after each page table reference. You also have to mention whether or not it is a hit or miss.

SOLUTION

Scheme (LRU approximation with a reference bit)

- The table has 4 fully associative entries (frames 0–3). Each entry holds a virtual page and its reference bit R.
- Every reference sets $R = 1$ for its page. After time-steps 4, 8, 12 and 16 the OS clears all R bits to 0.
- **Hit**: the page is already in the table. **Miss**: the page goes into a free entry if there is one. Otherwise it replaces the lowest-numbered entry with $R = 0$, i.e. a page not used in the current interval, which is the LRU approximation.

Each cell shows $page^R$. The table is the state *after* each reference, and a shaded cell is the entry just loaded (the victim on a replacement).

Step	Ref	H/M	Entry 0	Entry 1	Entry 2	Entry 3	Action
1	0	×	0^1	–	–	–	compulsory, free entry
2	0	✓	0^1	–	–	–	
3	2	×	0^1	2^1	–	–	free entry
4	7	×	0^1	2^1	7^1	–	free entry
after step 4: all $R \leftarrow 0$ $\Rightarrow$ $0^0, 2^0, 7^0, -$							
5	0	✓	0^1	2^0	7^0	–	
6	6	×	0^1	2^0	7^0	6^1	free entry
7	1	×	0^1	1^1	7^0	6^1	replace 2 ($R = 0$)
8	2	×	0^1	1^1	2^1	6^1	replace 7 ($R = 0$)
after step 8: all $R \leftarrow 0$ $\Rightarrow$ $0^0, 1^0, 2^0, 6^0$							
9	6	✓	0^0	1^0	2^0	6^1	
10	7	×	7^1	1^0	2^0	6^1	replace 0 ($R = 0$)
11	1	✓	7^1	1^1	2^0	6^1	
12	1	✓	7^1	1^1	2^0	6^1	
after step 12: all $R \leftarrow 0$ $\Rightarrow$ $7^0, 1^0, 2^0, 6^0$							
13	7	✓	7^1	1^0	2^0	6^0	
14	1	✓	7^1	1^1	2^0	6^0	
15	4	×	7^1	1^1	4^1	6^0	replace 2 ($R = 0$)
16	1	✓	7^1	1^1	4^1	6^0	(then all $R \leftarrow 0$)

Answer. 8 hits (steps 2, 5, 9, 11, 12, 13, 14, 16) and 8 misses (steps 1, 3, 4, 6 fill free entries; steps 7, 8, 10, 15 replace a page whose reference bit is 0). Hit rate $= 8/16 = 50\%$. Final table: $\{7, 1, 4, 6\}$.

NOTE The question does not say which entry to evict when several have $R = 0$. The tie-break used here is the lowest-numbered entry. It never happens here that all four R bits are 1 on a miss; if it did, entry 0 would be evicted. At step 15 this choice evicts page 2, which is also the true-LRU victim (last used at step 8, page 6 at step 9).

64. 2023-24 · Q3(a) · 10 marks · ★★★ High · Design · L27

REPEATED ×4 Same question in 4 papers — stacked below, solved once.

2023–2024 • Q3(a) CSE 209 • 29/04/2026 • MH05 — **MARKS 10**

Although both are the TLB and data cache are types of cache memory, explain how they differ. Illustrate, with a flow diagram, the interaction between the TLB and data cache during both read and write operations.

2018–2019 • Q3(a) CSE 305 • 29/10/2019 • MH05 — **MARKS 8**

With the help of a flowchart show how a real read or write request is processed in a system with a TLB interacting with an appropriate cache and virtual memory.

2017–2018 • Q4(d) CSE 305 • 23/09/2018 • MH05 — **MARKS 5**

Explain (with flow chart) the interaction between TLB and data cache for only write operation.

2015–2016 • Q3(d) CSE 305 • 18/01/2017 • MH05 — **MARKS 10**

Though both TLB and data cache are cache memory, how are they different from each other? Explain with flow chart the interaction between TLB and data cache for both read and write operation.

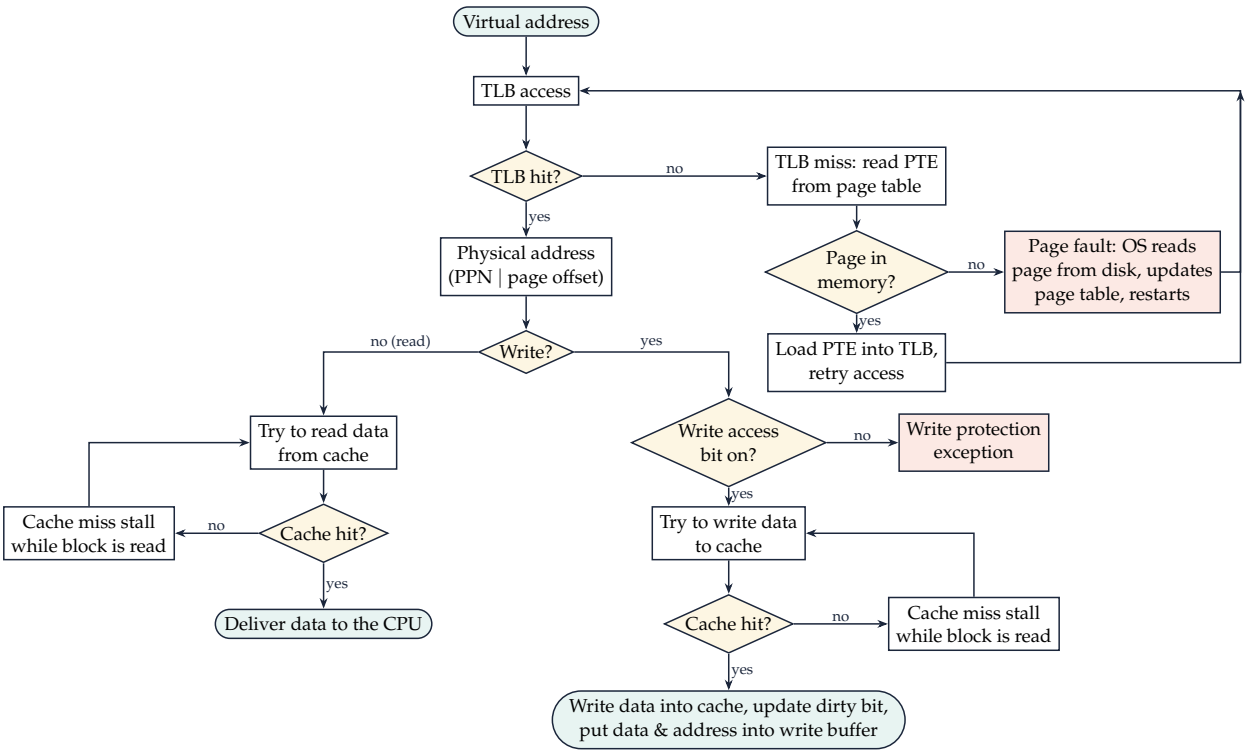
SOLUTION

TLB vs data cache (2023–2024 and 2015–2016 cards)

	TLB	Data cache
What it caches	Page-table entries: VPN → PPN translation plus valid, dirty, reference, protection bits	Copies of memory blocks (the program's data)
Looked up with	Virtual page number (tag = VPN or part of it)	Physical address (in a physically indexed/tagged cache)
Size / organisation	Very small (16–512 entries), usually fully or highly associative	Large (KB–MB), direct-mapped or set-associative
Unit	One PTE (maps a whole page, e.g. 4 KB)	One block of a few words
Miss handling	Read PTE from the page table in memory (hardware walker or OS exception); may turn into a page fault	Stall and fetch the block from L2/memory (pure hardware)
Order of use	First: produces the physical address	Second: accessed with that physical address

Flowchart of a read / write request (all four cards)

The TLB translates first; the physically addressed cache is accessed next. The right-hand branch (page table, page fault) is the virtual-memory part asked for in the 2018–2019 card; the 2017–2018 card needs only the *write* branch (right half below "Write?").



- **Read:** TLB hit gives the physical address → read the cache → on a hit deliver the word; on a miss stall, fetch the block and retry.
- **Write:** after translation the write-access (protection) bit is checked; then the cache is written. In a write-through cache the data and address also go to the write buffer; in a write-back cache only the dirty bit is set.
- **TLB miss:** the PTE is fetched from the page table; if the page is not in memory a page fault is raised and the OS brings the page from disk, then the instruction is restarted.

65. 2023-24 · Q3(b) · 15 marks · ★★★ High · Trace · L27

2023–2024 • Q3(b) CSE 209 • 29/04/2026 • MH06 **MARKS 15**

Virtual memory uses a page table to map virtual addresses to physical addresses. The following data constitutes a stream of virtual addresses as seen on a system. Assume 4 KB pages, a 4-entry fully associative TLB, and LRU replacement. If pages must be brought in from disk, increment the next largest page number (for example, the new pages will be assigned page numbers 13, 14, 15, and so on). The virtual page number is computed as the floor of the virtual address divided by the page size in bytes.

4669	2227	13916	34587	48870	12608	49225

TLB

Valid	Tag	Physical Page Number
1	11	12
1	7	4
1	3	6
0	4	9

Page Table

Valid	Physical Page or in Disk
1	5
0	Disk
0	Disk
1	6
1	9
1	11
0	Disk
1	4
0	Disk
0	Disk
1	3
1	12

Given the address stream shown and the initial TLB and page table states provided above,

(i) List for each reference if it is a hit in the TLB, a hit in the page table, or a page fault.

(ii) Show the final state of the page table and TLB.

SOLUTION

Page size $= 4096$ B $\Rightarrow$ VPN $= \lfloor \mathrm{VA}/4096 \rfloor$. The page-table rows are VPN 0–11 (top to bottom). In a fully associative TLB the tag is the whole VPN.

NOTE The recency order of the three initially valid TLB entries is not given. Assumed: the invalid entry is filled first, and the valid entries were last used in table order (tag 11 least recently used, then 7, then 3). VPN 12 lies beyond the 12 rows shown, so it is treated as not resident (page fault).

(i) Reference-by-reference

VA	VPN	Offset	Result	Action	TLB after (LRU → MRU)
4669	1	573	× TLB miss, **page fault**	page → PPN 13; fill invalid entry	11, 7, 3, **1**
2227	0	2227	× TLB miss, **PT hit** (PPN 5)	replace LRU tag 11	7, 3, 1, **0**
13916	3	1628	✓ **TLB hit** (PPN 6)	–	7, 1, 0, **3**
34587	8	1819	× TLB miss, **page fault**	page → PPN 14; replace tag 7	1, 0, 3, **8**
48870	11	3814	× TLB miss, **PT hit** (PPN 12)	replace tag 1	0, 3, 8, **11**
12608	3	320	✓ **TLB hit** (PPN 6)	–	0, 8, 11, **3**
49225	12	73	× TLB miss, **page fault**	page → PPN 15; replace tag 0	8, 11, 3, **12**

Answer. TLB hits: 13916, 12608. Page-table hits: 2227, 48870. Page faults: 4669, 34587, 49225.

(ii) Final state

TLB

Entry	Valid	Tag	PPN
0	1	**12**	**15**
1	1	**8**	**14**
2	1	3	6
3	1	**11**	**12**

Page table

VPN	Valid	PPN / Disk
0	1	5
1	**1**	**13**
2	0	Disk
3	1	6
4	1	9
5	1	11
6	0	Disk
7	1	4
8	**1**	**14**
9	0	Disk
10	1	3
11	1	12
12	**1**	**15**

(Entry 0 held tag 11, then tag 0, finally tag 12; entry 3 was the invalid entry, then tag 1, finally tag 11. Bold = changed.)

Answer. Final TLB: {V=1, tag 12, PPN 15}, {1, 8, 14}, {1, 3, 6}, {1, 11, 12}. Final page table: as initially, plus VPN 1 → 13, VPN 8 → 14, VPN 12 → 15 (all valid).

66. 2022-23 · Q10(b) · 10 marks · ★★★ High · Design · L27

2022–2023 • Q10(b) CSE 209 • 13/01/2025 • MH11 MARKS 10

Draw the block diagram of TLB and data cache interaction with necessary labels in a memory system with the following specifications.

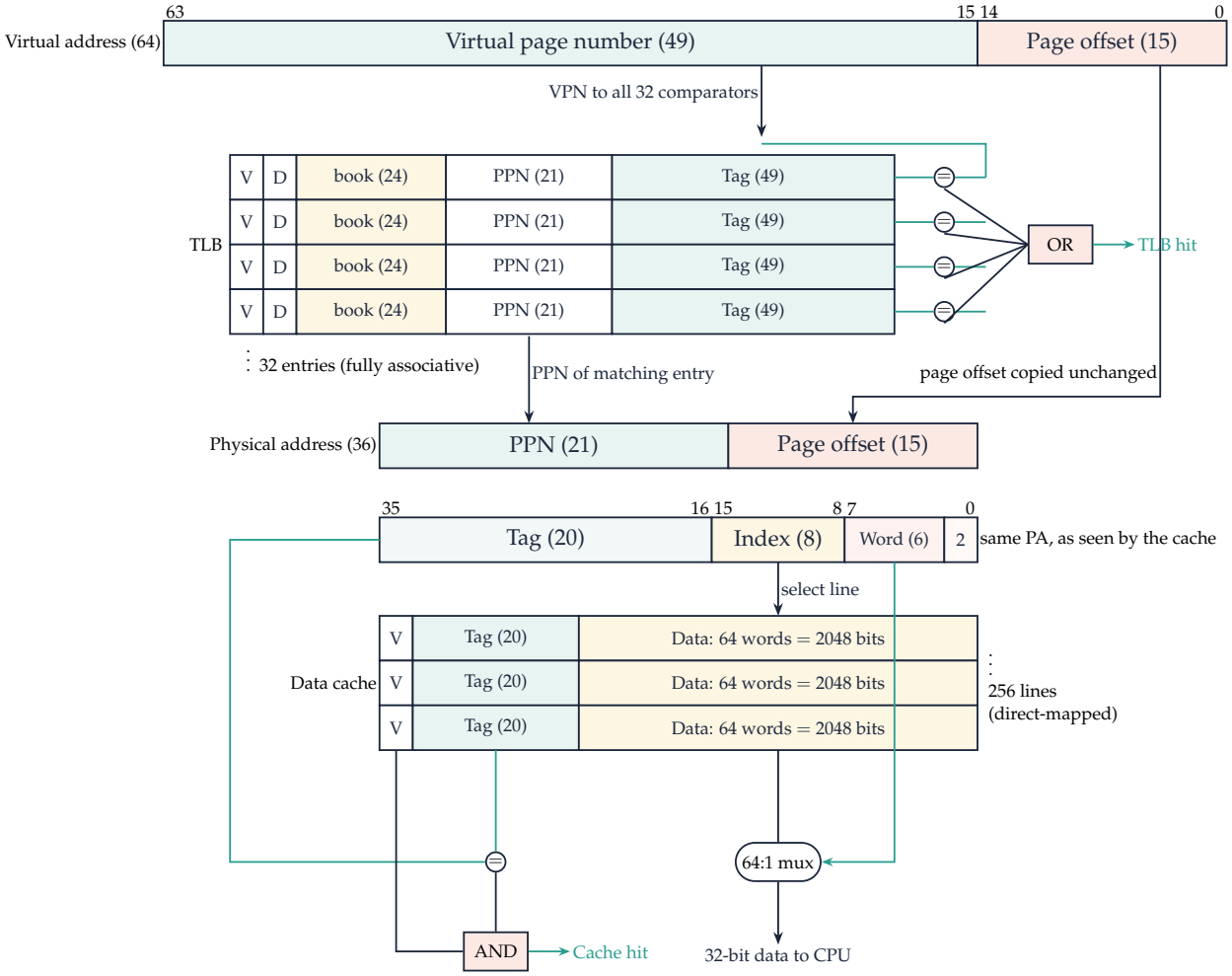
- Page size is 32 KB, and the virtual address is 64 bits long.
- The physical DRAM size is 64 GB.
- TLB cache has 32 entries and is fully associative. The cache has tag bits, a valid bit, a dirty bit, 24 bookkeeping bits, and the data bits.
- The data cache is 64 KB and direct-mapped. It has 64 words/blocks. It is a physically indexed, physically tagged cache.

SOLUTION

Field widths

- Page offset $= \log_2 32\,\text{K} = 15$ bits; VPN $= 64 - 15 = 49$ bits.
- PA $= \log_2 64\,\text{GB} = 36$ bits; PPN $= 36 - 15 = 21$ bits.
- TLB (fully associative, 32 entries): tag = whole VPN = 49 bits; each entry = 1 (V) +1 (D) +24 (bookkeeping) +49 (tag) +21 (PPN, the "data") = 96 bits $\Rightarrow 32 \times 96 = 3072$ bits; 32 parallel comparators.
- Data cache: block $= 64 \times 4 = 256$ B $\Rightarrow$ offset = 8 (6 word + 2 byte); lines $= 64\,\text{KB}/256 = 256$ $\Rightarrow$ index = 8; tag $= 36 - 8 - 8 = 20$ bits. Each line: V + 20-bit tag + 2048 data bits.

Block diagram



- The 49-bit VPN is compared simultaneously with all 32 TLB tags; on a TLB hit the matching entry supplies the 21-bit PPN (its dirty/bookkeeping bits are updated). A TLB miss is handled by reading the PTE from the page table.
- The PPN concatenated with the unchanged 15-bit page offset forms the 36-bit physical address.
- PIPT cache: PA bits 15–8 select one of 256 lines, PA bits 35–16 are compared with the stored tag (AND valid) $\Rightarrow$ cache hit; bits 7–2 drive the 64:1 multiplexer that selects the requested word.

Answer. VPN 49 / offset 15; PPN 21; TLB entry 96 bits ($32 \times 96 = 3072$ bits); cache split tag 20 | index 8 | offset 8, 256 lines of $1 + 20 + 2048$ bits (529,664 bits $\approx$ 64.7 KB).

67. 2022-23 · Q10(c) · 10 marks · ★★★ High · Theory · L27

2022–2023 • Q10(c) CSE 209 • 13/01/2025 • MH12 **MARKS 10**

Why must the operating system clear the TLB and data cache during a context switch? How does the operating system address these issues to avoid clearing the caches?

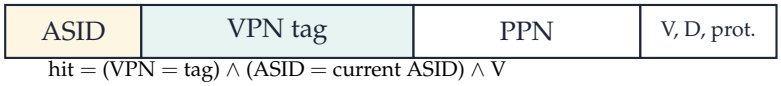
SOLUTION

Why flushing is needed

- Every process has its *own* virtual address space: the same virtual page number (e.g. VPN `0x400`) maps to different physical pages in process A and process B.
- TLB entries are tagged only with the VPN. After switching from A to B, a stale entry of A would **hit** for B's virtual address and return A's physical page ⇒ B reads/writes A's memory: wrong results and a protection violation.
- The same holds for a *virtually* addressed (virtually tagged) data cache: a block tagged with A's virtual address would hit for B's identical virtual address (homonym problem).
- Hence, without extra hardware, the OS must invalidate the TLB (and a virtual cache) on every switch — causing a burst of TLB/cache misses each time the process resumes.

How it is avoided

- **Address-space identifier (ASID / PID) in each entry:** the OS assigns each process an ASID and loads it into a CPU register on a context switch. A TLB (or virtual-cache) hit now requires *tag match AND ASID match*, so entries of several processes coexist and nothing has to be flushed.



$\text{hit} = (\text{VPN} = \text{tag}) \wedge (\text{ASID} = \text{current ASID}) \wedge \text{V}$

- **Global bit** for kernel/shared pages: such entries match for every ASID and are never flushed.
- **Physically indexed, physically tagged caches** (as in Q10(b)) need no flushing at all: physical addresses are unique across processes. Only the TLB needs the ASID.
- The OS flushes only when ASIDs run out and must be recycled, or when it changes a mapping (e.g. a page is moved to disk), in which case only the affected entries are invalidated.

68. 2021-22 · Q3(c) · 10 marks · ★★★ High · Theory · L27

2021–2022 • Q3(c) CSE 305 • 06/04/2023 • MH21 **MARKS 10**

You are given the following hit/miss status of TLB, Page Table and Cache.

Serial No.	TLB	Page Table	Cache
1	Hit	Hit	Hit
2	Hit	Miss	Hit
3	Miss	Miss	Hit
4	Miss	Miss	Miss
5	Miss	Hit	Hit

Table 3(c)

Explain which of the scenarios in Table 3(c) is possible or impossible and under what circumstance with proper reasoning.

SOLUTION

Two facts decide every row: (a) a TLB entry is only a copy of a *valid* PTE, so a TLB hit implies the page is in memory; (b) data can be in the cache only if its page is in main memory (the OS flushes a page's blocks when it moves the page to disk).

No.	TLB	PT	Cache	Verdict and reasoning
1	Hit	Hit	Hit	✓ **Possible** — the best case. (The page table is not actually consulted on a TLB hit, but its entry is valid.)
2	Hit	Miss	Hit	× **Impossible** — a translation cannot be in the TLB if the page is not present in memory.
3	Miss	Miss	Hit	× **Impossible** — data cannot be in the cache if its page is not in main memory.
4	Miss	Miss	Miss	✓ **Possible** — page fault: the OS loads the page from disk, updates the page table and TLB, the access is restarted and then misses in the cache.
5	Miss	Hit	Hit	✓ **Possible** — the PTE was evicted from the TLB (small TLB) while the page and its block are still in memory/cache; after the TLB refill, the retried access hits in the cache.

Answer. Possible: 1, 4, 5. Impossible: 2 and 3.

69. 2019-20 · Q7(b) · 5 marks · ★★★ High · Theory · L27

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2019–2020 • Q7(b) CSE 305 • 18/09/2021 • MH29 MARKS 5

The TLB acts as a cache of the page table for the entries that map to physical pages only as shown in Figure 7b. Write down the purpose of the tag bits in TLB.

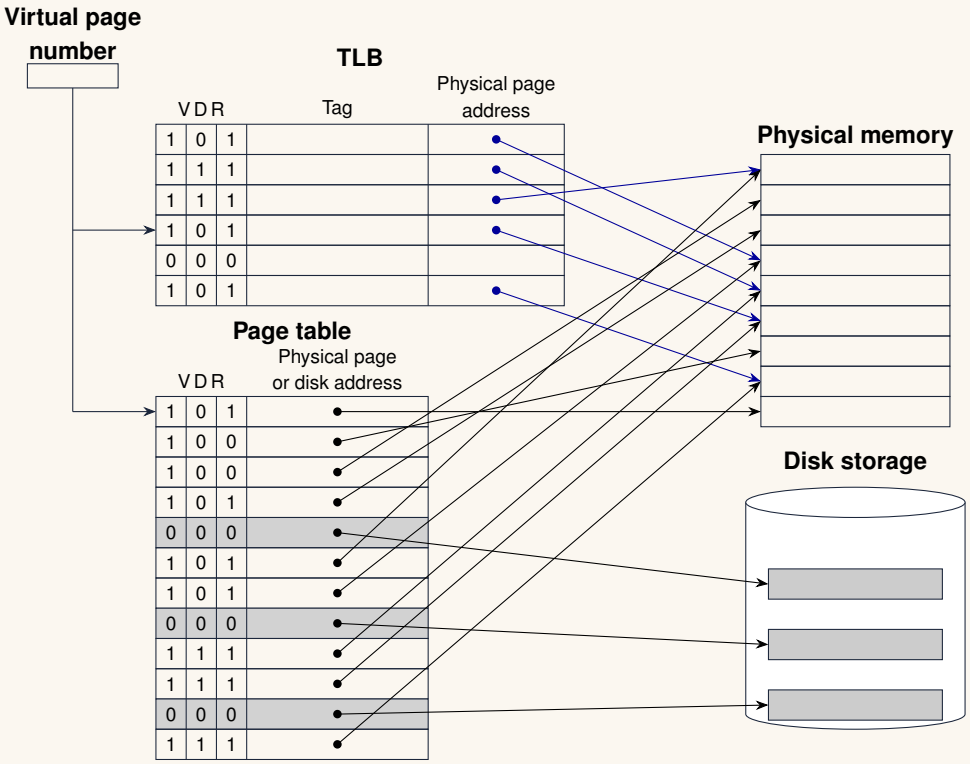


Figure 7b: TLB acts as a cache of the page table (arrows schematic, redrawn)

2012–2013 • Q6(c) CSE 305 • 10/05/2014 • MH29 MARKS 7

"The TLB acts as a cache of the page table for the entries that map to physical pages only" – Do you agree or disagree? Explain.

SOLUTION

2019–2020 Q7(b) • Purpose of the tag bits in the TLB

The page table has one entry for *every* virtual page, indexed directly by the VPN, so it needs no tag. The TLB is a small cache holding only a few of these entries, so many virtual pages compete for each TLB entry. The tag records *which* virtual page the cached translation belongs to:

- On a reference, the VPN from the virtual address is compared with the tag of every candidate entry (all entries in a fully associative TLB, as in Figure 7b; the entries of one set in a set-associative TLB).
- Tag match with $V = 1$: **TLB hit**. The physical page number is read out and joined to the page offset. No match: **TLB miss**, and the page table is consulted.
- Tag size: fully associative TLB, tag = the whole VPN. Set-associative TLB, tag = VPN bits above the set index.

Answer. The tag holds the virtual page number (or its upper part) of the cached page-table entry. It lets the hardware check whether the translation for the requested VPN is present in the TLB (hit) or not (miss).

2012–2013 Q6(c) • Agree or disagree?

Agree. In Figure 7b the TLB entries point only into physical memory. The page-table entries for non-resident pages ($V = 0$, shaded) point to disk, and none of them is cached in the TLB.

- A TLB entry stores only a *physical page number*, not a disk address. Its job is to speed up a translation that ends in main memory. A page on disk cannot be accessed by the CPU anyway; it must first be brought into memory.
- On a TLB miss the handler loads the PTE only if the page is present. If $V = 0$ in the page table, a *page fault* is raised instead. The OS reads the page from disk, updates the page table, and only then is the (now valid) translation loaded into the TLB.
- When the OS evicts a page to disk, it also invalidates any TLB entry for that page. So the TLB never holds a mapping to a non-resident page.

Answer. Agree: the TLB caches only valid page-table entries (pages resident in physical memory). Translations of pages that are on disk stay in the page table and are handled through a page fault, never through the TLB.

70. 2019-20 · Q7(c) · 10 marks · ★★★ High · Theory · L27

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2019–2020 • Q7(c) CSE 305 • 18/09/2021 • MH30 — **MARKS 10**

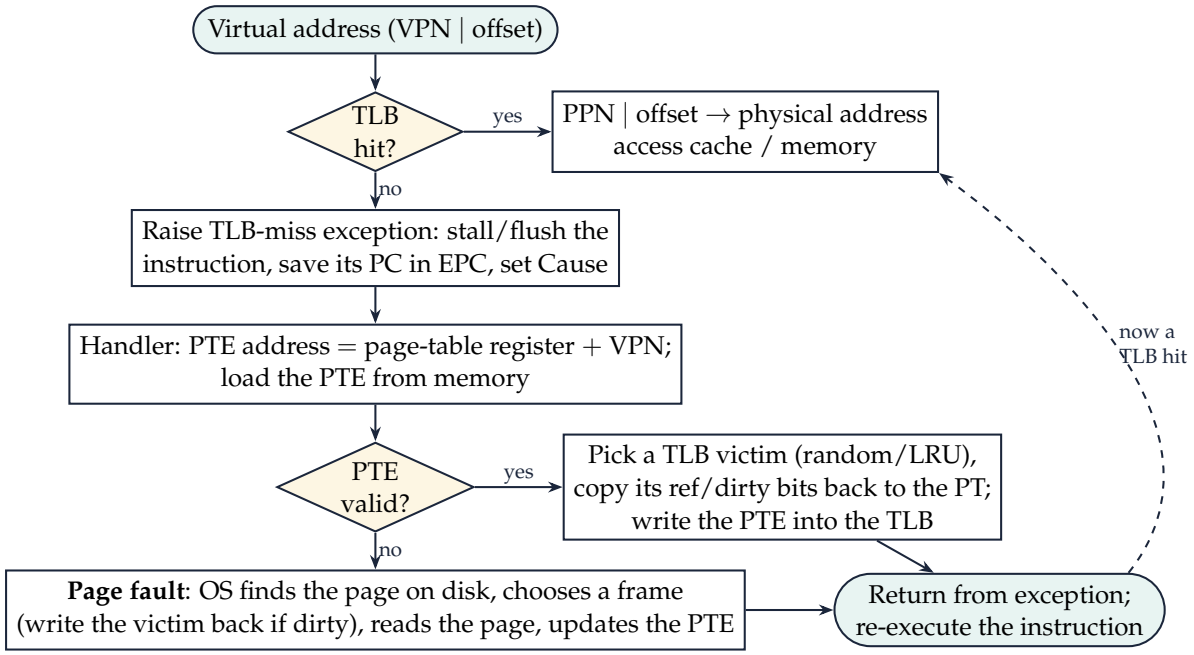
Explain how a TLB miss is handled.

2013–2014 • Q2(e) CSE 305 • 05/07/2015 • MH30 — **MARKS 10**

Write down the procedure to handle a TLB page miss.

SOLUTION

A TLB miss means one of two things: the page is in memory but its PTE is not in the TLB, or the page is not in memory at all. The handler must tell the two apart.



Procedure

(i) **Detect.** The VPN matches no valid TLB tag. The access is stopped before it can change any state. In MIPS the miss can occur in IF (instruction fetch) or MEM (load/store).

(ii) **Trap.** The miss is handled either by *hardware*, which walks the page table itself (x86), or by *software*: a special exception with a short, optimized handler (MIPS). In software, the PC of the faulting instruction goes into EPC and the faulting address into BadVAddr.

(iii) **Fetch the PTE.** The page-table register plus the VPN gives the address of the PTE, which is loaded from memory (it is often found in the cache).

(iv) **Page present ($V = 1$).** Choose a TLB entry to replace and copy its reference/dirty bits back to its PTE. Write the new VPN tag, PPN and status bits into the TLB (MIPS: `tlbwr`). This takes about 10–100 cycles.

(v) **Page not present ($V = 0$).** This is a page fault, a separate exception handled by the OS. It locates the page on disk and chooses a victim frame, writing the victim to disk first if it is dirty. It reads the page in, updates the page table and the TLB, and then makes the process runnable again. This takes millions of cycles.

(vi) **Restart.** Return from the exception and re-execute the faulting instruction, which now hits in the TLB.

Answer. On a TLB miss, read the PTE from the page table (via the page-table register and the VPN). If it is valid, load it into the TLB (replacing an entry and saving that entry's ref/dirty bits) and restart the instruction. If it is invalid, raise a page fault so the OS brings the page from disk and updates the page table/TLB, then restart the instruction.

71. 2023-24 · Q3(c) · 10 marks · ★★★ High · Theory · L28

REPEATED ×4 Same question in 4 papers — stacked below, solved once.

2023–2024 • Q3(c) CSE 209 • 29/04/2026 • MH07 — MARKS 10

What do you mean by cache coherence problem? Explain how snooping protocol solves the cache coherence problem with an example.

2021–2022 • Q3(d) CSE 305 • 06/04/2023 • MH07 — MARKS 5

Explain how snooping protocol solves the cache coherence problem with an example.

2015–2016 • Q8(a) CSE 305 • 18/01/2017 • MH07 — MARKS 10

What are the different methods to maintain coherence among all caches and global memory in a shared memory system? Describe briefly.

2013–2014 • Q1(c) CSE 305 • 05/07/2015 • MH07 — MARKS 9

State three conditions required for a cache/memory to be coherent.

SOLUTION

The cache coherence problem

When cores share one physical address space but each has a private cache, two caches can hold different values for the same address. Example (write-through caches, $X = 0$ in memory):

Step	Event	CPU A's cache	CPU B's cache	Memory
0	–			0
1	CPU A reads X	0		0
2	CPU B reads X	0	0	0
3	CPU A writes 1 to X	1	**0 (stale)**	1

After step 3, CPU B would read the old value 0 ⇒ the memory system is *incoherent*.

Conditions for coherence (2013–2014 card)

(i) **Program order:** if P writes X and then P reads X with no intervening writes by others, the read returns the value P wrote.

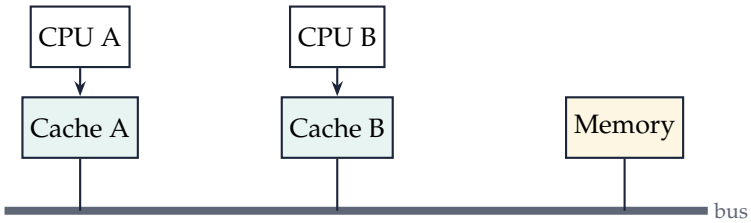
(ii) **Write propagation:** if P1 writes X and P2 reads X sufficiently later (no other writes in between), P2's read returns the value written by P1 (B reading X after step 3 must get 1).

(iii) **Write serialization:** two writes to the same location by any two processors are seen in the same order by all processors, so all end with the same final value.

Methods to maintain coherence (2015–2016 card)

- **Snooping protocols:** every cache controller monitors (snoops) the shared bus; coherence is kept by observing all bus transactions. Two variants: *write-invalidate* (a writer broadcasts an invalidate so other copies are discarded) and *write-update/broadcast* (the new value is broadcast and all copies are updated). Best for small systems with a shared bus.
- **Directory-based protocols:** a directory records, for every memory block, which caches hold a copy and in what state; a read/write consults the directory, which sends point-to-point invalidations/updates only to the sharers. Scales to large systems (no bus broadcast).
- The hardware supports *migration* (data moved to the local cache) and *replication* (read-shared copies in several caches) while keeping them coherent.

How snooping (write-invalidate) solves it — example



snoop: every controller watches every bus transaction

	CPU activity	Bus activity	A's cache	B's cache	Memory
0					0
1	CPU A reads X	Cache miss for X	0		0
2	CPU B reads X	Cache miss for X	0	0	0
3	CPU A writes 1 to X	**Invalidate for X**	1	**(invalid)**	0
4	CPU B reads X	Cache miss for X	1	1	1

- Before writing, A gets *exclusive* access: it broadcasts "invalidate X". B's controller, snooping the bus, sees the address and invalidates its copy.
- B's next read of X misses; A's cache (the owner of the dirty block) snoops this miss and supplies the up-to-date value 1, and memory is updated.

- Hence every read returns the most recently written value and writes to X are serialized on the bus.

72. 2021-22 · Q3(d) · 5 marks · ★★★ High · Theory · L28

Same question - solution already given under #71.

4 GPU + Integration — GPUs & Final Integration

Notes: CSE209_GPUs_Final_Integration_Notes.pdf

73. 2023-24 · Q4(a) · 10 marks · ★★★ High · Design · L33

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2023–2024 • Q4(a) CSE 209 • 29/04/2026 • GP01 **MARKS 10**

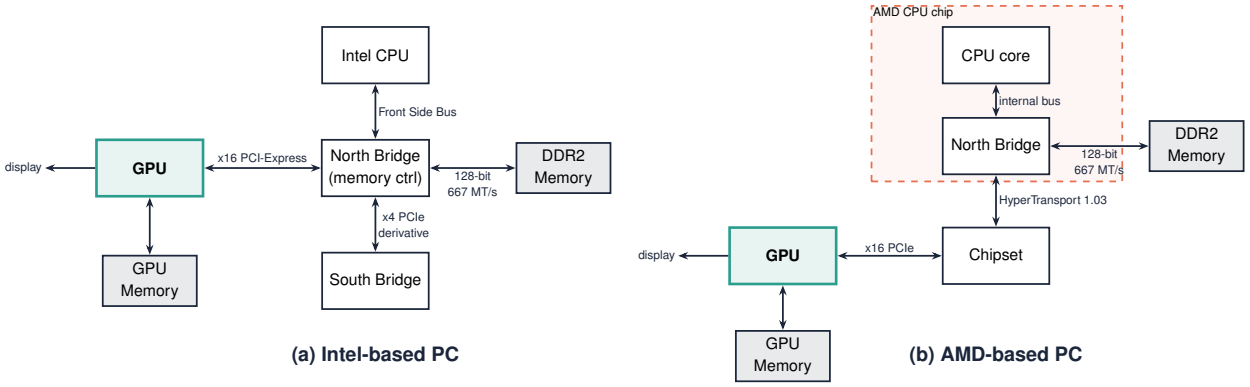
Show with block diagrams how contemporary PCs with intel and AMD CPUs connect a separate GPU. How do the CPU and GPU access each other's memory?

2021–2022 • Q4(b) CSE 305 • 06/04/2023 • GP01 **MARKS 10**

Show with block diagrams how contemporary PCs with Intel and AMD CPUs connect a separate GPU. How do the CPU and GPU access each other's memory?

SOLUTION

Block diagrams (discrete GPU)



- **Intel:** the CPU talks over the Front Side Bus to the North Bridge, which holds the memory controller for the DDR2 system memory. The GPU sits on a 16-lane *PCI-Express* link to the North Bridge (peak 16 GB/s, 8 GB/s each way) and has its own GDDR *GPU memory*. The South Bridge handles legacy I/O.
- **AMD:** the North Bridge (memory controller) is *integrated on the CPU die*, so system memory hangs directly off the CPU. The CPU reaches a separate chipset over HyperTransport, and the GPU is attached to the chipset by x16 PCI-Express, again with its own local memory.

How CPU and GPU access each other's memory

- Each processor has its own *local* memory and a separate physical address space. Either one can reach the other's memory across PCI-Express, but with much lower bandwidth and higher latency than its own local memory.
- **GPU → system memory:** the GPU issues *virtual* addresses, which an *MMU on the GPU* translates. The GPU page tables are managed by the OS kernel/driver. A system physical page is accessed

by either *coherent* or *non-coherent* PCIe transactions, as chosen by an attribute bit in the GPU page table.

- **CPU → GPU memory:** the GPU's local memory is mapped into an *aperture*, an address range in the PCI-Express address space. CPU loads and stores to that range are forwarded over PCIe to GPU memory.
- (In low-cost *UMA* systems the GPU has no dedicated memory and shares system memory, at lower performance.)

74. 2021-22 · Q4(b) · 10 marks · ★★★ High · Design · L33

Same question - solution already given under #73.

75. 2023-24 · Q4(c) · 10 marks · ★★★ High · Theory · L35

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2023–2024 • Q4(c) CSE 209 • 29/04/2026 • GP02 MARKS 10

What do you mean by kernel, grid, and thread block of the CUDA paradigm? Briefly describe how they map to the streaming processor (SP) cores and streaming multiprocessors (SM) of the unified GPU architecture of NVIDIA.

2021–2022 • Q4(c) CSE 305 • 06/04/2023 • GP02 MARKS 10

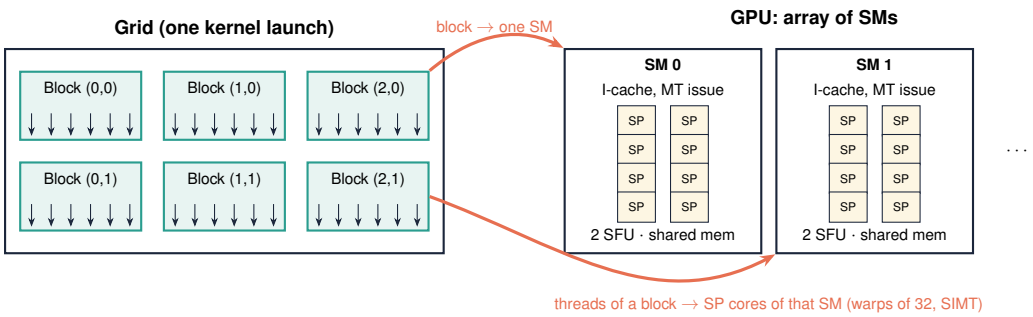
What do you mean by kernel, grid, and thread block of the CUDA paradigm? Briefly describe how they map to the streaming processor (SP) cores and streaming multiprocessors (SM) of the unified GPU architecture of NVIDIA.

SOLUTION

Definitions

- **Kernel:** a C/C++ function written for *one* sequential thread, launched to run in parallel over many threads (SPMD style). Example: `saxpy<<<nblocks, 256>>>(n, a, x, y)`, where each thread computes one $y[i]$.
- **Thread block:** a group of concurrent threads (up to 512 on Tesla) that run the same kernel. They can *cooperate* through per-block *shared memory* and *barrier synchronisation* (`__syncthreads()`). Each thread has an ID (`threadIdx`).
- **Grid:** the set of independent thread blocks (indexed by `blockIdx`) that execute one kernel launch. Blocks must be able to run in any order, in parallel or serially, which gives transparent scalability. Grids of dependent kernels are separated by an inter-kernel barrier.

Mapping onto the unified GPU (e.g. GeForce 8800 / Tesla: 14 SMs × 8 SPs = 112 SP cores)



- **Grid → whole GPU:** a kernel launch creates a grid. The GPU's work-distribution unit hands its thread blocks out to the available SMs. The number of blocks is set by the data size, not the hardware, so the same code runs on GPUs with more or fewer SMs.
- **Thread block → one SM:** each block runs entirely on a single SM, which may hold several blocks at once if registers and shared memory allow. The block's shared memory lives in that SM's on-chip shared memory, and `__syncthreads()` is an SM-local barrier.
- **Thread → SP core:** the SM's multithreaded issue unit groups the block's threads into *warps* of 32 and issues one instruction per warp (SIMT) across the 8 SP cores (plus the 2 SFUs for special functions). Each SP is hardware-multithreaded and keeps the state of many threads (up to 96), so it hides memory latency by switching warps. Per-thread local data lives in SP registers or local memory. Global memory is the DRAM shared by the whole grid.

76. 2021-22 · Q4(c) · 10 marks · ★★★ High · Theory · L35

Same question - solution already given under #75.

77. 2020-21 · Q1 · 15 marks · ★★★ High · Theory · L36

REPEATED ×4 **Same question in 4 papers — stacked below, solved once.**

2020–2021 • Q1 CSE 305 • 24/09/2022 • PF07 MARKS 15

You have surely learnt about the eight big ideas of computer architecture. They are again mentioned below to brush up your memory:

- Design for Moore's Law
- Use abstraction to simplify design
- Make the common case fast
- Performance via parallelism
- Performance via pipelining
- Performance via prediction
- Hierarchy of memories
- Dependability via redundancy

Now identify the statements below best relate to which of the above ideas. Each statement should relate to a single idea. Elaborate the justification behind your answer.

(a) Code reordering is done to avoid the use of load result in the next instruction.

(b) Different caches for data and memory can effectively increase bandwidth.

(c) As CPUs have hit their physical performance barrier due to power limit issues, GPUs have started gaining increased popularity among the programming community.

2020–2021 • Q10(a) CSE 305 • 24/09/2022 • PF07 **MARKS 12**

Briefly explain the eight great ideas in Computer Architecture.

2016–2017 • Q5(a) CSE 305 • 02/08/2017 • PF07 **MARKS 10**

"Computer architectures have invented several great ideas in the last 60 years of computer design. These ideas are so powerful that they have lasted long after the first computer that used them, with newer architects demonstrating their admiration by imitating their predecessors." These ideas include: (i) Design for Moore's Law (ii) Use abstraction to simplify design (iii) Make the common case faster (iv) Performance via parallelism (v) Performance via pipelining (vi) Performance via prediction (vii) Hierarchy of memories (viii) Dependability via redundancy. You are given several design features of MIPS and other modern computing systems in the following. Now, please **identify** that which of the above great ideas are behind these design decisions.

(i) While implementing instructions at hardware level, it is possible to implement all the instructions as single cycle instructions. However, modern designers do not use this single-cycle design as it is inefficient. (hint: for single cycle implementation, the clock cycle must have the same length for every instruction and the longest instruction determine the clock cycle length.)

(ii) Modern computers use a technique called prefetching. "In prefetching, a block of data is brought into the cache before it is actually referenced. Many microprocessors use hardware perfectching to access the block needed in the future that may be difficult for software to notice".

2014–2015 • Q5(a) CSE 305 • 26/01/2016 • PF07 **MARKS 20**

"Computer architects have invented several great ideas in the last 60 years of computer design. These ideas are so powerful that they have lasted long after the first computer that used them, with newer architects demonstrating their admiration by imitating their predecessors". These ideas include: 1. Design for Moore's Law 2. Use abstraction to simplify design 3. Make the common case faster 4. Performance via parallelism 5. Performance via pipelining 6. Performance via prediction 7. Hierarchy of memories 8. Dependability via redundancy.
You are given several design features of MIPS and other modern computing systems in the following. Now, please **identify** that which of the above great ideas are behind these design decisions.

1. In MIPS datapath, there is an adder which computes the value of **(PC+4)**, where PC means Program Counter. For branch addressing the MIPS address is actually relative to this **(PC+4)** rather than **(PC)**. This form of branch addressing is called PC-relative addressing.
2. In the past, most PCs (personal computers) and server systems used separate SRAM (static random access memory) chips for either their primary, secondary, or even tertiary caches. Today, all levels of caches are integrated onto the processor chip, so the market for separate SRAM chips has nearly evaporated.
3. A fast multiplication hardware, "unrolls the loop" by using 31 adders rather than using a

single 32-bit adder 31 times.

4. Procedures or Functions allow programmers to concentrate on just one portion of the task at a time, parameters act as an interface between the procedure and the rest of the program and data, since they can pass values and return results.

5. Modern computers use a technique called prefetching. In prefetching, a block of data is brought into the cache before it is actually referenced. Many microprocessors use hardware prefetching to access the block needed in the future that may be difficult for software to notice.

SOLUTION

The eight great ideas (answer to 2020–2021 Q10(a); used by every card)

Idea	What it means
Design for Moore's Law	Transistor counts double every 18–24 months. A design takes years, so architects target the technology that will exist when it ships, not when it starts.
Use abstraction to simplify design	Hide lower-level detail behind clean interfaces (ISA, ABI, procedures, OS) so that each level can be designed and understood separately. This raises productivity.
Make the common case fast	Optimise what happens most often; this pays off more than optimising rare cases (Amdahl's law).
Performance via parallelism	Do several operations at the same time: multiple ALUs or adders, multicore, SIMD, multiple issue.
Performance via pipelining	A special form of parallelism. Overlap the stages of successive operations like an assembly line, so throughput rises.
Performance via prediction	It can be faster on average to guess and start working than to wait, provided recovery from a wrong guess is cheap and guesses are usually right (branch prediction, prefetching).
Hierarchy of memories	Fastest, smallest, costliest memory at the top (caches), slower, larger, cheaper memory below (DRAM, disk). This gives the illusion of memory that is both fast and large.
Dependability via redundancy	Include spare or duplicate components so the system keeps working when one fails (RAID, ECC, duplicated servers).

2020–2021 Q1

(a) **Performance via pipelining.** In a pipeline, a loaded value is available only after MEM, so an instruction that uses it immediately causes a load-use hazard and a stall. Reordering independent instructions into the slot after the load keeps the pipeline full (no bubble). This problem exists only because the instructions are pipelined.

(b) **Hierarchy of memories.** Splitting the first cache level into an I-cache and a D-cache refines the top of the memory hierarchy: an instruction fetch and a data access can be served by the fast small memories in the same cycle, which doubles the bandwidth seen by the processor. (The statement means separate caches for data and *instructions*.)

(c) **Performance via parallelism.** The power wall stopped clock-rate growth, so single-core performance stalled. GPUs gain performance by running thousands of simple threads in parallel, which is why programmers moved to them.

2016–2017 Q5(a)

(i) **Performance via pipelining**, motivated by *make the common case fast*. In a single-cycle design the slowest instruction (`lw`) sets the clock period for every instruction, so common fast instructions are slowed down. Splitting execution into short equal stages and overlapping

them (pipelining) gives a short clock and a CPI near 1.

(ii) **Performance via prediction**, within the hierarchy of memories. The hardware guesses which block will be needed next and brings it into the cache early. A correct guess turns a miss into a hit; a wrong guess costs only some bandwidth.

2014–2015 Q5(a)

1. **Make the common case fast.** PC+4 has already been computed for the sequential (most common) path, and most branch targets are near the branch. Making the offset relative to PC+4 reuses that adder output and reaches nearby targets with a 16-bit offset.

2. **Design for Moore's Law.** The growing number of transistors per chip let whole cache levels move on die. Designers planned for this transistor budget, and separate SRAM chips disappeared.
3. **Performance via parallelism.** 31 adders work simultaneously (a tree of adders) instead of one adder being reused sequentially 31 times.
4. **Use abstraction to simplify design.** A procedure hides its implementation behind an interface (parameters and return value), so the programmer deals with one piece at a time.
5. **Performance via prediction.** The hardware predicts future accesses and fetches blocks before they are referenced. (The prefetched block then sits in the cache, part of the memory hierarchy.)

Answer. Q1: (a) pipelining, (b) hierarchy of memories, (c) parallelism. 2016–17: (i) pipelining (make the common case fast), (ii) prediction. 2014–15: 1 common case fast, 2 Moore's Law, 3 parallelism, 4 abstraction, 5 prediction.

78. 2020-21 · Q2 · 15 marks · ★★★ High · Theory · L36

2020–2021 • Q2 CSE 305 • 24/09/2022 • PF08 **MARKS 15**

What is your most favorite topic learnt from processor, memory hierarchy, and multiprocessors? Describe briefly.

SOLUTION

NOTE This is an open question; any well-explained topic earns credit. Below is a model answer on **pipelining** (from the processor part). A good answer names the topic, explains the core idea, gives a small example or diagram, and mentions its issues or limits.

Favourite topic: pipelining in the MIPS processor

Idea. Instruction execution is split into five stages, each taking one clock cycle: IF (fetch), ID (decode / register read), EX (ALU), MEM (data memory), WB (register write-back). Pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) separate the stages, so a new instruction can enter every cycle, like an assembly line.

	1	2	3	4	5	6	7
`lw $t0,0($s0)`	IF	ID	EX	MEM	WB		
`add $t1,$t2,$t3`		IF	ID	EX	MEM	WB	
`sub $t4,$t5,$t6`			IF	ID	EX	MEM	WB

Why I like it.

- Throughput rises without making any single instruction faster. Ideal speedup ≈ number

of stages. With stage times 200/100/200/200/100 ps, a single-cycle CPU needs 800 ps per instruction; the pipeline completes one every 200 ps, i.e. 4× for long programs.

- It shows a whole chain of design trade-offs, the **hazards**:
  - *Structural*: two stages need the same hardware. MIPS avoids this with separate I- and D-memory.
  - *Data*: an instruction needs a result that is not yet written back. Fixes: **forwarding** (EX/MEM, MEM/WB → ALU inputs), a hazard-detection unit that stalls one cycle for a load-use hazard, and compiler code scheduling.
  - *Control*: the branch outcome is not known in time. Fixes: move the branch comparison into ID, predict (static, or dynamic with 1-/2-bit predictors), or use delayed branches.

- It extends naturally to deeper pipelines and to multiple issue (superscalar/VLIW), which is the instruction-level parallelism used by modern CPUs.

Takeaway: Time $=$ IC $\times$ CPI $\times$ T_c. Pipelining cuts T_c while keeping CPI close to 1; hazards are what push CPI above 1.

Outside Notes — Questions outside the short slides (answered from the textbook)

Textbook topics: check with sir whether they are in this term's syllabus.

79. 2022-23 · Q4(c) · 7 marks · Multiprocessors / Flynn taxonomy

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2022–2023 • Q4(c) NOT IN SLIDES CSE 305 • 25/04/2024 • MP01 — MARKS 7

Briefly explain the following concepts:

(i) Shared Memory Multiprocessor (SMP)

(ii) SISD vs MIMD

2016–2017 • Q3(d) NOT IN SLIDES CSE 305 • 02/08/2017 • MP01 — MARKS 10

Describe SISD, SIMD and MISD architectures with block diagrams.

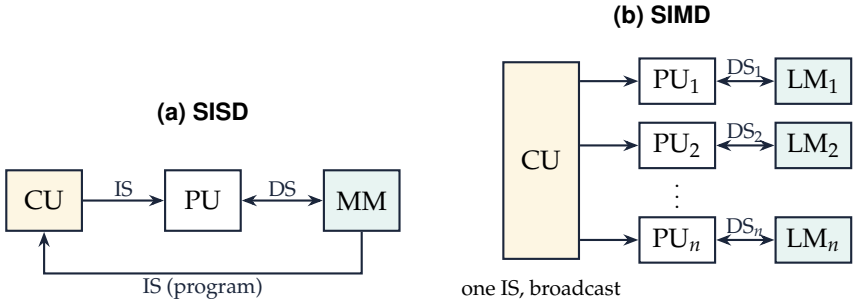
2014–2015 • Q1(a) NOT IN SLIDES CSE 305 • 26/01/2016 • MP01 — MARKS 10

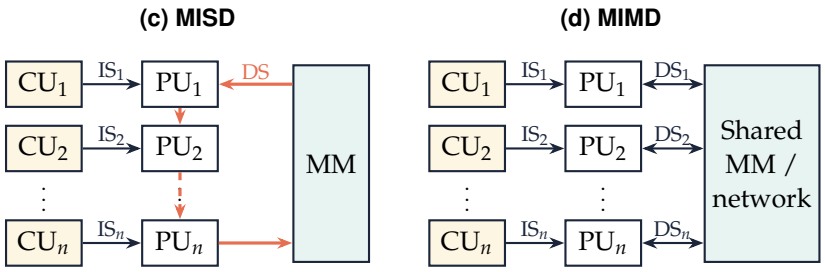
Describe SISD, SIMD, MIMD and MISD architectures with block diagrams.

SOLUTION

Flynn's taxonomy

Flynn (1966) classifies computers by how many **instruction streams** (IS) and how many **data streams** (DS) they handle at the same time. Notation in the diagrams: CU = control unit, PU = processing unit, MM = memory module, LM = local memory.

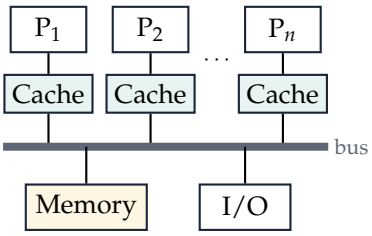




- **SISD** (single instruction, single data): one CU fetches one instruction stream and one PU operates on one data stream — the classical uniprocessor (von Neumann machine; pipelining/superscalar still count as SISD). Example: a single-core MIPS/Intel 8086.
- **SIMD** (single instruction, multiple data): one CU broadcasts each instruction to many PUs that work in lock-step, each on its own data element held in its local memory — data-level parallelism. Examples: array processors (ILLIAC IV), vector units (x86 SSE/AVX), GPU SIMD lanes.
- **MISD** (multiple instruction, single data): several CUs/PUs apply *different* instructions to the *same* data stream, which flows from PU to PU. No commercial machine fits exactly; closest are systolic arrays and redundant fault-tolerant flight computers (same data, different checks).
- **MIMD** (multiple instruction, multiple data): n independent processors, each with its own instruction stream and data stream, cooperating through shared memory or message passing — multicores, SMPs, clusters.

2022–2023 Q4(c)(i) • Shared Memory Multiprocessor (SMP)

An SMP is a MIMD machine in which all processors share **one physical address space**. Each core has private caches, and they all reach a common main memory and I/O through a bus or interconnection network. Processors communicate through ordinary `lw/sw` to shared variables. They coordinate with **locks/synchronization** (e.g. `ll/sc`), and the hardware keeps the caches coherent (snooping or a directory). It is *symmetric* because every processor has equal access to memory and I/O and any processor can run the OS. Memory access is UMA (uniform) or NUMA (non-uniform).



2022–2023 Q4(c)(ii) • SISD vs MIMD

	SISD	MIMD
Streams	1 instruction stream, 1 data stream	n instruction streams, n data streams
Hardware	one CU + one PU	n complete processors (CU+PU each)
Parallelism	only inside one stream (pipelining, ILP)	thread/process-level parallelism across processors
Communication	none needed	shared memory or message passing; needs synchronization & coherence
Example	single-core uniprocessor	multicore CPU, SMP server, cluster

2021–2022 • Q7(a) NOT IN SLIDES CSE 305 • 06/04/2023 • MP02 MARKS 5 + 5 + 5

Assume that you have four different threads as shown in figure (Figure: 7.a) below. Show how the four threads could be combined to execute on the processor more efficiently using the following three multithreading options. You only need to show the first 12 timeslots for each of the following options.

(i) A superscalar with coarse-grained multithreading

(ii) A superscalar with fine-grained multithreading

(iii) A superscalar with simultaneous multithreading

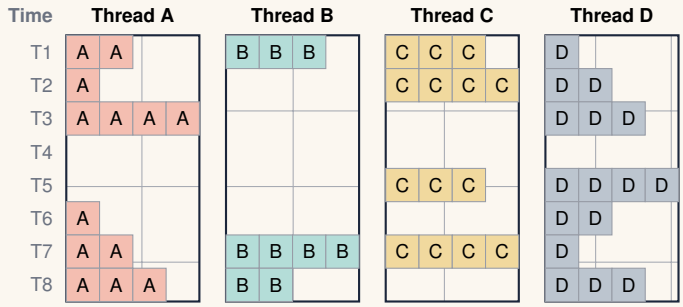


Figure 7.a

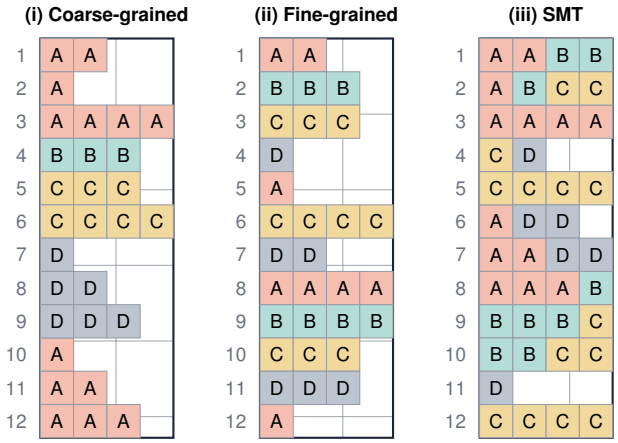
SOLUTION

How to read the figure, and the rules used

Each row is one clock cycle and the four columns are the four issue slots of a 4-wide superscalar. A thread's non-empty row is a group of independent instructions it can issue in that cycle. An **empty row is a stall**, e.g. waiting on a cache miss. Its latency is real: if a thread has g stall rows, it cannot issue its next group until $g + 1$ cycles after the previous group finished. On a multithreaded machine those cycles can be given to other threads.

- **Coarse-grained**: one thread runs alone until it reaches a stall (an empty row), then the processor switches to the next ready thread (A→B→C→D→A). The switch is assumed free. Real machines lose a few cycles refilling the pipeline. Only one thread is in the issue slots in any cycle.
- **Fine-grained**: switch thread *every cycle* in round-robin order, skipping a thread that is still stalled. One thread per cycle.
- **SMT**: in every cycle, fill the 4 slots with ready instructions from *any* threads. Fixed priority A > B > C > D is used here. A group that does not fit is split, and its remainder issues next cycle. One thread's next group must wait until its current group has fully issued.

First 12 timeslots



- *Coarse*: A issues T1–T3 and then stalls at T4 ⇒ switch to B (cycle 4). B stalls after one row ⇒ C issues T1–T2 (cycles 5–6) ⇒ D issues T1–T3 (cycles 7–9). A's stall has long since cleared, so A resumes with T6–T8 (cycles 10–12). Then B, C and D continue.
- *Fine*: A, B, C and D take turns. B is skipped from cycle 5 on because its 5-cycle stall is not over. A's 2-cycle stall is hidden by the other threads.
- *SMT*: cycle 1 = A(2)+B(2 of 3). Cycle 2 = A(1)+B(rest)+C(2 of 3). Cycle 3 = A(4). Cycle 4 = C(rest)+D(1). And so on.

Answer. Instructions issued in the first 12 cycles (48 slots): coarse-grained 29, fine-grained 31, SMT 42. All 52 instructions finish in 20, 20 and 16 cycles respectively, so SMT is the most efficient.

81. 2020-21 · Q9 · 15 marks · Multiprocessors / Amdahl's law

2020–2021 • Q9 NOT IN SLIDES CSE 305 • 24/09/2022 • MP04 MARKS 15

You want to perform two sums in a program: one is a sum of 8 scalar variables and another is a matrix sum of a pair of two-dimensional arrays of size 8 by 8. Assume you are doing these sums with 8 processors.

(a) Assume only the matrix sum is parallelizable, what speed-up will you get with 8 versus 16 processors?

(b) Again assuming only the matrix sum is parallelizable, calculate the speed-up when the matrices grow to 16 by 16.

(c) Devise a recursive algorithm to speed-up the scalar sum and calculate the new speed-up.

SOLUTION

Model (Patterson & Hennessy strong-scaling example)

Let one addition take time t. The scalar sum is counted as 8 additions (each variable added into a running sum), and it is sequential. The matrix sum $C = A + B$ is one independent addition per element, so it divides perfectly among p processors.

$$T_1 = (8 + n^2)\,t, \qquad T_p = \left(8 + \frac{n^2}{p}\right)t, \qquad \text{Speed-up} = \frac{T_1}{T_p}.$$

(a) 8×8 matrices ($n^2 = 64$)

$T_1 = 8 + 64 = 72t$. $p = 8$: $T_8 = 8 + 64/8 = 16t \Rightarrow 72/16 = 4.5$. $p = 16$: $T_{16} = 8 + 64/16 = 12t \Rightarrow 72/12 = 6.0$.

Answer. Speed-up 4.5 with 8 processors (56% of the ideal 8) and 6.0 with 16 processors (only 37.5% of the ideal 16). Doubling the processors gains just 33%, because the serial scalar sum dominates (Amdahl's law).

(b) 16×16 matrices ($n^2 = 256$)

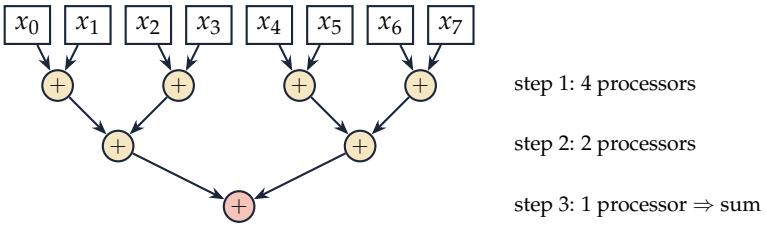
$T_1 = 8 + 256 = 264t$. $p = 8$: $T_8 = 8 + 32 = 40t \Rightarrow 264/40 = 6.6$. $p = 16$: $T_{16} = 8 + 16 = 24t \Rightarrow 264/24 = 11.0$.

Answer. Speed-up 6.6 (82.5% of ideal) with 8 processors and 11.0 (69% of ideal) with 16 processors. A bigger problem keeps the processors busier (strong scaling improves with problem size).

(c) Recursive (tree) reduction for the scalar sum

Split the set in halves, sum each half in parallel on different processors, then add the two partial sums. With s scalars this takes $\lceil \log_2 s \rceil$ addition steps instead of s.

```
int rsum(int x[], int lo, int hi) {   /* sums x[lo..hi-1] */
    if (hi - lo == 1) return x[lo];
    int mid = (lo + hi) / 2;
    int left, right;
    /* in parallel (different processors): */
    left  = rsum(x, lo, mid);
    right = rsum(x, mid, hi);
    return left + right;              /* one addition per level */
}
```



The scalar sum now takes $\log_2 8 = 3t$ instead of $8t$. The serial reference T_1 is unchanged.

	8×8 ($T_1 = 72t$)		16×16 ($T_1 = 264t$)	
	$p = 8$	$p = 16$	$p = 8$	$p = 16$
T_p (serial scalar sum)	$16t$	$12t$	$40t$	$24t$
Speed-up	4.5	6.0	6.6	11.0
T_p (tree scalar sum, $3t$)	$3 + 8 = 11t$	$3 + 4 = 7t$	$3 + 32 = 35t$	$3 + 16 = 19t$
New speed-up	**6.55**	**10.29**	7.54	13.89

Answer. With the recursive reduction, the original case (8 processors, 8×8) improves from 4.5 to $72/11 \approx 6.55$, and with 16 processors from 6.0 to $72/7 \approx 10.3$.